```
"""
RGCov estimation for high-dimensional mixed causal-noncausal VAR models
========================================================================

Implements the Generalized Covariance (GCov) estimator of Gourieroux &
Jasiak (2023) and its ridge-regularized extension (RGCov) of Giancaterini,
Hecq, Jasiak & Manafi Neyazi (2025), together with the practical fixes
needed to actually get the optimizer to converge at empirically relevant
dimensions:

  1. OLS-initialized warm start (the OLS-VAR coefficients are a
     reasonable starting point even under noncausality, since the GCov
     and OLS estimators coincide in the unconstrained causal case).
  2. Multi-start local optimization with explicit convergence
     diagnostics (objective value, gradient norm, iteration count,
     success flag) so "did not converge" is a checked fact, not an
     assumption.
  3. A diagonal-GCov fallback (Gourieroux & Jasiak 2017; Cubadda & Hecq
     2011): less efficient than full GCov/RGCov, but numerically much
     better behaved, and a legitimate estimator to report if RGCov
     itself will not certify.
  4. A ridge-parameter (delta) selection routine following the
     eigenvalue-distance-from-the-unit-circle heuristic used in
     Giancaterini et al. (2025, Section 5), formalized as a grid search
     with reported diagnostics rather than an ad hoc single choice.
  5. The residual-based portmanteau/specification test statistic
     (asymptotically chi-square under H0: iid errors, for delta -> 0).

References
----------
Gourieroux, C. & Jasiak, J. (2017). Noncausal vector autoregressive
    process. Journal of Econometrics, 200(1), 118-134.
Gourieroux, C. & Jasiak, J. (2023). Generalized Covariance Estimator.
    Journal of Business & Economic Statistics, 41(4), 1315-1327.
Giancaterini, F., Hecq, A., Jasiak, J. & Manafi Neyazi, A. (2025).
    Regularized Generalized Covariance (RGCov) Estimator.
    arXiv:2504.18678.
Cubadda, G., Giancaterini, F., Hecq, A. & Jasiak, J. (2024).
    Optimization of the generalized covariance estimator in noncausal
    processes. Statistics and Computing, 34(4), 127.

This is a research-code starting point, not a production package: for
large n * p you will want analytic gradients (see Appendix A.2.1 of
Gourieroux & Jasiak 2023) instead of the finite-difference gradient
that L-BFGS-B falls back to here.
"""

from __future__ import annotations

import numpy as np
from scipy.optimize import minimize
from scipy.stats import chi2
from dataclasses import dataclass, field
from typing import Callable, List, Tuple, Optional, Dict, Any



# ---------------------------------------------------------------------------
# 1. VAR bookkeeping: build lags, stack/unstack coefficient matrices
# ---------------------------------------------------------------------------
```

```
def build_lagged(Y: np.ndarray, p: int) -> Tuple[np.ndarray, np.ndarray]:
    """
    Y: (T, n) array of observations.
    Returns (X, Ycur) where Ycur[t] = Y_{t+p} and
    X[t] = [Y_{t+p-1}; Y_{t+p-2}; ...; Y_{t}]  (stacked lags, dimension n*p),
    for t = 0, ..., T-p-1.
    """
    T, n = Y.shape
    Ycur = Y[p:]
    X = np.column_stack([Y[p - lag - 1: T - lag - 1] for lag in range(p)])
    return X, Ycur


def stack_theta(Phi_list: List[np.ndarray]) -> np.ndarray:
    """Flatten [Phi_1, ..., Phi_p] (each n x n) into a single vector."""
    return np.concatenate([Phi.ravel() for Phi in Phi_list])


def unstack_theta(theta: np.ndarray, n: int, p: int) -> List[np.ndarray]:
    """Inverse of stack_theta."""
    return [theta[i * n * n:(i + 1) * n * n].reshape(n, n) for i in range(p)]


def var_residuals(Y: np.ndarray, p: int, Phi_list: List[np.ndarray]) -> np.ndarray:
    """Residuals u_t = Y_t - sum_j Phi_j Y_{t-j}, for t = p, ..., T-1."""
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]
    Phi_stack = np.column_stack(Phi_list)  # n x (n*p)
    fitted = X @ Phi_stack.T
    return Ycur - fitted


def ols_var(Y: np.ndarray, p: int) -> Tuple[List[np.ndarray], np.ndarray]:
    """OLS estimate of a VAR(p); used only as a warm start / comparator."""
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]
    Phi_stack, *_ = np.linalg.lstsq(X, Ycur, rcond=None)  # (n*p, n)
    Phi_stack = Phi_stack.T  # n x (n*p)
    Phi_list = [Phi_stack[:, i * n:(i + 1) * n] for i in range(p)]
    U = Ycur - X @ Phi_stack.T
    return Phi_list, U


# ---------------------------------------------------------------------------
# 2. Nonlinear transformations of residuals
# ---------------------------------------------------------------------------

def make_power_transforms(powers: Tuple[int, ...] = (1, 2, 3, 4)) -> List[Callable]:
    """
    Default transformation system a_j(u) = u^j for j in `powers`.
    Swap in |u|^p, sign(u), log|u| etc. (Giacaterini et al. 2025, Sec. 2.3)
    if your errors are heavy-tailed enough that integer power moments may
    not exist.
    """
    return [(lambda u, p=p: u ** p) for p in powers]


def transform_residuals(U: np.ndarray, transforms: List[Callable]) -> np.ndarray:
    """
    U: (T, n) residuals.
```

```
    Returns V: (T, n * J) where columns are [a_1(u_1),...,a_1(u_n),
                                             a_2(u_1),...,a_2(u_n), ...].
    """
    cols = [t(U[:, i]) for t in transforms for i in range(U.shape[1])]
    return np.column_stack(cols)


# ---------------------------------------------------------------------------
# 3. The (R)GCov objective
# ---------------------------------------------------------------------------

def sample_autocov(V: np.ndarray, h: int) -> np.ndarray:
    """Sample cross-covariance of V_t and V_{t-h}, demeaned, divided by T
    (not T-h) as recommended in Gourieroux & Jasiak (2023) to keep the
    sequence of sample autocovariances positive semi-definite."""
    Vc = V - V.mean(axis=0, keepdims=True)
    T = V.shape[0]
    if h == 0:
        return (Vc.T @ Vc) / T
    return (Vc[h:].T @ Vc[:-h]) / T


def gcov_objective(theta: np.ndarray, Y: np.ndarray, p: int, H: int,
                   transforms: List[Callable], delta: float = 0.0,
                   diagonal: bool = False) -> float:
    """
    L_T(theta, delta) = sum_{h=1}^{H} Tr[ R_hat^2(h; theta, delta) ],
    R_hat^2(h) = Gamma(h) Gamma(0,delta)^{-1} Gamma(h)' Gamma(0,delta)^{-1}.
    diagonal=True gives the diagonal-GCov variant (only the diagonal of
    Gamma(0) is used/inverted).
    """
    n = Y.shape[1]
    Phi_list = unstack_theta(theta, n, p)
    U = var_residuals(Y, p, Phi_list)
    V = transform_residuals(U, transforms)
    K = V.shape[1]

    Gamma0 = sample_autocov(V, 0)
    if diagonal:
        Gamma0_reg = np.diag(np.diag(Gamma0)) + delta * np.eye(K)
    else:
        Gamma0_reg = Gamma0 + delta * np.eye(K)

    try:
        Gamma0_inv = np.linalg.solve(Gamma0_reg, np.eye(K))
    except np.linalg.LinAlgError:
        Gamma0_inv = np.linalg.pinv(Gamma0_reg)

    total = 0.0
    for h in range(1, H + 1):
        Gh = sample_autocov(V, h)
        M = Gamma0_inv @ Gh @ Gamma0_inv @ Gh.T
        total += np.trace(M)
    return float(total)


# ---------------------------------------------------------------------------
# 4. Multi-start fitting with diagnostics
# ---------------------------------------------------------------------------

@dataclass
class FitResult:
```

```
    Phi_list: List[np.ndarray]
    theta: np.ndarray
    objective: float
    delta: float
    diagonal: bool
    diagnostics: List[Dict[str, Any]] = field(default_factory=list)


def fit_rgcov(Y: np.ndarray, p: int, H: int, transforms: List[Callable],
              delta: float = 0.0, diagonal: bool = False,
              n_starts: int = 6, perturb_scale: float = 0.05,
              seed: int = 0, maxiter: int = 1000) -> FitResult:
    """
    Fit RGCov (or plain GCov if delta=0) by multi-start L-BFGS-B, warm
    starting from OLS and from OLS + small random perturbations. Returns
    the best local optimum found, plus per-start diagnostics so you can
    see *how* well it converged rather than a single pass/fail.
    """
    n = Y.shape[1]
    Phi_ols, _ = ols_var(Y, p)
    theta_ols = stack_theta(Phi_ols)

    rng = np.random.default_rng(seed)
    starts = [theta_ols]
    scale = perturb_scale * (np.std(theta_ols) + 1e-8)
    for _ in range(n_starts - 1):
        starts.append(theta_ols + rng.normal(scale=scale, size=theta_ols.shape))

    diagnostics = []
    best_res = None
    for i, theta0 in enumerate(starts):
        res = minimize(
            gcov_objective, theta0,
            args=(Y, p, H, transforms, delta, diagonal),
            method="L-BFGS-B",
            options={"maxiter": maxiter, "ftol": 1e-12, "gtol": 1e-8},
        )
        gnorm = float(np.linalg.norm(res.jac)) if res.jac is not None else float("nan")
        diagnostics.append({
            "start": i,
            "start_type": "OLS" if i == 0 else "OLS+perturbation",
            "objective": float(res.fun),
            "success": bool(res.success),
            "n_iter": int(res.nit),
            "grad_norm": gnorm,
            "message": str(res.message),
        })
        if best_res is None or res.fun < best_res.fun:
            best_res = res

    Phi_hat = unstack_theta(best_res.x, n, p)
    return FitResult(Phi_list=Phi_hat, theta=best_res.x, objective=float(best_res.fun),
                     delta=delta, diagonal=diagonal, diagnostics=diagnostics)


# ---------------------------------------------------------------------------
# 5. Companion-matrix eigenvalues (root classification) and delta selection
# ---------------------------------------------------------------------------

def companion_eigenvalues(Phi_list: List[np.ndarray]) -> np.ndarray:
    """
```

```
    Eigenvalues of the VAR(p) companion matrix. NOTE on convention: this
    returns the roots of the *companion form* (causal <=> modulus < 1 in
    this convention), which is the mirror image of the "roots of
    det(I - Phi_1 z - ... - Phi_p z^p)" convention used in Gourieroux &
    Jasiak's noncausal-VAR papers (causal <=> modulus > 1 there). Pick
    one convention and state it; do not mix them in the same table.
    """
    n = Phi_list[0].shape[0]
    p = len(Phi_list)
    top = np.column_stack(Phi_list)  # n x (n*p)
    if p == 1:
        companion = top
    else:
        bottom = np.column_stack([np.eye(n * (p - 1)), np.zeros((n * (p - 1), n))])
        companion = np.vstack([top, bottom])
    return np.linalg.eigvals(companion)


def select_delta(Y: np.ndarray, p: int, H: int, transforms: List[Callable],
                 delta_grid: List[float], **fit_kwargs) -> Tuple[float, Dict[float, Dict]]:
    """
    Fit RGCov across a grid of delta values and report, for each: the
    objective value, the companion eigenvalues, and the distance of the
    closest eigenvalue to the unit circle (excluding near-zero
    eigenvalues, which usually signal a still-degenerate covariance
    matrix rather than a genuine root). Selects the delta that maximizes
    that distance, following the heuristic in Giancaterini et al. (2025,
    Sec. 5) -- treat this as a starting rule, not a proof of optimality;
    pair it with an out-of-sample check (e.g. holding out the last block
    of T and comparing forecast error) before finalizing delta.
    """
    results = {}
    for delta in delta_grid:
        fit = fit_rgcov(Y, p, H, transforms, delta=delta, **fit_kwargs)
        eig = companion_eigenvalues(fit.Phi_list)
        mod = np.abs(eig)
        mod_nonzero = mod[mod > 1e-6]
        dist_to_unit = float(np.min(np.abs(mod_nonzero - 1))) if len(mod_nonzero) else np.nan
        results[delta] = {
            "fit": fit,
            "eigenvalues": eig,
            "dist_to_unit": dist_to_unit,
            "objective": fit.objective,
            "any_start_converged": any(d["success"] for d in fit.diagnostics),
        }
    valid = {d: r for d, r in results.items() if not np.isnan(r["dist_to_unit"])}
    if not valid:
        raise RuntimeError("No delta in the grid produced a non-degenerate fit.")
    best_delta = max(valid, key=lambda d: valid[d]["dist_to_unit"])
    return best_delta, results


# ---------------------------------------------------------------------------
# 6. Specification / portmanteau test
# ---------------------------------------------------------------------------

def portmanteau_test(fit: FitResult, Y: np.ndarray, p: int, H: int,
                     transforms: List[Callable]) -> Dict[str, float]:
    """
    Residual-based (R)GCov portmanteau statistic. Chi-square asymptotics
    (Proposition 5 in Gourieroux & Jasiak 2023; Proposition 4 in
```

```
    Giancaterini et al. 2025) hold rigorously only as delta -> 0; for a
    fixed delta > 0 the exact asymptotic distribution is a *weighted*
    sum of chi-squares (their Proposition 3), so treat the chi-square
    p-value here as approximate when fit.delta > 0.
    """
    n = Y.shape[1]
    T_eff = Y.shape[0] - p
    K = n * len(transforms)
    dim_theta = fit.theta.shape[0]
    stat = T_eff * fit.objective
    df = K * K * H - dim_theta
    pval = float(1 - chi2.cdf(stat, df)) if df > 0 else float("nan")
    return {"statistic": float(stat), "df": int(df), "p_value": pval}


# ---------------------------------------------------------------------------
# 7. Demo: simulate a mixed causal-noncausal VAR(1) and fit it
# ---------------------------------------------------------------------------

def _simulate_mixed_var1(T: int, seed: int = 0) -> np.ndarray:
    """
    Bivariate mixed causal-noncausal VAR(1) with t(4) errors, following
    the design in Gourieroux & Jasiak (2023, Section 7.1): eigenvalues
    0.9 (causal) and 1.2 (noncausal). Simulated forward-backward via a
    long burn-in in both directions (a simple, not-optimized, way to
    approximate the strictly stationary two-sided solution).
    """
    rng = np.random.default_rng(seed)
    Phi = np.array([[0.9, -0.3], [0.0, 1.2]])
    burn = 2000
    N = T + 2 * burn
    eps = rng.standard_t(df=4, size=(N, 2))
    # Forward recursion is unstable for the noncausal root; instead use
    # the MA-infinity-style trick: simulate the causal part forward and
    # the noncausal part backward, then recombine via the Jordan blocks.
    eigvals, eigvecs = np.linalg.eig(Phi)
    A = eigvecs
    Ainv = np.linalg.inv(A)
    z = (Ainv @ eps.T).T  # decoupled scalar AR(1) innovations
    y_star = np.zeros((N, 2))
    for k in range(2):
        lam = eigvals[k].real
        if abs(lam) < 1:
            for t in range(1, N):
                y_star[t, k] = lam * y_star[t - 1, k] + z[t, k]
        else:
            for t in range(N - 2, -1, -1):
                y_star[t, k] = (y_star[t + 1, k] - z[t + 1, k]) / lam
    Y = (A @ y_star.T).T.real
    return Y[burn:burn + T]


if __name__ == "__main__":
    print("Simulating a bivariate mixed causal-noncausal VAR(1), T=400 ...")
    Y = _simulate_mixed_var1(T=400, seed=1)

    p, H = 1, 5
    transforms = make_power_transforms((1, 2))

    print("\n--- OLS (biased under noncausality; reference only) ---")
    Phi_ols, _ = ols_var(Y, p)
    print("Phi_OLS =\n", np.round(Phi_ols, 3))
```

```
    print("Phi_OLS =\n", np.round(Phi_ols[0], 3))

    print("\n--- Plain GCov (delta = 0) ---")
    fit0 = fit_rgcov(Y, p, H, transforms, delta=0.0, n_starts=6, seed=0)
    print("Phi_GCov =\n", np.round(fit0.Phi_list[0], 3))
    print("objective =", fit0.objective)
    for d in fit0.diagnostics:
        print(f"  start {d['start']:>2} ({d['start_type']:>18}): "
              f"obj={d['objective']:.6g}  success={d['success']}  "
              f"iters={d['n_iter']:>4}  |grad|={d['grad_norm']:.2e}")
    pm = portmanteau_test(fit0, Y, p, H, transforms)
    print("portmanteau:", pm)

    print("\n--- RGCov delta selection over a grid ---")
    delta_grid = [0.0, 0.05, 0.1, 0.2, 0.5, 1.0]
    best_delta, grid_results = select_delta(Y, p, H, transforms, delta_grid,
                                            n_starts=4, seed=0)
    print(f"selected delta = {best_delta}")
    for d, r in grid_results.items():
        print(f"  delta={d:>4}: obj={r['objective']:.4g}  "
              f"dist_to_unit={r['dist_to_unit']:.4f}  "
              f"any_start_converged={r['any_start_converged']}")
    best_fit = grid_results[best_delta]["fit"]
    print("\nSelected Phi_RGCov =\n", np.round(best_fit.Phi_list[0], 3))
    print("companion eigenvalues:", np.round(grid_results[best_delta]["eigenvalues"], 3))
```

```
Simulating a bivariate mixed causal-noncausal VAR(1), T=400 ...

--- OLS (biased under noncausality; reference only) ---
Phi_OLS =
 [[ 0.921 -0.3  ]
 [ 0.004  0.863]]

--- Plain GCov (delta = 0) ---
Phi_GCov =
 [[ 0.899 -0.287]
 [-0.011  1.153]]
objective = 0.18953683796325993
  start  0 (               OLS): obj=0.189537  success=True  iters=  16  |grad|=1.46e-06
  start  1 (  OLS+perturbation): obj=0.189537  success=True  iters=  18  |grad|=1.59e-08
  start  2 (  OLS+perturbation): obj=0.189537  success=True  iters=  15  |grad|=1.58e-07
  start  3 (  OLS+perturbation): obj=0.189537  success=True  iters=  17  |grad|=2.95e-08
  start  4 (  OLS+perturbation): obj=0.189537  success=True  iters=  16  |grad|=4.24e-08
  start  5 (  OLS+perturbation): obj=0.189537  success=True  iters=  15  |grad|=8.65e-08
portmanteau: {'statistic': 75.62519834734071, 'df': 76, 'p_value': 0.4905542166428285}

--- RGCov delta selection over a grid ---
selected delta = 0.5
  delta= 0.0: obj=0.1895  dist_to_unit=0.1131  any_start_converged=True
  delta=0.05: obj=0.1829  dist_to_unit=0.1153  any_start_converged=True
  delta= 0.1: obj=0.1767  dist_to_unit=0.1174  any_start_converged=True
  delta= 0.2: obj=0.1654  dist_to_unit=0.1213  any_start_converged=True
  delta= 0.5: obj=0.1387  dist_to_unit=0.1310  any_start_converged=True
  delta= 1.0: obj=0.109  dist_to_unit=0.1113  any_start_converged=True

Selected Phi_RGCov =
 [[ 0.898 -0.305]
 [-0.022  1.104]]
companion eigenvalues: [0.869 1.133]
```

Start coding or generate with AI.

```
"""
```

```
RGCov research-code framework
=============================

Includes:

1. VAR bookkeeping
2. OLS-VAR
3. Multi-task Lasso-VAR
4. GCov/RGCov-style objective
5. Diagonal-GCov
6. Multi-start optimization
7. Convergence diagnostics
8. Companion roots
9. Ljung–Box and portmanteau diagnostics
10. Causal and mixed-process simulation
11. Recursive residual bootstrap
12. Monte Carlo experiment
13. Empirical-data workflow template

Install:
    pip install numpy scipy pandas scikit-learn statsmodels

Run:
    python rgcov_all.py
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple

import numpy as np
import pandas as pd

from scipy.linalg import cho_factor, cho_solve
from scipy.optimize import minimize
from scipy.stats import chi2

from sklearn.linear_model import MultiTaskLasso
from statsmodels.stats.diagnostic import acorr_ljungbox


Array = np.ndarray
Transform = Callable[[Array], Array]


# ============================================================================
# 1. DATA UTILITIES
# ============================================================================

def validate_panel(Y: Array) -> Array:
    """
    Validate a panel with shape (T, n).
    """
    Y = np.asarray(Y, dtype=float)

    if Y.ndim != 2:
        raise ValueError("Y must be a two-dimensional array with shape (T, n).")

    if Y.shape[0] < 20:
        raise ValueError("Y must contain at least 20 observations.")
```

```
    if Y.shape[1] < 1:
        raise ValueError("Y must contain at least one series.")

    if not np.all(np.isfinite(Y)):
        raise ValueError("Y contains non-finite values.")

    return Y


def standardize_panel(Y: Array) -> Array:
    """
    Standardize each column to mean zero and variance one.
    """
    Y = validate_panel(Y)
    mean = Y.mean(axis=0, keepdims=True)
    std = Y.std(axis=0, ddof=1, keepdims=True)
    return (Y - mean) / np.maximum(std, 1e-12)


def prices_to_log_returns(prices: pd.DataFrame) -> pd.DataFrame:
    """
    Convert positive price data into log returns.
    """
    prices = prices.copy()
    prices = prices.replace([np.inf, -np.inf], np.nan)
    prices = prices.dropna(how="any")

    if (prices <= 0).any().any():
        raise ValueError("All prices must be strictly positive.")

    return np.log(prices).diff().dropna()


# ============================================================================
# 2. VAR BOOKKEEPING
# ============================================================================

def build_lagged(Y: Array, p: int) -> Tuple[Array, Array]:
    """
    Construct the VAR design matrix.

    Ycur[t] = Y[t + p]
    X[t] = [Y[t+p-1], ..., Y[t]]
    """
    Y = validate_panel(Y)
    T, n = Y.shape

    if p < 1 or p >= T:
        raise ValueError("p must satisfy 1 <= p < T.")

    Ycur = Y[p:]

    X = np.column_stack(
        [
            Y[p - lag - 1:T - lag - 1]
            for lag in range(p)
        ]
    )

    expected_shape = (T - p, n * p)
```

```
    if X.shape != expected_shape:
        raise RuntimeError(
            f"Unexpected X shape: {X.shape}; expected {expected_shape}."
        )

    return X, Ycur


def stack_theta(Phi_list: Sequence[Array]) -> Array:
    """
    Stack [Phi_1, ..., Phi_p] into a single vector.
    """
    if len(Phi_list) == 0:
        raise ValueError("Phi_list cannot be empty.")

    n = Phi_list[0].shape[0]

    for Phi in Phi_list:
        if Phi.shape != (n, n):
            raise ValueError("All coefficient matrices must be n x n.")

    return np.concatenate(
        [Phi.ravel(order="C") for Phi in Phi_list]
    )


def unstack_theta(theta: Array, n: int, p: int) -> List[Array]:
    """
    Convert a vector into [Phi_1, ..., Phi_p].
    """
    theta = np.asarray(theta, dtype=float)

    expected_size = n * n * p

    if theta.size != expected_size:
        raise ValueError(
            f"Expected {expected_size} parameters, got {theta.size}."
        )

    return [
        theta[j * n * n:(j + 1) * n * n].reshape(n, n)
        for j in range(p)
    ]


def var_residuals(
    Y: Array,
    p: int,
    Phi_list: Sequence[Array],
) -> Array:
    """
    Compute VAR residuals.
    """
    X, Ycur = build_lagged(Y, p)
    Phi_stack = np.column_stack(Phi_list)
    fitted = X @ Phi_stack.T
    return Ycur - fitted


# ============================================================================
# 3. OLS AND LASSO ESTIMATORS
```

```
# ============================================================================

@dataclass
class FitResult:
    theta: Array
    Phi_list: List[Array]
    objective: float
    delta: float = 0.0
    diagonal: bool = False
    diagnostics: List[Dict[str, Any]] = field(default_factory=list)


def fit_ols(
    Y: Array,
    p: int,
    ridge: float = 0.0,
) -> FitResult:
    """
    Fit equation-by-equation OLS-VAR.

    If ridge > 0, this is numerically stabilized ridge-OLS,
    not exact OLS.
    """
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]

    XtX = X.T @ X
    XtY = X.T @ Ycur

    if ridge > 0:
        XtX = XtX + ridge * np.eye(X.shape[1])

    B = np.linalg.solve(XtX, XtY).T

    Phi_list = [
        B[:, j * n:(j + 1) * n]
        for j in range(p)
    ]

    residuals = Ycur - X @ B.T
    mse = float(np.mean(residuals ** 2))

    return FitResult(
        theta=stack_theta(Phi_list),
        Phi_list=Phi_list,
        objective=mse,
    )


def fit_lasso(
    Y: Array,
    p: int,
    alpha: float = 0.01,
    max_iter: int = 10000,
) -> FitResult:
    """
    Fit a multi-task Lasso VAR.
    """
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]

    model = MultiTaskLasso(
```

```
    model = MultiTaskLasso(
        alpha=alpha,
        fit_intercept=False,
        max_iter=max_iter,
        tol=1e-8,
    )

    model.fit(X, Ycur)

    B = model.coef_

    Phi_list = [
        B[:, j * n:(j + 1) * n]
        for j in range(p)
    ]

    residuals = Ycur - model.predict(X)
    mse = float(np.mean(residuals ** 2))

    return FitResult(
        theta=stack_theta(Phi_list),
        Phi_list=Phi_list,
        objective=mse,
    )


# ============================================================================
# 4. TRANSFORMATIONS
# ============================================================================

def make_power_transforms(
    powers: Sequence[int] = (1, 2),
) -> List[Transform]:
    """
    Create power transformations.

    The signed-power form is more stable than raw u**p
    for odd/even powers with heavy-tailed data.
    """
    transforms = []

    for power in powers:
        transforms.append(
            lambda U, power=power:
            np.sign(U) * np.abs(U) ** power
        )

    return transforms


def make_bounded_transforms() -> List[Transform]:
    """
    Robust bounded transformations.
    """
    return [
        lambda U: U,
        lambda U: np.tanh(U),
        lambda U: np.sign(U) * np.sqrt(np.abs(U)),
        lambda U: np.log1p(U ** 2),
    ]
```

```
def transform_residuals(
    U: Array,
    transforms: Sequence[Transform],
    standardize: bool = True,
) -> Array:
    """
    Transform residuals and optionally standardize transformed columns.

    Output columns are concatenated transformation blocks.
    """
    blocks = []

    for transform in transforms:
        block = np.asarray(transform(U), dtype=float)

        if block.shape != U.shape:
            raise ValueError(
                "Each transformation must preserve the shape of U."
            )

        blocks.append(block)

    V = np.column_stack(blocks)

    if standardize:
        center = V.mean(axis=0, keepdims=True)
        scale = V.std(axis=0, ddof=1, keepdims=True)
        V = (V - center) / np.maximum(scale, 1e-10)

    return V


# ============================================================================
# 5. SAMPLE AUTOCOVARIANCES
# ============================================================================

def sample_autocov(V: Array, h: int) -> Array:
    """
    Compute the demeaned sample autocovariance matrix.

    Division is by T_eff for every h, following the
    positive-semidefinite normalization.
    """
    V = np.asarray(V, dtype=float)

    if V.ndim != 2:
        raise ValueError("V must be two-dimensional.")

    T = V.shape[0]

    if h < 0 or h >= T:
        raise ValueError("h must satisfy 0 <= h < T.")

    Vc = V - V.mean(axis=0, keepdims=True)

    if h == 0:
        return (Vc.T @ Vc) / T

    return (Vc[h:].T @ Vc[:-h]) / T
```

```
def regularize_covariance(
    Gamma0: Array,
    delta: float,
    diagonal: bool = False,
) -> Array:
    """
    Symmetrize Gamma0 and add covariance regularization.
    """
    Gamma0 = 0.5 * (Gamma0 + Gamma0.T)

    if diagonal:
        Gamma0 = np.diag(np.diag(Gamma0))

    Gamma0 = Gamma0 + delta * np.eye(Gamma0.shape[0])

    return Gamma0


# ============================================================================
# 6. GCOV / RGCOV OBJECTIVE
# ============================================================================

def rgcov_objective(
    theta: Array,
    Y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float = 0.0,
    diagonal: bool = False,
) -> float:
    """
    Full GCov/RGCov-style objective.

    Q(theta) =
        sum_h trace[
            Gamma0_delta^{-1}
            Gamma_h
            Gamma0_delta^{-1}
            Gamma_h'
        ]

    delta regularizes the transformed-residual covariance matrix.
    """
    Y = validate_panel(Y)

    if H < 1:
        raise ValueError("H must be at least one.")

    if delta < 0:
        raise ValueError("delta must be nonnegative.")

    n = Y.shape[1]
    Phi_list = unstack_theta(theta, n, p)

    U = var_residuals(Y, p, Phi_list)
    V = transform_residuals(U, transforms)

    Gamma0 = sample_autocov(V, 0)
    Gamma0 = regularize_covariance(
        Gamma0,
```

```
        delta=delta,
        diagonal=diagonal,
    )

    try:
        factor = cho_factor(
            Gamma0,
            lower=True,
            check_finite=False,
        )
    except np.linalg.LinAlgError:
        return 1e100

    objective = 0.0

    for h in range(1, H + 1):
        Gamma_h = sample_autocov(V, h)

        whitened = cho_solve(
            factor,
            Gamma_h,
            check_finite=False,
        )

        objective += np.sum(whitened ** 2)

    if not np.isfinite(objective):
        return 1e100

    return float(objective)


# ============================================================================
# 7. CONVERGENCE DIAGNOSTICS
# ============================================================================

def convergence_certificate(
    result: Any,
    gradient_tolerance: float = 1e-6,
) -> Dict[str, Any]:
    """
    Create an explicit convergence certificate.
    """
    if result.jac is None:
        gradient_inf_norm = np.inf
    else:
        gradient_inf_norm = float(
            np.linalg.norm(result.jac, ord=np.inf)
        )

    finite_solution = bool(
        np.isfinite(result.fun)
        and np.all(np.isfinite(result.x))
    )

    gradient_ok = bool(
        gradient_inf_norm <= gradient_tolerance
    )

    certified = bool(
        result.success
        and finite_solution
```

```
        and finite_solution
        and gradient_ok
    )

    return {
        "optimizer_success": bool(result.success),
        "finite_solution": finite_solution,
        "gradient_inf_norm": gradient_inf_norm,
        "gradient_ok": gradient_ok,
        "certified": certified,
        "iterations": int(result.nit),
        "function_evaluations": int(result.nfev),
        "message": str(result.message),
    }


# ============================================================================
# 8. MULTI-START RGCOV ESTIMATION
# ============================================================================

def fit_rgcov(
    Y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float = 0.0,
    diagonal: bool = False,
    n_starts: int = 8,
    perturbation_scale: float = 0.02,
    seed: int = 123,
    maxiter: int = 5000,
) -> FitResult:
    """
    Fit GCov/RGCov using OLS warm start plus perturbations.

    Note:
    The current version uses finite-difference gradients through
    scipy.optimize. For final publication results, replace this with
    analytic or automatic differentiation.
    """
    Y = validate_panel(Y)
    n = Y.shape[1]

    ols_fit = fit_ols(Y, p)
    theta_ols = ols_fit.theta

    rng = np.random.default_rng(seed)

    scale = perturbation_scale * max(
        np.std(theta_ols),
        1e-8,
    )

    starts = [theta_ols]

    for _ in range(n_starts - 1):
        starts.append(
            theta_ols
            + rng.normal(
                loc=0.0,
                scale=scale,
                size=theta_ols.shape,
```

```
            )
        )

    diagnostics = []
    best_result = None

    objective = lambda theta: rgcov_objective(
        theta=theta,
        Y=Y,
        p=p,
        H=H,
        transforms=transforms,
        delta=delta,
        diagonal=diagonal,
    )

    for start_id, theta0 in enumerate(starts):
        result = minimize(
            objective,
            theta0,
            method="L-BFGS-B",
            options={
                "maxiter": maxiter,
                "ftol": 1e-12,
                "gtol": 1e-8,
                "maxls": 50,
            },
        )

        certificate = convergence_certificate(result)

        diagnostics.append(
            {
                "start": start_id,
                "start_type": (
                    "OLS"
                    if start_id == 0
                    else "OLS_plus_perturbation"
                ),
                "objective": float(result.fun),
                **certificate,
            }
        )

        if (
            best_result is None
            or result.fun < best_result.fun
        ):
            best_result = result

    if best_result is None:
        raise RuntimeError(
            "No optimization result was produced."
        )

    Phi_hat = unstack_theta(
        best_result.x,
        n=n,
        p=p,
    )
```

```
    return FitResult(
        theta=best_result.x,
        Phi_list=Phi_hat,
        objective=float(best_result.fun),
        delta=delta,
        diagonal=diagonal,
        diagnostics=diagnostics,
    )


# ============================================================================
# 9. COMPANION MATRIX AND ROOT CLASSIFICATION
# ============================================================================

def companion_matrix(
    Phi_list: Sequence[Array],
) -> Array:
    """
    Companion matrix whose eigenvalues are companion-form eigenvalues.

    Under this convention:
        |eigenvalue| < 1: stable causal dynamics
        |eigenvalue| > 1: explosive companion dynamics
    """
    if len(Phi_list) == 0:
        raise ValueError("Phi_list cannot be empty.")

    n = Phi_list[0].shape[0]
    p = len(Phi_list)

    top = np.column_stack(Phi_list)

    if p == 1:
        return top

    bottom = np.hstack(
        [
            np.eye(n * (p - 1)),
            np.zeros((n * (p - 1), n)),
        ]
    )

    return np.vstack([top, bottom])


def companion_eigenvalues(
    Phi_list: Sequence[Array],
) -> Array:
    """
    Compute companion eigenvalues.
    """
    return np.linalg.eigvals(
        companion_matrix(Phi_list)
    )


def classify_roots(
    Phi_list: Sequence[Array],
    tolerance: float = 1e-6,
) -> Dict[str, Any]:
    """
```

```
    Classify companion eigenvalues.
    """
    eigenvalues = companion_eigenvalues(Phi_list)
    moduli = np.abs(eigenvalues)

    causal = moduli < 1.0 - tolerance
    noncausal = moduli > 1.0 + tolerance
    near_unit = ~(causal | noncausal)

    return {
        "eigenvalues": eigenvalues,
        "moduli": moduli,
        "causal_count": int(np.sum(causal)),
        "noncausal_count": int(np.sum(noncausal)),
        "near_unit_count": int(np.sum(near_unit)),
        "spectral_radius": float(np.max(moduli)),
        "minimum_distance_to_unit_circle": float(
            np.min(np.abs(moduli - 1.0))
        ),
    }


# ============================================================================
# 10. DELTA GRID SELECTION
# ============================================================================

def select_delta(
    Y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta_grid: Sequence[float],
    diagonal: bool = False,
    n_starts: int = 6,
    seed: int = 123,
    maxiter: int = 5000,
) -> Tuple[float, Dict[float, Dict[str, Any]]]:
    """
    Exploratory delta selection by distance from unit circle.

    This should not be the only selection rule in a final paper.
    Add validation-block and bootstrap-stability comparisons.
    """
    results = {}

    for delta in delta_grid:
        fit = fit_rgcov(
            Y=Y,
            p=p,
            H=H,
            transforms=transforms,
            delta=delta,
            diagonal=diagonal,
            n_starts=n_starts,
            seed=seed,
            maxiter=maxiter,
        )

        roots = classify_roots(fit.Phi_list)

        results[float(delta)] = {
            "fit": fit,
```

```
            "objective": fit.objective,
            "roots": roots,
            "distance_to_unit_circle": (
                roots["minimum_distance_to_unit_circle"]
            ),
            "any_certified_start": any(
                d["certified"]
                for d in fit.diagnostics
            ),
        }

    valid = {
        delta: result
        for delta, result in results.items()
        if np.isfinite(
            result["distance_to_unit_circle"]
        )
    }

    if not valid:
        raise RuntimeError(
            "No valid delta produced a finite solution."
        )

    selected_delta = max(
        valid,
        key=lambda d: valid[d]["distance_to_unit_circle"],
    )

    return selected_delta, results


# ============================================================================
# 11. RESIDUAL DIAGNOSTICS
# ============================================================================

def ljung_box_table(
    U: Array,
    lag: int = 10,
) -> pd.DataFrame:
    """
    Apply Ljung–Box tests separately to each residual series.
    """
    U = np.asarray(U, dtype=float)

    records = []

    for j in range(U.shape[1]):
        result = acorr_ljungbox(
            U[:, j],
            lags=[lag],
            return_df=True,
        )

        records.append(
            {
                "series": j,
                "lag": lag,
                "lb_stat": float(
                    result["lb_stat"].iloc[0]
                ),
```

```
                "p_value": float(
                    result["lb_pvalue"].iloc[0]
                ),
            }
        )

    return pd.DataFrame(records)


def portmanteau_test(
    fit: FitResult,
    Y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
) -> Dict[str, Any]:
    """
    Residual-based GCov portmanteau statistic.

    The chi-square p-value is labeled naive because fixed positive
    delta changes the limiting distribution.
    """
    U = var_residuals(
        Y=Y,
        p=p,
        Phi_list=fit.Phi_list,
    )

    V = transform_residuals(
        U,
        transforms,
        standardize=True,
    )

    Gamma0 = sample_autocov(V, 0)
    Gamma0 = 0.5 * (Gamma0 + Gamma0.T)

    try:
        factor = cho_factor(
            Gamma0,
            lower=True,
            check_finite=False,
        )
    except np.linalg.LinAlgError:
        return {
            "statistic": np.nan,
            "df_naive": np.nan,
            "p_value_naive": np.nan,
            "warning": "Gamma0 is not positive definite.",
        }

    q = 0.0

    for h in range(1, H + 1):
        Gamma_h = sample_autocov(V, h)
        whitened = cho_solve(
            factor,
            Gamma_h,
            check_finite=False,
        )
        q += np.sum(whitened ** 2)
```

```
    T_eff = len(U)
    K = V.shape[1]
    parameter_count = len(fit.theta)

    statistic = T_eff * q
    df = K * K * H - parameter_count

    if df > 0:
        p_value = float(
            1.0 - chi2.cdf(statistic, df)
        )
    else:
        p_value = np.nan

    return {
        "statistic": float(statistic),
        "df_naive": int(df),
        "p_value_naive": p_value,
        "delta": fit.delta,
        "warning": (
            "For fixed positive delta, use bootstrap or "
            "weighted-chi-square inference."
        ),
    }


# ============================================================================
# 12. SIMULATION
# ============================================================================

def simulate_var(
    T: int,
    n: int = 2,
    p: int = 1,
    Phi_list: Optional[Sequence[Array]] = None,
    innovation: str = "gaussian",
    df: int = 5,
    burn: int = 1000,
    seed: int = 123,
) -> Array:
    """
    Simulate a stable causal VAR.
    """
    rng = np.random.default_rng(seed)

    if Phi_list is None:
        Phi_list = [
            0.5 * np.eye(n)
            for _ in range(p)
        ]

    if len(Phi_list) != p:
        raise ValueError("Phi_list length must equal p.")

    total = T + burn
    Y = np.zeros((total, n))

    if innovation == "gaussian":
        E = rng.normal(size=(total, n))
    elif innovation == "student_t":
```

```
        E = rng.standard_t(df=df, size=(total, n))
    else:
        raise ValueError(
            "innovation must be 'gaussian' or 'student_t'."
        )

    for t in range(p, total):
        current = np.zeros(n)

        for j in range(p):
            current += Phi_list[j] @ Y[t - j - 1]

        Y[t] = current + E[t]

    return Y[burn:]


def simulate_mixed_diagonal(
    T: int,
    roots: Sequence[float] = (0.8, 1.2),
    df: int = 5,
    burn: int = 1000,
    seed: int = 123,
) -> Array:
    """
    Simulate a diagonal mixed causal–noncausal process.

    For |a| < 1:
        x_t = a x_{t-1} + e_t

    For |a| > 1:
        x_t = a x_{t-1} + e_t

    is generated using the backward stationary representation.
    """
    rng = np.random.default_rng(seed)

    n = len(roots)
    total = T + 2 * burn

    E = rng.standard_t(
        df=df,
        size=(total, n),
    )

    X = np.zeros((total, n))

    for j, root in enumerate(roots):
        if abs(root) < 1.0:
            for t in range(1, total):
                X[t, j] = (
                    root * X[t - 1, j]
                    + E[t, j]
                )
        else:
            for t in range(total - 2, -1, -1):
                X[t, j] = (
                    X[t + 1, j] - E[t + 1, j]
                ) / root

    return X[burn:burn + T]
```

```
# ============================================================================
# 13. RECURSIVE RESIDUAL BOOTSTRAP
# ============================================================================

def recursive_residual_bootstrap(
    Y: Array,
    Phi_list: Sequence[Array],
    residuals: Array,
    B: int = 500,
    seed: int = 123,
) -> List[Array]:
    """
    Generate bootstrap samples recursively from an estimated causal VAR.
    """
    Y = validate_panel(Y)
    residuals = np.asarray(residuals, dtype=float)

    T, n = Y.shape
    p = len(Phi_list)

    centered_residuals = (
        residuals
        - residuals.mean(axis=0, keepdims=True)
    )

    rng = np.random.default_rng(seed)
    bootstrap_samples = []

    for _ in range(B):
        innovations = centered_residuals[
            rng.integers(
                low=0,
                high=len(centered_residuals),
                size=T,
            )
        ]

        Y_boot = np.zeros_like(Y)

        for t in range(p, T):
            current = np.zeros(n)

            for j in range(p):
                current += (
                    Phi_list[j]
                    @ Y_boot[t - j - 1]
                )

            Y_boot[t] = current + innovations[t]

        bootstrap_samples.append(Y_boot)

    return bootstrap_samples


def bootstrap_spectral_radius(
    Y: Array,
    fit: FitResult,
    p: int,
    B: int = 500,
```

```
    seed: int = 123,
) -> Dict[str, Any]:
    """
    Bootstrap the spectral radius after re-estimating OLS.
    """
    U = var_residuals(
        Y=Y,
        p=p,
        Phi_list=fit.Phi_list,
    )

    samples = recursive_residual_bootstrap(
        Y=Y,
        Phi_list=fit.Phi_list,
        residuals=U,
        B=B,
        seed=seed,
    )

    radii = []
    noncausal_counts = []

    for sample in samples:
        boot_fit = fit_ols(sample, p)
        root_info = classify_roots(
            boot_fit.Phi_list
        )

        radii.append(root_info["spectral_radius"])
        noncausal_counts.append(
            root_info["noncausal_count"]
        )

    radii = np.asarray(radii)

    return {
        "spectral_radius": radii,
        "noncausal_counts": np.asarray(
            noncausal_counts
        ),
        "mean": float(np.mean(radii)),
        "std": float(np.std(radii, ddof=1)),
        "q025": float(np.quantile(radii, 0.025)),
        "q050": float(np.quantile(radii, 0.50)),
        "q975": float(np.quantile(radii, 0.975)),
        "probability_any_noncausal": float(
            np.mean(np.asarray(noncausal_counts) > 0)
        ),
    }


# ============================================================================
# 14. MONTE CARLO EXPERIMENT
# ============================================================================

def monte_carlo_experiment(
    replications: int = 100,
    T: int = 500,
    p: int = 1,
    n: int = 2,
    H: int = 3,
```

```
    delta: float = 0.1,
    seed: int = 123,
) -> pd.DataFrame:
    """
    Small demonstration Monte Carlo.

    The final paper should expand this to:
        - pure causal designs;
        - pure noncausal designs;
        - mixed-root designs;
        - near-unit-root designs;
        - multiple dimensions;
        - multiple innovation distributions;
        - GCov, RGCov, diagonal-GCov, OLS, and Lasso.
    """
    rng = np.random.default_rng(seed)
    records = []

    transforms = make_power_transforms(
        powers=(1, 2)
    )

    for r in range(replications):
        simulation_seed = int(
            rng.integers(0, 2**32 - 1)
        )

        Y = simulate_var(
            T=T,
            n=n,
            p=p,
            innovation="student_t",
            df=5,
            seed=simulation_seed,
        )

        ols_fit = fit_ols(Y, p)
        lasso_fit = fit_lasso(
            Y,
            p,
            alpha=0.01,
        )

        rgcov_fit = fit_rgcov(
            Y,
            p,
            H,
            transforms,
            delta=delta,
            n_starts=4,
            seed=simulation_seed,
            maxiter=1000,
        )

        ols_roots = classify_roots(
            ols_fit.Phi_list
        )

        lasso_roots = classify_roots(
            lasso_fit.Phi_list
        )
```

```
        rgcov_roots = classify_roots(
            rgcov_fit.Phi_list
        )

        certified = any(
            d["certified"]
            for d in rgcov_fit.diagnostics
        )

        records.extend(
            [
                {
                    "replication": r,
                    "estimator": "OLS",
                    "spectral_radius": (
                        ols_roots["spectral_radius"]
                    ),
                    "noncausal_count": (
                        ols_roots["noncausal_count"]
                    ),
                    "converged": True,
                },
                {
                    "replication": r,
                    "estimator": "Lasso",
                    "spectral_radius": (
                        lasso_roots["spectral_radius"]
                    ),
                    "noncausal_count": (
                        lasso_roots["noncausal_count"]
                    ),
                    "converged": True,
                },
                {
                    "replication": r,
                    "estimator": "RGCov",
                    "spectral_radius": (
                        rgcov_roots["spectral_radius"]
                    ),
                    "noncausal_count": (
                        rgcov_roots["noncausal_count"]
                    ),
                    "converged": certified,
                },
            ]
        )

    return pd.DataFrame(records)


# ============================================================================
# 15. REPORTING HELPERS
# ============================================================================

def print_fit_report(
    name: str,
    fit: FitResult,
) -> None:
    """
    Print a compact estimator report.
    """
```

```
    print(f"\n--- {name} ---")
    print("Objective:", fit.objective)
    print("Delta:", fit.delta)
    print("Diagonal:", fit.diagonal)

    roots = classify_roots(
        fit.Phi_list
    )

    print(
        "Spectral radius:",
        roots["spectral_radius"],
    )

    print(
        "Causal roots:",
        roots["causal_count"],
    )

    print(
        "Noncausal roots:",
        roots["noncausal_count"],
    )

    if fit.diagnostics:
        print("\nOptimization diagnostics:")

        for diagnostic in fit.diagnostics:
            print(diagnostic)


# ============================================================================
# 16. COMPLETE DEMONSTRATION
# ============================================================================

def main() -> None:
    print(
        "Simulating a bivariate mixed causal–noncausal process..."
    )

    Y_mixed = simulate_mixed_diagonal(
        T=400,
        roots=(0.9, 1.2),
        df=5,
        seed=1,
    )

    print("Mixed sample shape:", Y_mixed.shape)

    transforms = make_power_transforms(
        powers=(1, 2)
    )

    print("\nFitting OLS benchmark...")
    ols_fit = fit_ols(
        Y_mixed,
        p=1,
    )

    print_fit_report(
        "OLS",
```

```
    ols_fit,
)

print("\nFitting Lasso benchmark...")
lasso_fit = fit_lasso(
    Y_mixed,
    p=1,
    alpha=0.01,
)

print_fit_report(
    "Lasso",
    lasso_fit,
)

print("\nFitting RGCov...")
rgcov_fit = fit_rgcov(
    Y=Y_mixed,
    p=1,
    H=5,
    transforms=transforms,
    delta=0.1,
    diagonal=False,
    n_starts=4,
    seed=1,
    maxiter=1000,
)

print_fit_report(
    "RGCov",
    rgcov_fit,
)

print("\nRGCov portmanteau diagnostic...")
portmanteau = portmanteau_test(
    fit=rgcov_fit,
    Y=Y_mixed,
    p=1,
    H=5,
    transforms=transforms,
)

print(portmanteau)

print("\nOLS residual diagnostic...")
U_ols = var_residuals(
    Y_mixed,
    p=1,
    Phi_list=ols_fit.Phi_list,
)

print(
    ljung_box_table(
        U_ols,
        lag=10,
    )
)

print("\nBootstrap OLS root uncertainty...")
bootstrap = bootstrap_spectral_radius(
    Y=Y_mixed,
```

```
        fit=ols_fit,
        p=1,
        B=100,
        seed=1,
    )

    print(
        {
            key: value
            for key, value in bootstrap.items()
            if key not in {
                "spectral_radius",
                "noncausal_counts",
            }
        }
    )

    print("\nRunning small Monte Carlo...")
    mc = monte_carlo_experiment(
        replications=10,
        T=250,
        p=1,
        n=2,
        H=3,
        delta=0.1,
        seed=1,
    )

    print(
        mc.groupby("estimator")
        .agg(
            mean_spectral_radius=(
                "spectral_radius",
                "mean",
            ),
            mean_noncausal_count=(
                "noncausal_count",
                "mean",
            ),
            convergence_rate=(
                "converged",
                "mean",
            ),
        )
    )


if __name__ == "__main__":
    main()
```

```
Simulating a bivariate mixed causal–noncausal process...
Mixed sample shape: (400, 2)

Fitting OLS benchmark...

--- OLS ---
Objective: 1.5200375464013065
Delta: 0.0
Diagonal: False
Spectral radius: 0.878929993674945
Causal roots: 2
Noncausal roots: 0
```

```
Fitting Lasso benchmark...

--- Lasso ---
Objective: 1.5200589199526184
Delta: 0.0
Diagonal: False
Spectral radius: 0.8781277332546868
Causal roots: 2
Noncausal roots: 0

Fitting RGCov...

--- RGCov ---
Objective: 0.12495698697563383
Delta: 0.1
Diagonal: False
Spectral radius: 1.1956762721151553
Causal roots: 1
Noncausal roots: 1

Optimization diagnostics:
{'start': 0, 'start_type': 'OLS', 'objective': 0.12495698697563648, 'optimizer_success': True, 'fi
{'start': 1, 'start_type': 'OLS_plus_perturbation', 'objective': 0.12495698697563383, 'optimizer_s
{'start': 2, 'start_type': 'OLS_plus_perturbation', 'objective': 0.12495698697563531, 'optimizer_s
{'start': 3, 'start_type': 'OLS_plus_perturbation', 'objective': 0.12495698697563412, 'optimizer_s

RGCov portmanteau diagnostic...
{'statistic': 124.68560539993224, 'df_naive': 76, 'p_value_naive': 0.00036578216634908234, 'delta'

OLS residual diagnostic...
   series  lag    lb_stat   p_value
0       0   10  14.114356  0.167839
1       1   10   8.816387  0.549613

Bootstrap OLS root uncertainty...
{'mean': 0.8764123690327387, 'std': 0.02552272490789966, 'q025': 0.8276595586897462, 'q050': 0.875

Running small Monte Carlo...
           mean_spectral_radius  mean_noncausal_count  convergence_rate
estimator
Lasso                  0.528526                   0.0               1.0
OLS                    0.533392                   0.0               1.0
RGCov                  0.690234                   0.1               1.0
```

Start coding or generate with AI.

```
"""
RGCov / GCov Computational Framework
====================================

Single-file research prototype for:

    - OLS VAR
    - Lasso VAR
    - GCov
    - Regularized GCov (RGCov)
    - Diagonal-GCov
    - Multi-start optimization
    - Convergence diagnostics
    - Finite-difference gradient checking
    - VAR root/eigenvalue diagnostics
    - Causal VAR simulation
```

```
    - Mixed causal-noncausal simulation
    - Residual bootstrap
    - Portmanteau / serial-dependence diagnostics
    - Monte Carlo comparison

IMPORTANT METHODOLOGICAL NOTE
-----------------------------
The GCov/RGCov estimator implemented below estimates a conventional causal
VAR residual map:

    Y_t = Phi_1 Y_{t-1} + ... + Phi_p Y_{t-p} + U_t.

The function simulate_mixed_var() generates data from a genuine two-sided
mixed causal-noncausal system:

    Pi(L) Phi(L^{-1}) Y_t = epsilon_t.

Therefore, the present estimator should NOT yet be described as a complete
joint estimator of Pi(L) and Phi(L^{-1}). A joint mixed-model estimator
requires a separate parameterization of both lag and lead polynomials.

Requirements
------------
pip install numpy scipy pandas scikit-learn
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Callable, Sequence
import time

import numpy as np
import pandas as pd

from scipy.linalg import cho_factor, cho_solve
from scipy.optimize import minimize
from scipy.stats import chi2

from sklearn.linear_model import MultiTaskLasso


# ============================================================
# 1. BASIC TYPES
# ============================================================

Array = np.ndarray
Transform = Callable[[Array], Array]


# ============================================================
# 2. DATA VALIDATION AND LAG CONSTRUCTION
# ============================================================

def validate_y(y: Array) -> Array:
    """
    Validate a multivariate time-series matrix.

    Expected shape:
        (T, n)

    Where:
```

```
    where:
        T = number of observations
        n = number of variables
    """

    y = np.asarray(y, dtype=float)

    if y.ndim != 2:
        raise ValueError("y must have shape (T, n).")

    if not np.all(np.isfinite(y)):
        raise ValueError("y contains NaN or infinite values.")

    if y.shape[0] < 3:
        raise ValueError("Too few observations.")

    return y


def build_lagged(
    y: Array,
    p: int,
) -> tuple[Array, Array]:
    """
    Construct standard VAR lag matrix.

    Model:

        Y_t = Phi_1 Y_{t-1}
            + ...
            + Phi_p Y_{t-p}
            + U_t
    """

    y = validate_y(y)

    T, n = y.shape

    if p < 1 or p >= T:
        raise ValueError("p must satisfy 1 <= p < T.")

    y_current = y[p:]

    x = np.column_stack(
        [
            y[p - j - 1:T - j - 1]
            for j in range(p)
        ]
    )

    expected_shape = (T - p, n * p)

    if x.shape != expected_shape:
        raise RuntimeError(
            f"Unexpected lagged design shape. "
            f"Expected {expected_shape}, got {x.shape}."
        )

    return x, y_current


def build_mixed_design(
```

```
    y: Array,
    r: int,
    s: int,
) -> tuple[Array, Array]:
    """
    Construct a direct lead-lag design.

    This is useful as a benchmark or initializer.

    It is NOT by itself the structural mixed
    Pi(L) Phi(L^{-1}) estimator.
    """

    y = validate_y(y)

    T, n = y.shape

    if r < 0 or s < 0:
        raise ValueError("r and s must be nonnegative.")

    if r + s == 0:
        raise ValueError("At least one lag or lead is required.")

    if r + s >= T:
        raise ValueError("r + s is too large.")

    start = r
    end = T - s if s > 0 else T

    target = y[start:end]

    columns = []

    # lag terms
    for j in range(1, r + 1):
        columns.append(
            y[start - j:end - j]
        )

    # lead terms
    for j in range(1, s + 1):
        columns.append(
            y[start + j:end + j]
        )

    x = np.column_stack(columns)

    return x, target


# ============================================================
# 3. PARAMETER STACKING
# ============================================================

def stack_theta(
    phi_list: Sequence[Array],
) -> Array:
    """
    Convert coefficient matrices into one parameter vector.
    """
```

```
    if len(phi_list) == 0:
        raise ValueError("phi_list cannot be empty.")

    n = np.asarray(phi_list[0]).shape[0]

    for phi in phi_list:

        phi = np.asarray(phi)

        if phi.shape != (n, n):
            raise ValueError(
                "All coefficient matrices must have shape (n, n)."
            )

    return np.concatenate(
        [
            np.asarray(phi, dtype=float).ravel(order="C")
            for phi in phi_list
        ]
    )


def unstack_theta(
    theta: Array,
    n: int,
    p: int,
) -> list[Array]:
    """
    Convert parameter vector into p coefficient matrices.
    """

    theta = np.asarray(theta, dtype=float)

    expected = n * n * p

    if theta.size != expected:
        raise ValueError(
            f"Expected {expected} parameters, "
            f"got {theta.size}."
        )

    phi_list = []

    for j in range(p):

        start = j * n * n
        stop = (j + 1) * n * n

        phi = theta[start:stop].reshape(
            n,
            n,
            order="C",
        )

        phi_list.append(phi)

    return phi_list


# ============================================================
# 4. OLS VAR
```

```
# ============================================================

def ols_var(
    y: Array,
    p: int,
) -> tuple[list[Array], Array]:
    """
    Estimate a VAR(p) by multivariate least squares.
    """

    x, y_current = build_lagged(y, p)

    coef, *_ = np.linalg.lstsq(
        x,
        y_current,
        rcond=None,
    )

    # np.linalg.lstsq gives:
    # x @ coef ~= y
    #
    # coef shape = (np, n)
    #
    # Convert to:
    # B shape = (n, np)

    B = coef.T

    n = y.shape[1]

    phi_list = [
        B[:, j * n:(j + 1) * n]
        for j in range(p)
    ]

    residuals = (
        y_current
        -
        x @ B.T
    )

    return phi_list, residuals


# ============================================================
# 5. LASSO VAR
# ============================================================

def lasso_var(
    y: Array,
    p: int,
    alpha: float = 0.01,
    max_iter: int = 20000,
) -> tuple[list[Array], Array]:
    """
    Multi-output Lasso benchmark.
    """

    x, y_current = build_lagged(y, p)

    model = MultiTaskLasso(
        alpha=alpha,
```

```
        alpha=alpha,
        fit_intercept=False,
        max_iter=max_iter,
    )

    model.fit(
        x,
        y_current,
    )

    B = model.coef_

    n = y.shape[1]

    phi_list = [
        B[:, j * n:(j + 1) * n]
        for j in range(p)
    ]

    residuals = (
        y_current
        -
        model.predict(x)
    )

    return phi_list, residuals


# ============================================================
# 6. VAR RESIDUALS
# ============================================================

def var_residuals(
    y: Array,
    p: int,
    phi_list: Sequence[Array],
) -> Array:
    """
    Compute VAR residuals.
    """

    x, y_current = build_lagged(
        y,
        p,
    )

    phi_stack = np.column_stack(
        phi_list
    )

    fitted = (
        x @ phi_stack.T
    )

    residuals = (
        y_current
        -
        fitted
    )

    return residuals
```

```
# ============================================================
# 7. NONLINEAR TRANSFORMATIONS
# ============================================================

def make_transforms(
    powers: Sequence[int] = (1, 2),
) -> list[Transform]:
    """
    Create signed-power residual transformations.

    Examples:

        power = 1:
            u

        power = 2:
            sign(u) |u|^2

        power = 3:
            sign(u) |u|^3
    """

    transforms = []

    for power in powers:

        def transform(
            u,
            power=power,
        ):
            return (
                np.sign(u)
                *
                np.abs(u) ** power
            )

        transforms.append(transform)

    return transforms


def transform_residuals(
    residuals: Array,
    transforms: Sequence[Transform],
    standardize: bool = True,
) -> Array:
    """
    Apply transformations and concatenate them columnwise.
    """

    residuals = np.asarray(
        residuals,
        dtype=float,
    )

    columns = []

    for transform in transforms:

        transformed = np.asarray(
```

```
            transform(residuals),
            dtype=float,
        )

        if transformed.shape != residuals.shape:
            raise ValueError(
                "Each transform must preserve residual shape."
            )

        columns.append(transformed)

    v = np.column_stack(columns)

    if standardize:

        center = v.mean(
            axis=0,
            keepdims=True,
        )

        scale = v.std(
            axis=0,
            ddof=1,
            keepdims=True,
        )

        scale = np.maximum(
            scale,
            1e-10,
        )

        v = (
            v - center
        ) / scale

    return v


# ============================================================
# 8. AUTOCOVARIANCE MATRICES
# ============================================================

def sample_autocov(
    v: Array,
    h: int,
) -> Array:
    """
    Sample autocovariance:

        Gamma_h =
        1/T sum_t
        (v_t - vbar)
        (v_{t-h} - vbar)'
    """

    v = np.asarray(
        v,
        dtype=float,
    )

    if v.ndim != 2:
```

```
        raise ValueError(
            "v must be two-dimensional."
        )

    T = v.shape[0]

    if h < 0 or h >= T:
        raise ValueError(
            "Invalid autocovariance lag."
        )

    vc = (
        v
        -
        v.mean(
            axis=0,
            keepdims=True,
        )
    )

    if h == 0:

        return (
            vc.T @ vc
        ) / T

    return (
        vc[h:].T
        @
        vc[:-h]
    ) / T


# ============================================================
# 9. COVARIANCE REGULARIZATION
# ============================================================

def regularize_covariance(
    gamma0: Array,
    delta: float,
) -> Array:
    """
    Ridge regularization:

        Gamma_0,delta =
        Gamma_0 + delta I
    """

    if delta < 0:
        raise ValueError(
            "delta must be nonnegative."
        )

    gamma0 = np.asarray(
        gamma0,
        dtype=float,
    )

    gamma0 = 0.5 * (
        gamma0
        +
        gamma0.T
```

```
    )

    return (
        gamma0
        +
        delta
        *
        np.eye(
            gamma0.shape[0]
        )
    )


def diagonal_covariance(
    gamma0: Array,
    delta: float = 0.0,
) -> Array:
    """
    Diagonal-GCov covariance approximation.
    """

    diagonal = (
        np.diag(gamma0).copy()
        +
        delta
    )

    diagonal = np.maximum(
        diagonal,
        1e-12,
    )

    return np.diag(
        diagonal
    )


# ============================================================
# 10. GCOV TRACE TERM
# ============================================================

def gcov_trace_term(
    gamma_h: Array,
    gamma0_factor,
) -> float:
    """
    Compute

        tr(
            Gamma_h Gamma_0^{-1}
            Gamma_h' Gamma_0^{-1}
        )

    using Cholesky-based linear solves.

    IMPORTANT:
    This is NOT generally equal to

        || Gamma_0^{-1} Gamma_h ||_F^2

    unless additional commutation/symmetry conditions hold.
```

```
    """

    dimension = gamma_h.shape[0]

    identity = np.eye(
        dimension
    )

    gamma0_inverse = cho_solve(
        gamma0_factor,
        identity,
        check_finite=False,
    )

    value = np.trace(
        gamma_h
        @ gamma0_inverse
        @ gamma_h.T
        @ gamma0_inverse
    )

    return float(value)


# ============================================================
# 11. GCOV / RGCOV OBJECTIVE
# ============================================================

def gcov_objective(
    theta: Array,
    y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float = 0.0,
    diagonal: bool = False,
) -> float:
    """
    GCov/RGCov objective:

        Q(theta)
        =
        sum_{h=1}^H
        tr[
            Gamma_h
            Gamma_0^{-1}
            Gamma_h'
            Gamma_0^{-1}
        ]

    RGCov uses:

        Gamma_0 + delta I

    Diagonal-GCov uses only diagonal elements
    of Gamma_0.
    """

    y = validate_y(y)

    n = y.shape[1]
```

```
    phi_list = unstack_theta(
        theta,
        n,
        p,
    )

    residuals = var_residuals(
        y,
        p,
        phi_list,
    )

    v = transform_residuals(
        residuals,
        transforms,
        standardize=True,
    )

    if H >= v.shape[0]:
        return 1e100

    gamma0 = sample_autocov(
        v,
        0,
    )

    if diagonal:

        gamma0 = diagonal_covariance(
            gamma0,
            delta,
        )

    else:

        gamma0 = regularize_covariance(
            gamma0,
            delta,
        )

    try:

        factor = cho_factor(
            gamma0,
            lower=True,
            check_finite=False,
        )

    except np.linalg.LinAlgError:

        return 1e100

    objective = 0.0

    for h in range(
        1,
        H + 1,
    ):

        gamma_h = sample_autocov(
```

```
            v,
            h,
        )

        objective += gcov_trace_term(
            gamma_h,
            factor,
        )

    if not np.isfinite(
        objective
    ):
        return 1e100

    return float(objective)


# ============================================================
# 12. LITERAL TRACE OBJECTIVE
# ============================================================

def literal_trace_objective(
    theta: Array,
    y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float = 0.0,
) -> float:
    """
    Slow/reference implementation.

    Useful for unit testing the optimized objective.
    """

    n = y.shape[1]

    phi_list = unstack_theta(
        theta,
        n,
        p,
    )

    residuals = var_residuals(
        y,
        p,
        phi_list,
    )

    v = transform_residuals(
        residuals,
        transforms,
    )

    gamma0 = regularize_covariance(
        sample_autocov(v, 0),
        delta,
    )

    gamma0_inv = np.linalg.inv(
        gamma0
    )
```

```
    )

    objective = 0.0

    for h in range(
        1,
        H + 1,
    ):

        gamma_h = sample_autocov(
            v,
            h,
        )

        objective += np.trace(
            gamma_h
            @ gamma0_inv
            @ gamma_h.T
            @ gamma0_inv
        )

    return float(objective)


# ============================================================
# 13. FINITE-DIFFERENCE GRADIENT
# ============================================================

def finite_difference_gradient(
    fun: Callable[[Array], float],
    x: Array,
    eps: float = 1e-6,
) -> Array:
    """
    Central finite-difference gradient.
    """

    x = np.asarray(
        x,
        dtype=float,
    )

    gradient = np.zeros_like(
        x
    )

    for j in range(
        x.size
    ):

        step = (
            eps
            *
            max(
                1.0,
                abs(x[j]),
            )
        )

        x_plus = x.copy()
        x_minus = x.copy()
```

```
        x_plus[j] += step
        x_minus[j] -= step

        f_plus = fun(
            x_plus
        )

        f_minus = fun(
            x_minus
        )

        gradient[j] = (
            f_plus
            -
            f_minus
        ) / (
            2.0
            *
            step
        )

    return gradient


def check_gradient(
    fun,
    gradient_fun,
    x,
    eps=1e-6,
):
    """
    Compare an analytic/AD gradient with finite differences.
    """

    gradient_fd = finite_difference_gradient(
        fun,
        x,
        eps,
    )

    gradient_model = np.asarray(
        gradient_fun(x),
        dtype=float,
    )

    absolute_error = np.max(
        np.abs(
            gradient_model
            -
            gradient_fd
        )
    )

    relative_error = np.max(
        np.abs(
            gradient_model
            -
            gradient_fd
        )
        /
        np.maximum(
```

```
            1e-8,
            np.abs(
                gradient_fd
            ),
        )
    )

    return {
        "max_absolute_error":
            float(absolute_error),

        "max_relative_error":
            float(relative_error),
    }


# ============================================================
# 14. OPTIMIZATION DIAGNOSTICS
# ============================================================

@dataclass
class StartDiagnostic:

    start: int

    objective: float

    success: bool

    certified: bool

    gradient_inf_norm: float

    iterations: int

    function_evaluations: int

    message: str


@dataclass
class RGCovFit:

    theta: Array

    phi_list: list[Array]

    objective: float

    delta: float

    diagnostics: list[StartDiagnostic]

    method: str


def certify_result(
    result,
    gradient_tolerance: float = 1e-5,
) -> tuple[bool, float]:
    """
```

```
    Basic numerical convergence certificate.
    """

    jac = getattr(
        result,
        "jac",
        None,
    )

    if jac is None:

        gradient_inf_norm = np.inf

    else:

        gradient_inf_norm = float(
            np.linalg.norm(
                jac,
                ord=np.inf,
            )
        )

    finite = (
        np.isfinite(
            result.fun
        )
        and
        np.all(
            np.isfinite(
                result.x
            )
        )
    )

    certified = bool(
        result.success
        and
        finite
        and
        gradient_inf_norm
        <=
        gradient_tolerance
    )

    return (
        certified,
        gradient_inf_norm,
    )


# ============================================================
# 15. INTERNAL MULTI-START ESTIMATOR
# ============================================================

def _fit_gcov_family(
    y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float,
    diagonal: bool,
    n_starts: int,
```

```
    perturbation_scale: float,
    seed: int,
    maxiter: int,
) -> RGCovFit:

    y = validate_y(y)

    rng = np.random.default_rng(
        seed
    )

    # OLS initialization

    phi_ols, _ = ols_var(
        y,
        p,
    )

    theta_ols = stack_theta(
        phi_ols
    )

    scale = (
        perturbation_scale
        *
        max(
            float(
                np.std(
                    theta_ols
                )
            ),
            1e-4,
        )
    )

    starts = [
        theta_ols
    ]

    for _ in range(
        n_starts - 1
    ):

        perturbation = rng.normal(
            scale=scale,
            size=theta_ols.shape,
        )

        starts.append(
            theta_ols
            +
            perturbation
        )

    diagnostics = []

    best_result = None

    def objective(
        theta,
    ):
```

```
        return gcov_objective(
            theta=theta,
            y=y,
            p=p,
            H=H,
            transforms=transforms,
            delta=delta,
            diagonal=diagonal,
        )

    for start_index, theta0 in enumerate(
        starts
    ):

        result = minimize(
            objective,
            theta0,
            method="L-BFGS-B",
            jac=None,
            options={
                "maxiter": maxiter,
                "ftol": 1e-12,
                "gtol": 1e-7,
                "maxls": 50,
            },
        )

        (
            certified,
            gradient_inf_norm,
        ) = certify_result(
            result
        )

        diagnostic = StartDiagnostic(
            start=start_index,

            objective=float(
                result.fun
            ),

            success=bool(
                result.success
            ),

            certified=certified,

            gradient_inf_norm=
                gradient_inf_norm,

            iterations=int(
                result.nit
            ),

            function_evaluations=int(
                result.nfev
            ),

            message=str(
                result.message
```

```
            ),
        )

        diagnostics.append(
            diagnostic
        )

        if (
            best_result is None
            or
            result.fun
            <
            best_result.fun
        ):

            best_result = result

    if best_result is None:

        raise RuntimeError(
            "No optimization result was produced."
        )

    n = y.shape[1]

    phi_hat = unstack_theta(
        best_result.x,
        n,
        p,
    )

    if diagonal:

        method_name = (
            "Diagonal-GCov"
        )

    elif delta == 0:

        method_name = (
            "GCov"
        )

    else:

        method_name = (
            "RGCov"
        )

    return RGCovFit(
        theta=np.asarray(
            best_result.x
        ),

        phi_list=phi_hat,

        objective=float(
            best_result.fun
        ),

        delta=float(
```

```
            delta
        ),

        diagnostics=diagnostics,

        method=method_name,
    )


# ============================================================
# 16. PUBLIC GCOV ESTIMATORS
# ============================================================

def fit_gcov(
    y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    n_starts: int = 10,
    perturbation_scale: float = 0.02,
    seed: int = 123,
    maxiter: int = 5000,
) -> RGCovFit:

    return _fit_gcov_family(
        y=y,
        p=p,
        H=H,
        transforms=transforms,
        delta=0.0,
        diagonal=False,
        n_starts=n_starts,
        perturbation_scale=
            perturbation_scale,
        seed=seed,
        maxiter=maxiter,
    )


def fit_rgcov(
    y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float = 1e-3,
    n_starts: int = 10,
    perturbation_scale: float = 0.02,
    seed: int = 123,
    maxiter: int = 5000,
) -> RGCovFit:

    return _fit_gcov_family(
        y=y,
        p=p,
        H=H,
        transforms=transforms,
        delta=delta,
        diagonal=False,
        n_starts=n_starts,
        perturbation_scale=
            perturbation_scale,
        seed=seed,
```

```
        maxiter=maxiter,
    )


def fit_diagonal_gcov(
    y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float = 1e-6,
    n_starts: int = 10,
    perturbation_scale: float = 0.02,
    seed: int = 123,
    maxiter: int = 5000,
) -> RGCovFit:

    return _fit_gcov_family(
        y=y,
        p=p,
        H=H,
        transforms=transforms,
        delta=delta,
        diagonal=True,
        n_starts=n_starts,
        perturbation_scale=
            perturbation_scale,
        seed=seed,
        maxiter=maxiter,
    )


# ============================================================
# 17. COMPANION MATRIX
# ============================================================

def companion_matrix(
    phi_list: Sequence[Array],
) -> Array:
    """
    Construct standard VAR companion matrix.
    """

    if len(phi_list) == 0:
        raise ValueError(
            "phi_list cannot be empty."
        )

    p = len(phi_list)

    n = phi_list[0].shape[0]

    if p == 1:

        return np.asarray(
            phi_list[0],
            dtype=float,
        )

    top = np.column_stack(
        phi_list
    )
```

```
    bottom = np.column_stack(
        [
            np.eye(
                n * (p - 1)
            ),

            np.zeros(
                (
                    n * (p - 1),
                    n,
                )
            ),
        ]
    )

    companion = np.row_stack(
        [
            top,
            bottom,
        ]
    )

    return companion


def companion_eigenvalues(
    phi_list: Sequence[Array],
) -> Array:
    """
    Companion eigenvalues.

    These should not be called polynomial roots
    without explicitly stating the convention.
    """

    companion = companion_matrix(
        phi_list
    )

    return np.linalg.eigvals(
        companion
    )


def classify_var_roots(
    phi_list: Sequence[Array],
    tolerance: float = 1e-6,
) -> dict:
    """
    Classify companion eigenvalues relative to unit circle.
    """

    eigenvalues = companion_eigenvalues(
        phi_list
    )

    modulus = np.abs(
        eigenvalues
    )
```

```
    inside = int(
        np.sum(
            modulus
            <
            1.0 - tolerance
        )
    )

    boundary = int(
        np.sum(
            np.abs(
                modulus - 1.0
            )
            <=
            tolerance
        )
    )

    outside = int(
        np.sum(
            modulus
            >
            1.0 + tolerance
        )
    )

    return {

        "eigenvalues":
            eigenvalues,

        "inside_unit_circle":
            inside,

        "on_boundary":
            boundary,

        "outside_unit_circle":
            outside,

        "stable":
            bool(
                np.all(
                    modulus
                    <
                    1.0 - tolerance
                )
            ),

        "spectral_radius":
            float(
                np.max(
                    modulus
                )
            ),
    }


# ============================================================
# 18. INNOVATION GENERATORS
# ============================================================
```

```
def generate_innovations(
    rng: np.random.Generator,
    T: int,
    n: int,
    innovation: str,
) -> Array:
    """
    Gaussian, Student-t5 or simple skewed heavy-tail innovations.
    """

    innovation = innovation.lower()

    if innovation == "gaussian":

        return rng.normal(
            size=(T, n)
        )

    if innovation in {
        "t5",
        "student-t5",
        "student_t5",
    }:

        # Standardize t5 variance to approximately 1.

        return (
            rng.standard_t(
                df=5,
                size=(T, n),
            )
            /
            np.sqrt(
                5.0 / 3.0
            )
        )

    if innovation in {
        "skewed",
        "skewed_t",
    }:

        z = (
            rng.standard_t(
                df=5,
                size=(T, n),
            )
            /
            np.sqrt(
                5.0 / 3.0
            )
        )

        x = (
            z
            +
            0.6
            *
            (
                z ** 2
                -
```

```
                1.0
            )
        )

        x = (
            x
            -
            x.mean(
                axis=0,
                keepdims=True,
            )
        )

        x = (
            x
            /
            np.maximum(
                x.std(
                    axis=0,
                    ddof=1,
                    keepdims=True,
                ),
                1e-10,
            )
        )

        return x

    raise ValueError(
        "innovation must be "
        "'gaussian', 't5', or 'skewed'."
    )


# ============================================================
# 19. CAUSAL VAR SIMULATION
# ============================================================

def simulate_var(
    T: int,
    phi_list: Sequence[Array],
    burn: int = 300,
    innovation: str = "t5",
    seed: int = 123,
) -> Array:
    """
    Simulate a causal VAR:

        Y_t =
        sum_j Phi_j Y_{t-j}
        + epsilon_t
    """

    rng = np.random.default_rng(
        seed
    )

    p = len(
        phi_list
    )
```

```
    if p == 0:
        raise ValueError(
            "At least one coefficient matrix is required."
        )

    n = phi_list[0].shape[0]

    total = (
        T
        +
        burn
        +
        p
        +
        5
    )

    epsilon = generate_innovations(
        rng,
        total,
        n,
        innovation,
    )

    y = np.zeros(
        (total, n)
    )

    for t in range(
        p,
        total,
    ):

        autoregressive = np.zeros(
            n
        )

        for j in range(
            p
        ):

            autoregressive += (
                phi_list[j]
                @
                y[t - j - 1]
            )

        y[t] = (
            autoregressive
            +
            epsilon[t]
        )

    return y[
        burn + p:
        burn + p + T
    ]


# ============================================================
# 20. MIXED CAUSAL-NONCAUSAL SIMULATION
```

```
# ============================================================

def simulate_mixed_var(
    T: int,
    Pi_list: Sequence[Array],
    Phi_list: Sequence[Array],
    burn: int = 500,
    innovation: str = "t5",
    seed: int = 123,
) -> Array:
    r"""
    Simulate:

        Pi(L) Phi(L^{-1}) Y_t = epsilon_t

    where

        Pi(L)
        =
        I - Pi_1 L - ... - Pi_r L^r

    and

        Phi(L^{-1})
        =
        I - Phi_1 L^{-1}
          - ... - Phi_s L^{-s}.

    Procedure
    ---------
    1. Generate X_t satisfying

           Pi(L) X_t = epsilon_t

       using a forward recursion.

    2. Solve

           Phi(L^{-1}) Y_t = X_t

       using a backward recursion.

    3. Simulate a long window and discard boundaries.

    This provides a practical finite-window approximation
    to a two-sided mixed process.
    """

    if (
        len(Pi_list) == 0
        and
        len(Phi_list) == 0
    ):
        raise ValueError(
            "Need a causal or noncausal component."
        )

    reference = (
        Pi_list[0]
        if len(Pi_list) > 0
        else Phi_list[0]
    )
```

```
    )

    n = reference.shape[0]

    r = len(
        Pi_list
    )

    s = len(
        Phi_list
    )

    rng = np.random.default_rng(
        seed
    )

    margin = max(
        r,
        s,
        1,
    )

    total = (
        T
        +
        2 * burn
        +
        2 * margin
        +
        10
    )

    epsilon = generate_innovations(
        rng,
        total,
        n,
        innovation,
    )

    # ----------------------------------
    # Step 1:
    # Pi(L) X_t = epsilon_t
    # ----------------------------------

    x = np.zeros(
        (total, n)
    )

    for t in range(
        r,
        total,
    ):

        causal_part = np.zeros(
            n
        )

        for j in range(
            r
        ):

            causal_part += (
```

```
            Pi_list[j]
            @
            x[t - j - 1]
        )

    x[t] = (
        causal_part
        +
        epsilon[t]
    )

# ----------------------------------
# Step 2:
# Phi(L^-1) Y_t = X_t
# ----------------------------------

y = np.zeros_like(
    x
)

last_start = (
    total
    -
    s
    -
    1
)

for t in range(
    last_start,
    -1,
    -1,
):

    future_part = np.zeros(
        n
    )

    for j in range(
        s
    ):

        future_part += (
            Phi_list[j]
            @
            y[t + j + 1]
        )

    y[t] = (
        x[t]
        +
        future_part
    )

start = (
    burn
    +
    margin
)

stop = (
```

```
        start
        +
        T
    )

    return y[
        start:stop
    ]


# ============================================================
# 21. SERIAL-DEPENDENCE TEST
# ============================================================

def portmanteau_stat(
    v: Array,
    H: int,
) -> dict:
    """
    Simple GCov-style portmanteau statistic.

    This is mainly a diagnostic implementation.
    """

    v = np.asarray(
        v,
        dtype=float,
    )

    T = v.shape[0]

    gamma0 = sample_autocov(
        v,
        0,
    )

    gamma0_inverse = np.linalg.pinv(
        gamma0
    )

    q = 0.0

    for h in range(
        1,
        H + 1,
    ):

        gamma_h = sample_autocov(
            v,
            h,
        )

        q += np.trace(
            gamma_h
            @ gamma0_inverse
            @ gamma_h.T
            @ gamma0_inverse
        )

    statistic = (
        T * q
```

```
    )

    dimension = v.shape[1]

    degrees_of_freedom = (
        H
        *
        dimension ** 2
    )

    pvalue = chi2.sf(
        statistic,
        degrees_of_freedom,
    )

    return {

        "statistic":
            float(statistic),

        "df":
            int(
                degrees_of_freedom
            ),

        "pvalue":
            float(pvalue),
    }


# ============================================================
# 22. CONVERGENCE SUMMARY
# ============================================================

def convergence_summary(
    fit: RGCovFit,
) -> dict:

    diagnostics = (
        fit.diagnostics
    )

    successful = sum(
        diagnostic.success
        for diagnostic
        in diagnostics
    )

    certified = sum(
        diagnostic.certified
        for diagnostic
        in diagnostics
    )

    iterations = [
        diagnostic.iterations
        for diagnostic
        in diagnostics
    ]

    return {
```

```
        "method":
            fit.method,

        "number_of_starts":
            len(diagnostics),

        "successful_starts":
            int(successful),

        "certified_starts":
            int(certified),

        "best_objective":
            float(
                fit.objective
            ),

        "median_iterations":
            float(
                np.median(
                    iterations
                )
            ),
    }


def diagnostics_frame(
    fit: RGCovFit,
) -> pd.DataFrame:
    """
    Convert optimization diagnostics into a DataFrame.
    """

    return pd.DataFrame(
        [
            diagnostic.__dict__
            for diagnostic
            in fit.diagnostics
        ]
    )


# ============================================================
# 23. DELTA SELECTION
# ============================================================

def select_delta(
    y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    deltas: Sequence[float] = (
        1e-6,
        1e-5,
        1e-4,
        1e-3,
        1e-2,
    ),
    n_starts: int = 3,
    seed: int = 123,
):
```

```
    """
    Simple objective-based delta comparison.

    For a final paper, tuning should be justified more carefully,
    e.g. by a theoretically motivated rule or validation criterion.
    """

    fits = []

    for delta in deltas:

        fit = fit_rgcov(
            y=y,
            p=p,
            H=H,
            transforms=transforms,
            delta=float(delta),
            n_starts=n_starts,
            seed=seed,
        )

        fits.append(
            fit
        )

    best_fit = min(
        fits,
        key=lambda x: x.objective,
    )

    return (
        best_fit,
        fits,
    )


# ============================================================
# 24. RESIDUAL BOOTSTRAP
# ============================================================

def residual_bootstrap_var(
    y: Array,
    p: int,
    B: int = 199,
    burn: int = 300,
    seed: int = 123,
) -> dict:
    """
    Residual bootstrap for the OLS VAR benchmark.

    Returns:
        coefficient percentile intervals
        spectral-radius percentile interval
    """

    y = validate_y(y)

    rng = np.random.default_rng(
        seed
    )
```

```
    phi_hat, residuals = ols_var(
        y,
        p,
    )

    residuals = (
        residuals
        -
        residuals.mean(
            axis=0,
            keepdims=True,
        )
    )

    T, n = y.shape

    coefficient_draws = []

    radius_draws = []

    total = (
        T
        +
        burn
        +
        p
    )

    for _ in range(
        B
    ):

        indices = rng.integers(
            low=0,
            high=len(residuals),
            size=total,
        )

        epsilon = residuals[
            indices
        ]

        y_bootstrap = np.zeros(
            (total, n)
        )

        for t in range(
            p,
            total,
        ):

            fitted = np.zeros(
                n
            )

            for j in range(
                p
            ):

                fitted += (
                    phi_hat[j]
                    @
```

```
                    @
                    y_bootstrap[
                        t - j - 1
                    ]
                )

            y_bootstrap[t] = (
                fitted
                +
                epsilon[t]
            )

        bootstrap_sample = (
            y_bootstrap[-T:]
        )

        phi_bootstrap, _ = ols_var(
            bootstrap_sample,
            p,
        )

        theta_bootstrap = stack_theta(
            phi_bootstrap
        )

        coefficient_draws.append(
            theta_bootstrap
        )

        eigenvalues = companion_eigenvalues(
            phi_bootstrap
        )

        radius_draws.append(
            np.max(
                np.abs(
                    eigenvalues
                )
            )
        )

    coefficient_draws = np.vstack(
        coefficient_draws
    )

    radius_draws = np.asarray(
        radius_draws
    )

    return {

        "coefficient_lower":
            np.quantile(
                coefficient_draws,
                0.025,
                axis=0,
            ),

        "coefficient_upper":
            np.quantile(
                coefficient_draws,
                0.975,
```

```
                axis=0,
            ),

        "spectral_radius_lower":
            float(
                np.quantile(
                    radius_draws,
                    0.025,
                )
            ),

        "spectral_radius_upper":
            float(
                np.quantile(
                    radius_draws,
                    0.975,
                )
            ),
    }


# ============================================================
# 25. OBJECTIVE CONSISTENCY TEST
# ============================================================

def test_objective_equivalence():
    """
    Numerically verify that the efficient objective
    matches the literal trace expression.
    """

    A = np.array(
        [
            [0.50, 0.05],
            [0.00, 0.40],
        ]
    )

    y = simulate_var(
        T=150,
        phi_list=[A],
        innovation="t5",
        seed=2,
    )

    phi_ols, _ = ols_var(
        y,
        p=1,
    )

    theta = stack_theta(
        phi_ols
    )

    transforms = make_transforms(
        powers=(1, 2),
    )

    fast = gcov_objective(
        theta=theta,
        y=y,
```

```
        p=1,
        H=2,
        transforms=transforms,
        delta=1e-3,
    )

    literal = literal_trace_objective(
        theta=theta,
        y=y,
        p=1,
        H=2,
        transforms=transforms,
        delta=1e-3,
    )

    error = abs(
        fast
        -
        literal
    )

    print(
        "\nObjective consistency test"
    )

    print(
        "Fast objective:",
        fast,
    )

    print(
        "Literal objective:",
        literal,
    )

    print(
        "Absolute difference:",
        error,
    )

    assert np.allclose(
        fast,
        literal,
        rtol=1e-9,
        atol=1e-10,
    )

    print(
        "PASS"
    )


# ============================================================
# 26. SMALL MONTE CARLO EXPERIMENT
# ============================================================

def monte_carlo_experiment(
    replications: int = 20,
    T: int = 250,
    n: int = 2,
    p: int = 1,
```

```
    H: int = 2,
    innovation: str = "t5",
    design: str = "causal",
    delta: float = 1e-3,
    n_starts: int = 3,
    seed: int = 123,
) -> pd.DataFrame:
    """
    Small Monte Carlo framework.

    For the final paper, extend this across:

        N = 2, 5, 10, 20
        T = 250, 500, 1000
        p = 1, 2
        causal/noncausal/mixed designs
        Gaussian/t5/skewed innovations
        >= 500 replications
    """

    rng = np.random.default_rng(
        seed
    )

    transforms = make_transforms(
        powers=(1, 2),
    )

    # Stable diagonal matrices used for illustration.

    Pi = (
        0.55
        *
        np.eye(n)
    )

    Phi = (
        0.35
        *
        np.eye(n)
    )

    results = []

    for replication in range(
        replications
    ):

        replication_seed = int(
            rng.integers(
                1,
                2_000_000_000,
            )
        )

        if design == "causal":

            y = simulate_var(
                T=T,
                phi_list=[Pi],
                innovation=innovation,
```

```
            seed=replication_seed,
        )

    elif design == "mixed":

        y = simulate_mixed_var(
            T=T,
            Pi_list=[Pi],
            Phi_list=[Phi],
            innovation=innovation,
            seed=replication_seed,
        )

    else:

        raise ValueError(
            "design must be 'causal' or 'mixed'."
        )

    # ----------------------------------
    # OLS
    # ----------------------------------

    start_time = time.perf_counter()

    phi_ols, _ = ols_var(
        y,
        p,
    )

    runtime = (
        time.perf_counter()
        -
        start_time
    )

    root_info = classify_var_roots(
        phi_ols
    )

    results.append(
        {
            "replication":
                replication,

            "design":
                design,

            "estimator":
                "OLS",

            "spectral_radius":
                root_info[
                    "spectral_radius"
                ],

            "stable":
                root_info[
                    "stable"
                ],

            "converged":
```

```
                True,

            "objective":
                np.nan,

            "runtime_seconds":
                runtime,
        }
    )

    # ----------------------------------
    # LASSO
    # ----------------------------------

    start_time = time.perf_counter()

    phi_lasso, _ = lasso_var(
        y,
        p,
        alpha=0.01,
    )

    runtime = (
        time.perf_counter()
        -
        start_time
    )

    root_info = classify_var_roots(
        phi_lasso
    )

    results.append(
        {
            "replication":
                replication,

            "design":
                design,

            "estimator":
                "Lasso",

            "spectral_radius":
                root_info[
                    "spectral_radius"
                ],

            "stable":
                root_info[
                    "stable"
                ],

            "converged":
                True,

            "objective":
                np.nan,

            "runtime_seconds":
                runtime,
```

```
            }
        )

        # ----------------------------------
        # GCov
        # ----------------------------------

        start_time = time.perf_counter()

        fit = fit_gcov(
            y=y,
            p=p,
            H=H,
            transforms=transforms,
            n_starts=n_starts,
            seed=replication_seed,
        )

        runtime = (
            time.perf_counter()
            -
            start_time
        )

        root_info = classify_var_roots(
            fit.phi_list
        )

        results.append(
            {
                "replication":
                    replication,

                "design":
                    design,

                "estimator":
                    "GCov",

                "spectral_radius":
                    root_info[
                        "spectral_radius"
                    ],

                "stable":
                    root_info[
                        "stable"
                    ],

                "converged":
                    any(
                        diagnostic.certified
                        for diagnostic
                        in fit.diagnostics
                    ),

                "objective":
                    fit.objective,

                "runtime_seconds":
                    runtime,
```

```
        }
    )

    # ----------------------------------
    # DIAGONAL-GCOV
    # ----------------------------------

    start_time = time.perf_counter()

    fit = fit_diagonal_gcov(
        y=y,
        p=p,
        H=H,
        transforms=transforms,
        delta=1e-6,
        n_starts=n_starts,
        seed=replication_seed,
    )

    runtime = (
        time.perf_counter()
        -
        start_time
    )

    root_info = classify_var_roots(
        fit.phi_list
    )

    results.append(
        {
            "replication":
                replication,

            "design":
                design,

            "estimator":
                "Diagonal-GCov",

            "spectral_radius":
                root_info[
                    "spectral_radius"
                ],

            "stable":
                root_info[
                    "stable"
                ],

            "converged":
                any(
                    diagnostic.certified
                    for diagnostic
                    in fit.diagnostics
                ),

            "objective":
                fit.objective,

            "runtime_seconds":
                runtime
```

```
                runtime,
            }
        )

        # ----------------------------------
        # RGCOV
        # ----------------------------------

        start_time = time.perf_counter()

        fit = fit_rgcov(
            y=y,
            p=p,
            H=H,
            transforms=transforms,
            delta=delta,
            n_starts=n_starts,
            seed=replication_seed,
        )

        runtime = (
            time.perf_counter()
            -
            start_time
        )

        root_info = classify_var_roots(
            fit.phi_list
        )

        results.append(
            {
                "replication":
                    replication,

                "design":
                    design,

                "estimator":
                    "RGCov",

                "spectral_radius":
                    root_info[
                        "spectral_radius"
                    ],

                "stable":
                    root_info[
                        "stable"
                    ],

                "converged":
                    any(
                        diagnostic.certified
                        for diagnostic
                        in fit.diagnostics
                    ),

                "objective":
                    fit.objective,

                "runtime_seconds":
```

```
                    runtime,
            }
        )

        print(
            f"Replication "
            f"{replication + 1}/"
            f"{replications} completed."
        )

    return pd.DataFrame(
        results
    )


# ============================================================
# 27. MONTE CARLO SUMMARY
# ============================================================

def summarize_monte_carlo(
    results: pd.DataFrame,
) -> pd.DataFrame:
    """
    Produce basic computational summary.
    """

    summary = (
        results
        .groupby(
            "estimator"
        )
        .agg(
            convergence_rate=(
                "converged",
                "mean",
            ),

            stable_rate=(
                "stable",
                "mean",
            ),

            mean_spectral_radius=(
                "spectral_radius",
                "mean",
            ),

            median_runtime=(
                "runtime_seconds",
                "median",
            ),

            mean_objective=(
                "objective",
                "mean",
            ),
        )
        .reset_index()
    )

    return summary
```

```
# ============================================================
# 28. EMPIRICAL DATA HELPER
# ============================================================

def load_empirical_csv(
    path: str,
) -> tuple[pd.DataFrame, Array]:
    """
    Load numerical columns from a CSV and standardize them.
    """

    dataframe = pd.read_csv(
        path
    )

    numeric = (
        dataframe
        .select_dtypes(
            include=[
                np.number
            ]
        )
        .dropna()
        .copy()
    )

    if numeric.shape[1] < 2:
        raise ValueError(
            "Need at least two numerical variables."
        )

    standard_deviation = numeric.std(
        ddof=1
    )

    standard_deviation = (
        standard_deviation
        .replace(
            0,
            np.nan,
        )
    )

    standardized = (
        numeric
        -
        numeric.mean()
    ) / standard_deviation

    standardized = (
        standardized
        .dropna(
            axis=1
        )
    )

    y = standardized.to_numpy(
        dtype=float
    )
```

```
    return (
        standardized,
        y,
    )


# ============================================================
# 29. EMPIRICAL ANALYSIS
# ============================================================

def empirical_analysis(
    csv_path: str,
    p: int = 1,
    H: int = 2,
    delta: float = 1e-3,
    n_starts: int = 10,
):
    """
    Basic empirical workflow.
    """

    dataframe, y = load_empirical_csv(
        csv_path
    )

    transforms = make_transforms(
        powers=(1, 2),
    )

    # OLS

    phi_ols, residuals_ols = ols_var(
        y,
        p,
    )

    # Lasso

    phi_lasso, residuals_lasso = lasso_var(
        y,
        p,
    )

    # RGCov

    rgcov_fit = fit_rgcov(
        y=y,
        p=p,
        H=H,
        transforms=transforms,
        delta=delta,
        n_starts=n_starts,
    )

    print(
        "\n=============================="
    )

    print(
        "EMPIRICAL RESULTS"
    )
```

```
)

print(
    "=============================="
)

print(
    "\nVariables:"
)

print(
    list(
        dataframe.columns
    )
)

print(
    "\nOLS root diagnostics:"
)

print(
    classify_var_roots(
        phi_ols
    )
)

print(
    "\nLasso root diagnostics:"
)

print(
    classify_var_roots(
        phi_lasso
    )
)

print(
    "\nRGCov root diagnostics:"
)

print(
    classify_var_roots(
        rgcov_fit.phi_list
    )
)

print(
    "\nRGCov convergence:"
)

print(
    convergence_summary(
        rgcov_fit
    )
)

# transformed residual diagnostics

transformed_ols = transform_residuals(
    residuals_ols,
    transforms,
)
```

```
    print(
        "\nOLS transformed-residual "
        "serial-dependence diagnostic:"
    )

    print(
        portmanteau_stat(
            transformed_ols,
            H=H,
        )
    )

    return {

        "data":
            dataframe,

        "ols_phi":
            phi_ols,

        "lasso_phi":
            phi_lasso,

        "rgcov_fit":
            rgcov_fit,

        "rgcov_diagnostics":
            diagnostics_frame(
                rgcov_fit
            ),
    }


# ============================================================
# 30. DEMONSTRATION
# ============================================================

def demonstration():
    """
    Run a small reproducible example.
    """

    print(
        "=" * 70
    )

    print(
        "RGCov Computational Demonstration"
    )

    print(
        "=" * 70
    )

    # ------------------------------------------------------
    # True causal VAR
    # ------------------------------------------------------

    A = np.array(
        [
```

```
            [0.55, 0.10],
            [0.05, 0.40],
        ]
    )

    print(
        "\nTrue VAR coefficient matrix:"
    )

    print(
        A
    )

    # ------------------------------------------------------------
    # Simulate data
    # ------------------------------------------------------------

    y = simulate_var(
        T=500,
        phi_list=[A],
        innovation="t5",
        seed=123,
    )

    print(
        "\nSimulated data shape:",
        y.shape,
    )

    # ------------------------------------------------------------
    # Transformations
    # ------------------------------------------------------------

    transforms = make_transforms(
        powers=(1, 2),
    )

    # ------------------------------------------------------------
    # OLS
    # ------------------------------------------------------------

    phi_ols, residuals_ols = ols_var(
        y,
        p=1,
    )

    print(
        "\nOLS estimate:"
    )

    print(
        phi_ols[0]
    )

    print(
        "\nOLS root diagnostics:"
    )

    print(
        classify_var_roots(
            phi_ols
```

```
    )
)

# ----------------------------------------------------------
# LASSO
# ----------------------------------------------------------

phi_lasso, residuals_lasso = lasso_var(
    y,
    p=1,
    alpha=0.01,
)

print(
    "\nLasso estimate:"
)

print(
    phi_lasso[0]
)

# ----------------------------------------------------------
# GCOV
# ----------------------------------------------------------

print(
    "\nEstimating GCov..."
)

gcov_fit = fit_gcov(
    y=y,
    p=1,
    H=2,
    transforms=transforms,
    n_starts=5,
    seed=123,
)

print(
    "\nGCov estimate:"
)

print(
    gcov_fit.phi_list[0]
)

print(
    "\nGCov convergence:"
)

print(
    convergence_summary(
        gcov_fit
    )
)

# ----------------------------------------------------------
# DIAGONAL-GCOV
# ----------------------------------------------------------

print(
    "\nEstimating Diagonal-GCov..."
```

```
    )

    diagonal_fit = fit_diagonal_gcov(
        y=y,
        p=1,
        H=2,
        transforms=transforms,
        delta=1e-6,
        n_starts=5,
        seed=123,
    )

    print(
        "\nDiagonal-GCov estimate:"
    )

    print(
        diagonal_fit.phi_list[0]
    )

    # ----------------------------------------------------------
    # RGCOV
    # ----------------------------------------------------------

    print(
        "\nEstimating RGCov..."
    )

    rgcov_fit = fit_rgcov(
        y=y,
        p=1,
        H=2,
        transforms=transforms,
        delta=1e-3,
        n_starts=5,
        seed=123,
    )

    print(
        "\nRGCov estimate:"
    )

    print(
        rgcov_fit.phi_list[0]
    )

    print(
        "\nRGCov root diagnostics:"
    )

    print(
        classify_var_roots(
            rgcov_fit.phi_list
        )
    )

    print(
        "\nRGCov convergence:"
    )

    print(
```

```
    convergence_summary(
        rgcov_fit
    )
)

print(
    "\nIndividual optimization starts:"
)

print(
    diagnostics_frame(
        rgcov_fit
    )
)

# ----------------------------------------------------------
# Objective validation
# ----------------------------------------------------------

test_objective_equivalence()

# ----------------------------------------------------------
# Residual serial dependence
# ----------------------------------------------------------

transformed_residuals = transform_residuals(
    residuals_ols,
    transforms,
)

serial_test = portmanteau_stat(
    transformed_residuals,
    H=2,
)

print(
    "\nTransformed residual "
    "serial-dependence test:"
)

print(
    serial_test
)

# ----------------------------------------------------------
# Mixed causal/noncausal simulation
# ----------------------------------------------------------

Pi = np.array(
    [
        [0.50, 0.05],
        [0.00, 0.40],
    ]
)

Phi = np.array(
    [
        [0.30, 0.00],
        [0.05, 0.25],
    ]
)
```

```
    mixed_y = simulate_mixed_var(
        T=500,
        Pi_list=[Pi],
        Phi_list=[Phi],
        innovation="t5",
        seed=456,
    )

    print(
        "\nMixed causal-noncausal "
        "simulation shape:"
    )

    print(
        mixed_y.shape
    )

    print(
        "\nIMPORTANT:"
    )

    print(
        "The mixed process above is a genuine "
        "two-sided simulation, but fit_rgcov() "
        "still estimates a causal VAR parameterization."
    )


# ============================================================
# 31. MAIN
# ============================================================

if __name__ == "__main__":

    # ------------------------------------------------------
    # Run basic demonstration
    # ------------------------------------------------------

    demonstration()

    # ------------------------------------------------------
    # OPTIONAL MONTE CARLO
    #
    # Uncomment this section when ready.
    # Start with 5-20 replications.
    # Do NOT immediately use 500 replications while debugging.
    # ------------------------------------------------------

    """
    results = monte_carlo_experiment(
        replications=20,
        T=250,
        n=2,
        p=1,
        H=2,
        innovation="t5",
        design="causal",
        delta=1e-3,
        n_starts=3,
        seed=123,
    )
```

```
)

print(
    "\nMonte Carlo results:"
)

print(
    results.head()
)

summary = summarize_monte_carlo(
    results
)

print(
    "\nMonte Carlo summary:"
)

print(
    summary
)

results.to_csv(
    "monte_carlo_results.csv",
    index=False,
)

summary.to_csv(
    "monte_carlo_summary.csv",
    index=False,
)
"""

# ------------------------------------------------------------
# OPTIONAL MIXED-DGP MONTE CARLO
# ------------------------------------------------------------

"""
mixed_results = monte_carlo_experiment(
    replications=20,
    T=250,
    n=2,
    p=1,
    H=2,
    innovation="t5",
    design="mixed",
    delta=1e-3,
    n_starts=3,
    seed=123,
)

print(
    summarize_monte_carlo(
        mixed_results
    )
)

mixed_results.to_csv(
    "mixed_monte_carlo_results.csv",
    index=False,
)
"""
```

```
# ----------------------------------------------------------
# OPTIONAL BOOTSTRAP
# ----------------------------------------------------------

"""
A = np.array(
    [
        [0.55, 0.10],
        [0.05, 0.40],
    ]
)

y = simulate_var(
    T=500,
    phi_list=[A],
    innovation="t5",
    seed=123,
)

bootstrap_results = residual_bootstrap_var(
    y=y,
    p=1,
    B=199,
    seed=123,
)

print(
    bootstrap_results
)
"""

# ----------------------------------------------------------
# OPTIONAL EMPIRICAL APPLICATION
#
# Replace macro_financial_data.csv with your dataset.
# ----------------------------------------------------------

"""
empirical_results = empirical_analysis(
    csv_path="macro_financial_data.csv",
    p=1,
    H=2,
    delta=1e-3,
    n_starts=10,
)

empirical_results[
    "rgcov_diagnostics"
].to_csv(
    "rgcov_diagnostics.csv",
    index=False,
)
"""
```

```
GCov convergence:
{'method': 'GCov', 'number_of_starts': 5, 'successful_starts': 5, 'certified_starts': 5, 'best_o

Estimating Diagonal-GCov...

Diagonal-GCov estimate:
[[0.44359113 0.05291822]
 [0.03968854 0.37681906]]

Estimating RGCov...

RGCov estimate:
[[0.47432689 0.04529672]
 [0.04973438 0.36295128]]

RGCov root diagnostics:
{'eigenvalues': array([0.49180968, 0.34546849]), 'inside_unit_circle': 2, 'on_boundary': 0, 'out

RGCov convergence:
{'method': 'RGCov', 'number_of_starts': 5, 'successful_starts': 5, 'certified_starts': 5, 'best_

Individual optimization starts:
   start  objective  success  certified  gradient_inf_norm  iterations  \
0      0   0.064884     True       True       2.081668e-08           7
1      1   0.064884     True       True       1.318390e-07           7
2      2   0.064884     True       True       5.273559e-08           7
3      3   0.064884     True       True       5.134781e-08           7
4      4   0.064884     True       True       6.106227e-08           7

   function_evaluations                                        message
0                    45   CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL
1                    45  CONVERGENCE: RELATIVE REDUCTION OF F <= FACTR*...
2                    45   CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL
3                    45   CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL
4                    45   CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL

Objective consistency test
Fast objective: 0.1916129523677697
Literal objective: 0.19161295236776982
Absolute difference: 1.1102230246251565e-16
PASS

Transformed residual serial-dependence test:
{'statistic': 34.443429384727146, 'df': 32, 'pvalue': 0.3516297853165009}

Mixed causal-noncausal simulation shape:
(500, 2)

IMPORTANT:
The mixed process above is a genuine two-sided simulation, but fit_rgcov() still estimates a cau
```

```
"""
RGCov: Complete single-file research workflow
==============================================

Install:
    pip install numpy scipy pandas scikit-learn statsmodels

Run:
    python rgcov_all_code.py

Important:
    This is a research prototype. The optimizer currently uses
    finite-difference gradients. For final publication results,
```

```
    replace them with analytic or automatic differentiation and
    validate the gradient numerically.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Callable, Optional, Sequence

import numpy as np
import pandas as pd

from scipy.linalg import cho_factor, cho_solve
from scipy.optimize import minimize
from scipy.stats import chi2

from sklearn.linear_model import MultiTaskLasso
from statsmodels.stats.diagnostic import acorr_ljungbox


Array = np.ndarray
Transform = Callable[[Array], Array]


# ============================================================================
# 1. DATA UTILITIES
# ============================================================================

def validate_panel(Y: Array) -> Array:
    Y = np.asarray(Y, dtype=float)

    if Y.ndim != 2:
        raise ValueError("Y must have shape (T, n).")

    if Y.shape[0] < 20:
        raise ValueError("Y must contain at least 20 observations.")

    if Y.shape[1] < 1:
        raise ValueError("Y must contain at least one series.")

    if not np.all(np.isfinite(Y)):
        raise ValueError("Y contains non-finite values.")

    return Y


def standardize_panel(Y: Array) -> Array:
    Y = validate_panel(Y)
    mean = Y.mean(axis=0, keepdims=True)
    std = Y.std(axis=0, ddof=1, keepdims=True)
    return (Y - mean) / np.maximum(std, 1e-12)


def prices_to_log_returns(
    prices: pd.DataFrame,
) -> pd.DataFrame:
    prices = prices.copy()
    prices = prices.replace([np.inf, -np.inf], np.nan)
    prices = prices.dropna(how="any")

    if (prices <= 0).any().any():
```

```
        raise ValueError("All prices must be positive.")

    return np.log(prices).diff().dropna()


# ============================================================================
# 2. VAR BOOKKEEPING
# ============================================================================

def build_lagged(
    Y: Array,
    p: int,
) -> tuple[Array, Array]:
    """
    Build the VAR design matrix.

    Ycur[t] = Y[t+p]
    X[t]    = [Y[t+p-1], ..., Y[t]]
    """
    Y = validate_panel(Y)
    T, n = Y.shape

    if p < 1 or p >= T:
        raise ValueError("p must satisfy 1 <= p < T.")

    Ycur = Y[p:]

    X = np.column_stack(
        [
            Y[p - lag - 1:T - lag - 1]
            for lag in range(p)
        ]
    )

    return X, Ycur


def stack_theta(
    Phi_list: Sequence[Array],
) -> Array:
    if len(Phi_list) == 0:
        raise ValueError("Phi_list cannot be empty.")

    n = Phi_list[0].shape[0]

    for Phi in Phi_list:
        if Phi.shape != (n, n):
            raise ValueError("All coefficient matrices must be n x n.")

    return np.concatenate(
        [Phi.ravel(order="C") for Phi in Phi_list]
    )


def unstack_theta(
    theta: Array,
    n: int,
    p: int,
) -> list[Array]:
    theta = np.asarray(theta, dtype=float)
    expected = n * n * p
```

```
    if theta.size != expected:
        raise ValueError(
            f"Expected {expected} parameters, got {theta.size}."
        )

    return [
        theta[j * n * n:(j + 1) * n * n].reshape(n, n)
        for j in range(p)
    ]


def var_residuals(
    Y: Array,
    p: int,
    Phi_list: Sequence[Array],
) -> Array:
    X, Ycur = build_lagged(Y, p)
    Phi_stack = np.column_stack(Phi_list)
    fitted = X @ Phi_stack.T
    return Ycur - fitted


# ============================================================================
# 3. FIT RESULT
# ============================================================================

@dataclass
class FitResult:
    theta: Array
    Phi_list: list[Array]
    objective: float
    delta: float = 0.0
    diagonal: bool = False
    diagnostics: list[dict[str, Any]] = field(
        default_factory=list
    )


# ============================================================================
# 4. OLS AND LASSO
# ============================================================================

def fit_ols(
    Y: Array,
    p: int,
    ridge: float = 0.0,
) -> FitResult:
    """
    Fit OLS-VAR.

    If ridge > 0, this becomes ridge-OLS.
    """
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]

    XtX = X.T @ X
    XtY = X.T @ Ycur

    if ridge > 0:
        XtX = XtX + ridge * np.eye(X.shape[1])
```

```
    B = np.linalg.solve(XtX, XtY).T

    Phi_list = [
        B[:, j * n:(j + 1) * n]
        for j in range(p)
    ]

    residuals = Ycur - X @ B.T
    mse = float(np.mean(residuals ** 2))

    return FitResult(
        theta=stack_theta(Phi_list),
        Phi_list=Phi_list,
        objective=mse,
    )


def fit_lasso(
    Y: Array,
    p: int,
    alpha: float = 0.01,
    max_iter: int = 10000,
) -> FitResult:
    """
    Fit a multi-task Lasso VAR.
    """
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]

    model = MultiTaskLasso(
        alpha=alpha,
        fit_intercept=False,
        max_iter=max_iter,
        tol=1e-8,
    )

    model.fit(X, Ycur)

    B = model.coef_

    Phi_list = [
        B[:, j * n:(j + 1) * n]
        for j in range(p)
    ]

    residuals = Ycur - model.predict(X)
    mse = float(np.mean(residuals ** 2))

    return FitResult(
        theta=stack_theta(Phi_list),
        Phi_list=Phi_list,
        objective=mse,
    )


# ============================================================================
# 5. RESIDUAL TRANSFORMATIONS
# ============================================================================

def make_power_transforms(
```

```
    powers: Sequence[int] = (1, 2),
) -> list[Transform]:
    """
    Signed-power transformations.

    Example:
        powers=(1, 2)
    produces:
        u
        sign(u)*|u|^2
    """
    return [
        lambda U, power=power:
        np.sign(U) * np.abs(U) ** power
        for power in powers
    ]


def make_bounded_transforms() -> list[Transform]:
    return [
        lambda U: U,
        lambda U: np.tanh(U),
        lambda U: np.sign(U) * np.sqrt(np.abs(U)),
        lambda U: np.log1p(U ** 2),
    ]


def transform_residuals(
    U: Array,
    transforms: Sequence[Transform],
    standardize: bool = True,
) -> Array:
    blocks = []

    for transform in transforms:
        block = np.asarray(
            transform(U),
            dtype=float,
        )

        if block.shape != U.shape:
            raise ValueError(
                "Each transformation must preserve U's shape."
            )

        blocks.append(block)

    V = np.column_stack(blocks)

    if standardize:
        V = (
            V - V.mean(axis=0, keepdims=True)
        ) / (
            V.std(axis=0, ddof=1, keepdims=True)
            + 1e-10
        )

    return V


# ============================================================================
```

```
# 6. SAMPLE AUTOCOVARIANCES
# ============================================================================

def sample_autocov(
    V: Array,
    h: int,
) -> Array:
    V = np.asarray(V, dtype=float)

    if V.ndim != 2:
        raise ValueError("V must be two-dimensional.")

    T = V.shape[0]

    if h < 0 or h >= T:
        raise ValueError("Invalid autocovariance lag.")

    Vc = V - V.mean(axis=0, keepdims=True)

    if h == 0:
        return (Vc.T @ Vc) / T

    return (Vc[h:].T @ Vc[:-h]) / T


def regularize_covariance(
    Gamma0: Array,
    delta: float,
    diagonal: bool = False,
) -> Array:
    Gamma0 = 0.5 * (Gamma0 + Gamma0.T)

    if diagonal:
        Gamma0 = np.diag(np.diag(Gamma0))

    return Gamma0 + delta * np.eye(Gamma0.shape[0])


# ============================================================================
# 7. GCOV / RGCOV OBJECTIVE
# ============================================================================

def rgcov_objective(
    theta: Array,
    Y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float = 0.0,
    diagonal: bool = False,
) -> float:
    """
    GCov/RGCov objective:

    Q(theta) =
        sum_h trace[
            Gamma0_delta^{-1}
            Gamma_h
            Gamma0_delta^{-1}
            Gamma_h'
        ]
```

```
        delta regularizes Gamma0.
        """
        Y = validate_panel(Y)

        if H < 1:
            raise ValueError("H must be at least one.")

        if delta < 0:
            raise ValueError("delta must be nonnegative.")

        n = Y.shape[1]

        Phi_list = unstack_theta(
            theta,
            n=n,
            p=p,
        )

        U = var_residuals(
            Y=Y,
            p=p,
            Phi_list=Phi_list,
        )

        V = transform_residuals(
            U,
            transforms,
            standardize=True,
        )

        Gamma0 = sample_autocov(V, 0)

        Gamma0 = regularize_covariance(
            Gamma0,
            delta=delta,
            diagonal=diagonal,
        )

        try:
            factor = cho_factor(
                Gamma0,
                lower=True,
                check_finite=False,
            )
        except np.linalg.LinAlgError:
            return 1e100

        objective = 0.0

        for h in range(1, H + 1):
            Gamma_h = sample_autocov(V, h)

            whitened = cho_solve(
                factor,
                Gamma_h,
                check_finite=False,
            )

            objective += np.sum(whitened ** 2)
```

```
    if not np.isfinite(objective):
        return 1e100

    return float(objective)


# ============================================================================
# 8. OPTIMIZATION DIAGNOSTICS
# ============================================================================

def convergence_certificate(
    result: Any,
    gradient_tolerance: float = 1e-6,
) -> dict[str, Any]:
    if result.jac is None:
        gradient_inf_norm = np.inf
    else:
        gradient_inf_norm = float(
            np.linalg.norm(
                result.jac,
                ord=np.inf,
            )
        )

    finite_solution = bool(
        np.isfinite(result.fun)
        and np.all(np.isfinite(result.x))
    )

    gradient_ok = bool(
        gradient_inf_norm <= gradient_tolerance
    )

    certified = bool(
        result.success
        and finite_solution
        and gradient_ok
    )

    return {
        "optimizer_success": bool(result.success),
        "finite_solution": finite_solution,
        "gradient_inf_norm": gradient_inf_norm,
        "gradient_ok": gradient_ok,
        "certified": certified,
        "iterations": int(result.nit),
        "function_evaluations": int(result.nfev),
        "message": str(result.message),
    }


# ============================================================================
# 9. MULTI-START RGCOV
# ============================================================================

def fit_rgcov(
    Y: Array,
    p: int,
    H: int,
    transforms: Sequence[Transform],
    delta: float = 0.0,
```

```
    diagonal: bool = False,
    n_starts: int = 8,
    perturbation_scale: float = 0.02,
    seed: int = 123,
    maxiter: int = 5000,
) -> FitResult:
    """
    Fit GCov/RGCov by OLS warm start plus perturbations.

    This version uses finite-difference gradients through scipy.
    """
    Y = validate_panel(Y)
    n = Y.shape[1]

    ols_fit = fit_ols(Y, p)
    theta_ols = ols_fit.theta

    rng = np.random.default_rng(seed)

    scale = perturbation_scale * max(
        np.std(theta_ols),
        1e-8,
    )

    starts = [theta_ols]

    for _ in range(n_starts - 1):
        starts.append(
            theta_ols
            + rng.normal(
                0.0,
                scale,
                size=theta_ols.shape,
            )
        )

    objective = lambda theta: rgcov_objective(
        theta=theta,
        Y=Y,
        p=p,
        H=H,
        transforms=transforms,
        delta=delta,
        diagonal=diagonal,
    )

    diagnostics = []
    best_result = None

    for start_id, theta0 in enumerate(starts):
        result = minimize(
            objective,
            theta0,
            method="L-BFGS-B",
            options={
                "maxiter": maxiter,
                "ftol": 1e-12,
                "gtol": 1e-8,
                "maxls": 50,
            },
        )
```

```
            certificate = convergence_certificate(result)

            diagnostics.append(
                {
                    "start": start_id,
                    "start_type": (
                        "OLS"
                        if start_id == 0
                        else "OLS_plus_perturbation"
                    ),
                    "objective": float(result.fun),
                    **certificate,
                }
            )

            if (
                best_result is None
                or result.fun < best_result.fun
            ):
                best_result = result

        if best_result is None:
            raise RuntimeError(
                "No optimization result was produced."
            )

        Phi_hat = unstack_theta(
            best_result.x,
            n=n,
            p=p,
        )

        return FitResult(
            theta=best_result.x,
            Phi_list=Phi_hat,
            objective=float(best_result.fun),
            delta=delta,
            diagonal=diagonal,
            diagnostics=diagnostics,
        )


    # ============================================================================
    # 10. COMPANION ROOTS
    # ============================================================================

    def companion_matrix(
        Phi_list: Sequence[Array],
    ) -> Array:
        """
        Companion matrix convention:

            |lambda| < 1: stable companion dynamics
            |lambda| > 1: explosive companion dynamics
        """
        if len(Phi_list) == 0:
            raise ValueError("Phi_list cannot be empty.")

        n = Phi_list[0].shape[0]
        p = len(Phi_list)
```

```
    top = np.column_stack(Phi_list)

    if p == 1:
        return top

    bottom = np.hstack(
        [
            np.eye(n * (p - 1)),
            np.zeros((n * (p - 1), n)),
        ]
    )

    return np.vstack([top, bottom])


def companion_eigenvalues(
    Phi_list: Sequence[Array],
) -> Array:
    return np.linalg.eigvals(
        companion_matrix(Phi_list)
    )


def classify_roots(
    Phi_list: Sequence[Array],
    tolerance: float = 1e-6,
) -> dict[str, Any]:
    eigenvalues = companion_eigenvalues(Phi_list)
    moduli = np.abs(eigenvalues)

    causal = moduli < 1.0 - tolerance
    noncausal = moduli > 1.0 + tolerance
    near_unit = ~(causal | noncausal)

    return {
        "eigenvalues": eigenvalues,
        "moduli": moduli,
        "causal_count": int(causal.sum()),
        "noncausal_count": int(noncausal.sum()),
        "near_unit_count": int(near_unit.sum()),
        "spectral_radius": float(moduli.max()),
        "minimum_distance_to_unit_circle": float(
            np.min(np.abs(moduli - 1.0))
        ),
    }


# ============================================================================
# 11. DELTA SENSITIVITY
# ============================================================================

def delta_sensitivity(
    Y: Array,
    p: int = 1,
    H: int = 5,
    delta_grid: Sequence[float] = (
        0.0,
        0.001,
        0.01,
        0.05,
```

```
            0.1,
            0.2,
            0.5,
            1.0,
        ),
        diagonal: bool = False,
    ) -> pd.DataFrame:
        transforms = make_power_transforms((1, 2))
        records = []

        for delta in delta_grid:
            fit = fit_rgcov(
                Y=Y,
                p=p,
                H=H,
                transforms=transforms,
                delta=delta,
                diagonal=diagonal,
                n_starts=6,
                seed=123,
                maxiter=3000,
            )

            roots = classify_roots(
                fit.Phi_list
            )

            records.append(
                {
                    "delta": delta,
                    "objective": fit.objective,
                    "spectral_radius": roots[
                        "spectral_radius"
                    ],
                    "causal_count": roots[
                        "causal_count"
                    ],
                    "noncausal_count": roots[
                        "noncausal_count"
                    ],
                    "distance_to_unit": roots[
                        "minimum_distance_to_unit_circle"
                    ],
                    "certified": any(
                        d["certified"]
                        for d in fit.diagnostics
                    ),
                }
            )

        return pd.DataFrame(records)


    # ============================================================================
    # 12. DIAGNOSTIC TESTS
    # ============================================================================

    def ljung_box_table(
        U: Array,
        lag: int = 10,
    ) -> pd.DataFrame:
```

```
        records = []

        for j in range(U.shape[1]):
            result = acorr_ljungbox(
                U[:, j],
                lags=[lag],
                return_df=True,
            )

            records.append(
                {
                    "series": j,
                    "lag": lag,
                    "lb_stat": float(
                        result["lb_stat"].iloc[0]
                    ),
                    "p_value": float(
                        result["lb_pvalue"].iloc[0]
                    ),
                }
            )

        return pd.DataFrame(records)


    def portmanteau_test(
        fit: FitResult,
        Y: Array,
        p: int,
        H: int,
        transforms: Sequence[Transform],
    ) -> dict[str, Any]:
        """
        Naive chi-square diagnostic.

        For fixed positive delta, use bootstrap or a weighted
        chi-square approximation in the final paper.
        """
        U = var_residuals(
            Y=Y,
            p=p,
            Phi_list=fit.Phi_list,
        )

        V = transform_residuals(
            U,
            transforms,
            standardize=True,
        )

        Gamma0 = sample_autocov(V, 0)
        Gamma0 = 0.5 * (Gamma0 + Gamma0.T)

        try:
            factor = cho_factor(
                Gamma0,
                lower=True,
                check_finite=False,
            )
        except np.linalg.LinAlgError:
            return {
```

```
                "statistic": np.nan,
                "df_naive": np.nan,
                "p_value_naive": np.nan,
                "warning": (
                    "Gamma0 is not positive definite."
                ),
            }

        q = 0.0

        for h in range(1, H + 1):
            Gamma_h = sample_autocov(V, h)

            whitened = cho_solve(
                factor,
                Gamma_h,
                check_finite=False,
            )

            q += np.sum(whitened ** 2)

        T_eff = len(U)
        K = V.shape[1]
        parameter_count = len(fit.theta)

        statistic = T_eff * q
        degrees_of_freedom = (
            K * K * H - parameter_count
        )

        if degrees_of_freedom > 0:
            p_value = float(
                1.0
                - chi2.cdf(
                    statistic,
                    degrees_of_freedom,
                )
            )
        else:
            p_value = np.nan

        return {
            "statistic": float(statistic),
            "df_naive": int(degrees_of_freedom),
            "p_value_naive": p_value,
            "delta": fit.delta,
            "warning": (
                "For fixed positive delta, use bootstrap "
                "or weighted-chi-square inference."
            ),
        }


    # ============================================================================
    # 13. SIMULATORS
    # ============================================================================

    def simulate_var(
        T: int,
        n: int = 2,
        p: int = 1,
```

```
    Phi_list: Optional[Sequence[Array]] = None,
    innovation: str = "gaussian",
    df: int = 5,
    burn: int = 1000,
    seed: int = 123,
) -> Array:
    """
    Simulate a causal VAR.
    """
    rng = np.random.default_rng(seed)

    if Phi_list is None:
        Phi_list = [
            0.5 * np.eye(n)
            for _ in range(p)
        ]

    if len(Phi_list) != p:
        raise ValueError(
            "Phi_list length must equal p."
        )

    total = T + burn
    Y = np.zeros((total, n))

    if innovation == "gaussian":
        E = rng.normal(size=(total, n))
    elif innovation == "student_t":
        E = rng.standard_t(
            df=df,
            size=(total, n),
        )
    else:
        raise ValueError(
            "innovation must be gaussian or student_t."
        )

    for t in range(p, total):
        current = np.zeros(n)

        for j in range(p):
            current += (
                Phi_list[j]
                @ Y[t - j - 1]
            )

        Y[t] = current + E[t]

    return Y[burn:]


def simulate_mixed_diagonal(
    T: int,
    roots: Sequence[float] = (0.9, 1.2),
    df: int = 5,
    burn: int = 1000,
    seed: int = 123,
) -> Array:
    """
    Simulate a diagonal mixed causal–noncausal process.
```

```
    Roots with modulus below one are simulated forward.
    Roots with modulus above one are simulated backward.
    """
    rng = np.random.default_rng(seed)

    n = len(roots)
    total = T + 2 * burn

    E = rng.standard_t(
        df=df,
        size=(total, n),
    )

    X = np.zeros((total, n))

    for j, root in enumerate(roots):
        if abs(root) < 1.0:
            for t in range(1, total):
                X[t, j] = (
                    root * X[t - 1, j]
                    + E[t, j]
                )
        else:
            for t in range(total - 2, -1, -1):
                X[t, j] = (
                    X[t + 1, j]
                    - E[t + 1, j]
                ) / root

    return X[burn:burn + T]


# ============================================================================
# 14. RECURSIVE RESIDUAL BOOTSTRAP
# ============================================================================

def recursive_residual_bootstrap(
    Y: Array,
    Phi_list: Sequence[Array],
    residuals: Array,
    B: int = 500,
    seed: int = 123,
) -> list[Array]:
    """
    Generate bootstrap series recursively from a fitted causal VAR.
    """
    Y = validate_panel(Y)
    residuals = np.asarray(
        residuals,
        dtype=float,
    )

    T, n = Y.shape
    p = len(Phi_list)

    centered_residuals = (
        residuals
        - residuals.mean(axis=0, keepdims=True)
    )

    rng = np.random.default_rng(seed)
```

```
        bootstrap_samples = []

        for _ in range(B):
            innovations = centered_residuals[
                rng.integers(
                    0,
                    len(centered_residuals),
                    size=T,
                )
            ]

            Y_boot = np.zeros_like(Y)

            for t in range(p, T):
                current = np.zeros(n)

                for j in range(p):
                    current += (
                        Phi_list[j]
                        @ Y_boot[t - j - 1]
                    )

                Y_boot[t] = current + innovations[t]

            bootstrap_samples.append(Y_boot)

        return bootstrap_samples


    def bootstrap_spectral_radius(
        Y: Array,
        fit: FitResult,
        p: int,
        B: int = 500,
        seed: int = 123,
    ) -> dict[str, Any]:
        """
        Bootstrap OLS spectral-radius uncertainty.
        """
        U = var_residuals(
            Y=Y,
            p=p,
            Phi_list=fit.Phi_list,
        )

        samples = recursive_residual_bootstrap(
            Y=Y,
            Phi_list=fit.Phi_list,
            residuals=U,
            B=B,
            seed=seed,
        )

        radii = []
        noncausal_counts = []

        for sample in samples:
            boot_fit = fit_ols(
                sample,
                p,
            )
```

```
            roots = classify_roots(
                boot_fit.Phi_list
            )

            radii.append(
                roots["spectral_radius"]
            )

            noncausal_counts.append(
                roots["noncausal_count"]
            )

        radii = np.asarray(radii)
        noncausal_counts = np.asarray(
            noncausal_counts
        )

        return {
            "mean": float(radii.mean()),
            "std": float(radii.std(ddof=1)),
            "q025": float(np.quantile(radii, 0.025)),
            "q050": float(np.quantile(radii, 0.50)),
            "q975": float(np.quantile(radii, 0.975)),
            "probability_any_noncausal": float(
                np.mean(noncausal_counts > 0)
            ),
            "spectral_radius": radii,
            "noncausal_counts": noncausal_counts,
        }


    # ============================================================================
    # 15. ROOT-RECOVERY MONTE CARLO
    # ============================================================================

    def evaluate_root_recovery(
        replications: int = 100,
        T: int = 400,
        roots: Sequence[float] = (0.9, 1.2),
        H: int = 5,
        delta: float = 0.1,
    ) -> pd.DataFrame:
        """
        Repeated mixed-root recovery experiment.
        """
        records = []

        transforms = make_power_transforms(
            powers=(1, 2)
        )

        true_moduli = np.sort(
            np.abs(np.asarray(roots))
        )

        for replication in range(replications):
            Y = simulate_mixed_diagonal(
                T=T,
                roots=roots,
                df=5,
```

```
                seed=1000 + replication,
            )

            fit = fit_rgcov(
                Y=Y,
                p=1,
                H=H,
                transforms=transforms,
                delta=delta,
                n_starts=6,
                seed=2000 + replication,
                maxiter=3000,
            )

            roots_hat = classify_roots(
                fit.Phi_list
            )

            estimated_moduli = np.sort(
                roots_hat["moduli"]
            )

            certified = any(
                d["certified"]
                for d in fit.diagnostics
            )

            records.append(
                {
                    "replication": replication,
                    "certified": certified,
                    "correct_root_count": (
                        roots_hat["causal_count"] == 1
                        and roots_hat["noncausal_count"] == 1
                    ),
                    "root_modulus_error": float(
                        np.mean(
                            np.abs(
                                estimated_moduli
                                - true_moduli
                            )
                        )
                    ),
                    "spectral_radius": roots_hat[
                        "spectral_radius"
                    ],
                    "objective": fit.objective,
                }
            )

        return pd.DataFrame(records)


    # ============================================================================
    # 16. GENERAL MONTE CARLO
    # ============================================================================

    def monte_carlo_experiment(
        replications: int = 100,
        T: int = 250,
        p: int = 1,
```

```
    n: int = 2,
    H: int = 3,
    delta: float = 0.1,
    seed: int = 123,
) -> pd.DataFrame:
    """
    Compare OLS, Lasso, and RGCov.
    """
    rng = np.random.default_rng(seed)

    transforms = make_power_transforms(
        powers=(1, 2)
    )

    records = []

    for replication in range(replications):
        simulation_seed = int(
            rng.integers(
                0,
                2**32 - 1,
            )
        )

        Y = simulate_var(
            T=T,
            n=n,
            p=p,
            innovation="student_t",
            df=5,
            seed=simulation_seed,
        )

        fits = {
            "OLS": fit_ols(Y, p),
            "Lasso": fit_lasso(
                Y,
                p,
                alpha=0.01,
            ),
            "RGCov": fit_rgcov(
                Y,
                p,
                H,
                transforms,
                delta=delta,
                n_starts=4,
                seed=replication,
                maxiter=1000,
            ),
        }

        for estimator_name, fit in fits.items():
            roots = classify_roots(
                fit.Phi_list
            )

            if estimator_name == "RGCov":
                converged = any(
                    d["certified"]
                    for d in fit.diagnostics
```

```
                )
            else:
                converged = True

            records.append(
                {
                    "replication": replication,
                    "estimator": estimator_name,
                    "spectral_radius": roots[
                        "spectral_radius"
                    ],
                    "noncausal_count": roots[
                        "noncausal_count"
                    ],
                    "converged": converged,
                }
            )

    return pd.DataFrame(records)


# ============================================================================
# 17. REPORTING
# ============================================================================

def print_fit_report(
    name: str,
    fit: FitResult,
) -> None:
    roots = classify_roots(
        fit.Phi_list
    )

    print(f"\n--- {name} ---")
    print("Objective:", fit.objective)
    print("Delta:", fit.delta)
    print("Diagonal:", fit.diagonal)
    print(
        "Spectral radius:",
        roots["spectral_radius"],
    )
    print(
        "Causal roots:",
        roots["causal_count"],
    )
    print(
        "Noncausal roots:",
        roots["noncausal_count"],
    )

    if fit.diagnostics:
        print("\nOptimization diagnostics:")

        for diagnostic in fit.diagnostics:
            print(diagnostic)


# ============================================================================
# 18. MAIN DEMONSTRATION
# ============================================================================
```

```
def main() -> None:
    print(
        "Simulating a bivariate mixed causal–noncausal process..."
    )

    Y = simulate_mixed_diagonal(
        T=400,
        roots=(0.9, 1.2),
        df=5,
        seed=1,
    )

    print(
        "Mixed sample shape:",
        Y.shape,
    )

    transforms = make_power_transforms(
        powers=(1, 2)
    )

    print("\nFitting OLS benchmark...")

    ols_fit = fit_ols(
        Y,
        p=1,
    )

    print_fit_report(
        "OLS",
        ols_fit,
    )

    print("\nFitting Lasso benchmark...")

    lasso_fit = fit_lasso(
        Y,
        p=1,
        alpha=0.01,
    )

    print_fit_report(
        "Lasso",
        lasso_fit,
    )

    print("\nFitting RGCov...")

    rgcov_fit = fit_rgcov(
        Y=Y,
        p=1,
        H=5,
        transforms=transforms,
        delta=0.1,
        diagonal=False,
        n_starts=4,
        seed=1,
        maxiter=1000,
    )

    print_fit_report(
```

```
    "RGCov",
    rgcov_fit,
)

print("\nRGCov portmanteau diagnostic...")

print(
    portmanteau_test(
        fit=rgcov_fit,
        Y=Y,
        p=1,
        H=5,
        transforms=transforms,
    )
)

print("\nOLS residual diagnostic...")

OLS_residuals = var_residuals(
    Y=Y,
    p=1,
    Phi_list=ols_fit.Phi_list,
)

print(
    ljung_box_table(
        OLS_residuals,
        lag=10,
    )
)

print("\nBootstrap OLS root uncertainty...")

bootstrap = bootstrap_spectral_radius(
    Y=Y,
    fit=ols_fit,
    p=1,
    B=100,
    seed=1,
)

print(
    {
        key: value
        for key, value in bootstrap.items()
        if key not in {
            "spectral_radius",
            "noncausal_counts",
        }
    }
)

print("\nDelta sensitivity...")

print(
    delta_sensitivity(
        Y,
        p=1,
        H=5,
    )
)
```

```
        print("\nRoot-recovery Monte Carlo...")

        recovery = evaluate_root_recovery(
            replications=20,
            T=400,
            roots=(0.9, 1.2),
            H=5,
            delta=0.1,
        )

        print(
            recovery.mean(
                numeric_only=True
            )
        )

        print("\nGeneral Monte Carlo...")

        monte_carlo = monte_carlo_experiment(
            replications=10,
            T=250,
            p=1,
            n=2,
            H=3,
            delta=0.1,
            seed=1,
        )

        print(
            monte_carlo
            .groupby("estimator")
            .mean(numeric_only=True)
        )


    if __name__ == "__main__":
        main()
```

```
1  0.001   0.307982         1.194809             1                1
2  0.010   0.273137         1.195276             1                1
3  0.050   0.179518         1.195710             1                1
4  0.100   0.124957         1.195676             1                1
5  0.200   0.078916         1.196442             1                1
6  0.500   0.049379         0.877122             2                0
7  1.000   0.025401         0.893905             2                0

   distance_to_unit  certified
0          0.133796       True
1          0.133810       True
2          0.133890       True
3          0.133692       True
4          0.132924       True
5          0.131092       True
6          0.122878       True
7          0.106095       True

Root-recovery Monte Carlo...
replication           9.500000
certified             1.000000
correct_root_count    0.650000
root_modulus_error    0.096311
spectral_radius       1.139889
objective             0.216820
dtype: float64

General Monte Carlo...
           replication  spectral_radius  noncausal_count  converged
estimator
Lasso              4.5         0.528526              0.0        1.0
OLS                4.5         0.533392              0.0        1.0
RGCov              4.5         0.690234              0.1        1.0
```

```
"""
RGCov: Publication-Ready Research Workflow
==========================================
Install:
    pip install numpy scipy pandas scikit-learn statsmodels tqdm

Run:
    python rgcov_all_code.py

Notes:
    - The ridge penalty (lambda) and box constraints (|beta| <= 5) are now
      explicitly enforced as defined in Section 3.1 and 5.1 of the paper.
    - The objective function is fully vectorized for massive speedups.
    - For final submission, consider replacing scipy's finite-difference
      gradients with JAX/PyTorch autodiff for exact analytic gradients.
"""

from __future__ import annotations
import logging
from dataclasses import dataclass, field
from typing import Any, Callable, Optional, Sequence

import numpy as np
import pandas as pd
from tqdm import tqdm

from scipy.linalg import cho_factor, cho_solve
from scipy.optimize import minimize
from scipy.stats import chi2
```

```
from sklearn.linear_model import MultiTaskLasso
from statsmodels.stats.diagnostic import acorr_ljungbox

# Configure logging for optimization diagnostics
logging.basicConfig(level=logging.INFO, format='%(message)s')
logger = logging.getLogger(__name__)

Array = np.ndarray
Transform = Callable[[Array], Array]

# ============================================================================
# 1. DATA & VAR UTILITIES
# ============================================================================

def validate_panel(Y: Array) -> Array:
    Y = np.asarray(Y, dtype=float)
    if Y.ndim != 2 or Y.shape[0] < 20 or Y.shape[1] < 1:
        raise ValueError("Y must have shape (T, n) with T >= 20, n >= 1.")
    if not np.all(np.isfinite(Y)):
        raise ValueError("Y contains non-finite values.")
    return Y

def build_lagged(Y: Array, p: int) -> tuple[Array, Array]:
    Y = validate_panel(Y)
    T, n = Y.shape
    if p < 1 or p >= T:
        raise ValueError("p must satisfy 1 <= p < T.")
    Ycur = Y[p:]
    X = np.column_stack([Y[p - lag - 1:T - lag - 1] for lag in range(p)])
    return X, Ycur

def stack_theta(Phi_list: Sequence[Array]) -> Array:
    return np.concatenate([Phi.ravel(order="C") for Phi in Phi_list])

def unstack_theta(theta: Array, n: int, p: int) -> list[Array]:
    return [theta[j * n * n:(j + 1) * n * n].reshape(n, n) for j in range(p)]

def var_residuals(Y: Array, p: int, Phi_list: Sequence[Array]) -> Array:
    X, Ycur = build_lagged(Y, p)
    Phi_stack = np.column_stack(Phi_list)
    return Ycur - X @ Phi_stack.T

# ============================================================================
# 2. FIT RESULT & BENCHMARKS
# ============================================================================

@dataclass
class FitResult:
    theta: Array
    Phi_list: list[Array]
    objective: float
    delta: float = 0.0
    ridge_lambda: float = 0.0
    diagonal: bool = False
    diagnostics: list[dict[str, Any]] = field(default_factory=list)

def fit_ols(Y: Array, p: int, ridge: float = 0.0) -> FitResult:
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]
    XtX = X.T @ X
```

```
    if ridge > 0:
        XtX += ridge * np.eye(X.shape[1])
    B = np.linalg.solve(XtX, X.T @ Ycur).T
    Phi_list = [B[:, j * n:(j + 1) * n] for j in range(p)]
    mse = float(np.mean((Ycur - X @ B.T) ** 2))
    return FitResult(theta=stack_theta(Phi_list), Phi_list=Phi_list, objective=mse)

def fit_lasso(Y: Array, p: int, alpha: float = 0.01) -> FitResult:
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]
    model = MultiTaskLasso(alpha=alpha, fit_intercept=False, max_iter=10000, tol=1e-8)
    model.fit(X, Ycur)
    B = model.coef_
    Phi_list = [B[:, j * n:(j + 1) * n] for j in range(p)]
    mse = float(np.mean((Ycur - model.predict(X)) ** 2))
    return FitResult(theta=stack_theta(Phi_list), Phi_list=Phi_list, objective=mse)

# ============================================================================
# 3. RGCOV OBJECTIVE (VECTORIZED & WITH RIDGE PENALTY)
# ============================================================================

def make_power_transforms(powers: Sequence[int] = (1, 2)) -> list[Transform]:
    return [lambda U, p=p: np.sign(U) * np.abs(U) ** p for p in powers]

def rgcov_objective(
    theta: Array, Y: Array, p: int, H: int, transforms: Sequence[Transform],
    delta: float = 0.0, diagonal: bool = False, ridge_lambda: float = 0.0
) -> float:
    """
    Vectorized RGCov objective with explicit ridge penalty.
    Q(theta) = sum_h || Gamma_0^{-1/2} Gamma_h Gamma_0^{-1/2} ||_F^2 + lambda * ||theta||^2
    """
    n = Y.shape[1]
    Phi_list = unstack_theta(theta, n, p)
    X, Ycur = build_lagged(Y, p)
    U = Ycur - X @ np.column_stack(Phi_list).T

    # Apply transforms and standardize
    V = np.column_stack([t(U) for t in transforms])
    V = (V - V.mean(axis=0, keepdims=True)) / (V.std(axis=0, ddof=1, keepdims=True) + 1e-10)

    T_eff = V.shape[0]
    V_c = V - V.mean(axis=0, keepdims=True)

    # Gamma_0 and regularization
    Gamma0 = (V_c.T @ V_c) / T_eff
    Gamma0 = 0.5 * (Gamma0 + Gamma0.T)
    if diagonal:
        Gamma0 = np.diag(np.diag(Gamma0))
    Gamma0 += delta * np.eye(Gamma0.shape[0])

    try:
        # Cholesky whitening for numerical stability
        L = np.linalg.cholesky(Gamma0)
        L_inv = np.linalg.solve(L, np.eye(L.shape[0]))
        W = L_inv.T @ L_inv  # This is Gamma_0^{-1}
    except np.linalg.LinAlgError:
        return 1e100

    obj = 0.0
    for h in range(1, H + 1):
```

```
            Gamma_h = (V_c[h:].T @ V_c[:-h]) / T_eff
            W_Gamma_h = W @ Gamma_h
            # trace(W Gamma_h W Gamma_h') = || W^{1/2} Gamma_h W^{1/2} ||_F^2
            obj += np.sum(W_Gamma_h * W_Gamma_h.T)

        # CRITICAL FIX: Add the ridge penalty defined in Section 3.1
        if ridge_lambda > 0:
            obj += ridge_lambda * np.sum(theta ** 2)

        return obj if np.isfinite(obj) else 1e100

    # ============================================================================
    # 4. MULTI-START OPTIMIZER (WITH BOUNDS)
    # ============================================================================

    def fit_rgcov(
        Y: Array, p: int, H: int, transforms: Sequence[Transform],
        delta: float = 0.1, diagonal: bool = False, ridge_lambda: float = 0.01,
        n_starts: int = 8, perturbation_scale: float = 0.02, seed: int = 123, maxiter: int = 5000
    ) -> FitResult:
        Y = validate_panel(Y)
        n = Y.shape[1]
        ols_fit = fit_ols(Y, p)
        theta_ols = ols_fit.theta

        rng = np.random.default_rng(seed)
        scale = perturbation_scale * max(np.std(theta_ols), 1e-8)

        # Multi-start initialization
        starts = [theta_ols] + [
            theta_ols + rng.normal(0.0, scale, size=theta_ols.shape)
            for _ in range(n_starts - 1)
        ]

        # CRITICAL FIX: Enforce box constraints |beta_ijk| <= 5 (Section 5.1)
        bounds = [(-5.0, 5.0)] * len(theta_ols)

        objective = lambda th: rgcov_objective(
            th, Y, p, H, transforms, delta, diagonal, ridge_lambda
        )

        diagnostics = []
        best_result = None

        for start_id, theta0 in enumerate(starts):
            result = minimize(
                objective, theta0, method="L-BFGS-B", bounds=bounds,
                options={"maxiter": maxiter, "ftol": 1e-12, "gtol": 1e-8, "maxls": 50}
            )

            grad_norm = float(np.linalg.norm(result.jac, ord=np.inf)) if result.jac is not None else
            cert = bool(result.success and np.isfinite(result.fun) and grad_norm <= 1e-6)

            diagnostics.append({
                "start": start_id,
                "start_type": "OLS" if start_id == 0 else "OLS_plus_perturbation",
                "objective": float(result.fun),
                "optimizer_success": bool(result.success),
                "gradient_inf_norm": grad_norm,
                "certified": cert,
                "message": str(result.message)
```

```
            })

            if best_result is None or result.fun < best_result.fun:
                best_result = result

        return FitResult(
            theta=best_result.x, Phi_list=unstack_theta(best_result.x, n, p),
            objective=float(best_result.fun), delta=delta, ridge_lambda=ridge_lambda,
            diagonal=diagonal, diagnostics=diagnostics
        )

    # ============================================================================
    # 5. ROOTS, SIMULATORS & DIAGNOSTICS
    # ============================================================================

    def classify_roots(Phi_list: Sequence[Array], tol: float = 1e-6) -> dict[str, Any]:
        if len(Phi_list) == 1:
            eigvals = np.linalg.eigvals(Phi_list[0])
        else:
            n = Phi_list[0].shape[0]
            top = np.column_stack(Phi_list)
            bottom = np.hstack([np.eye(n * (len(Phi_list) - 1)), np.zeros((n * (len(Phi_list) - 1), r
            eigvals = np.linalg.eigvals(np.vstack([top, bottom]))

        moduli = np.abs(eigvals)
        return {
            "moduli": moduli,
            "causal_count": int(np.sum(moduli < 1.0 - tol)),
            "noncausal_count": int(np.sum(moduli > 1.0 + tol)),
            "spectral_radius": float(moduli.max()),
            "distance_to_unit": float(np.min(np.abs(moduli - 1.0)))
        }

    def simulate_mixed_diagonal(T: int, roots: Sequence[float] = (0.9, 1.2), df: int = 5, burn: int =
        rng = np.random.default_rng(seed)
        n = len(roots)
        total = T + 2 * burn
        E = rng.standard_t(df=df, size=(total, n))
        X = np.zeros((total, n))

        for j, root in enumerate(roots):
            if abs(root) < 1.0:
                for t in range(1, total):
                    X[t, j] = root * X[t - 1, j] + E[t, j]
            else:
                # Backward simulation for noncausal root
                for t in range(total - 2, -1, -1):
                    X[t, j] = (X[t + 1, j] - E[t + 1, j]) / root
        return X[burn:burn + T]

    def portmanteau_test(fit: FitResult, Y: Array, p: int, H: int, transforms: Sequence[Transform]) -
        U = var_residuals(Y, p, fit.Phi_list)
        V = np.column_stack([t(U) for t in transforms])
        V = (V - V.mean(axis=0)) / (V.std(axis=0, ddof=1) + 1e-10)
        T_eff = len(U)
        V_c = V - V.mean(axis=0)
        Gamma0 = (V_c.T @ V_c) / T_eff + fit.delta * np.eye(V.shape[1])

        try:
            factor = cho_factor(Gamma0, lower=True)
            q = sum(np.sum(cho_solve(factor, (V_c[h:].T @ V_c[:-h]) / T_eff) ** 2) for h in range(1,
```

```
            stat = T_eff * q
            df = V.shape[1]**2 * H - len(fit.theta)
            p_val = 1.0 - chi2.cdf(stat, df) if df > 0 else np.nan
            return {"statistic": stat, "df_naive": df, "p_value_naive": p_val, "delta": fit.delta}
        except np.linalg.LinAlgError:
            return {"statistic": np.nan, "df_naive": np.nan, "p_value_naive": np.nan}

    # ============================================================================
    # 6. MONTE CARLO & BOOTSTRAP (WITH PROGRESS BARS)
    # ============================================================================

    def evaluate_root_recovery(replications: int = 100, T: int = 400, roots: Sequence[float] = (0.9,
        transforms = make_power_transforms((1, 2))
        true_moduli = np.sort(np.abs(roots))
        records = []

        for rep in tqdm(range(replications), desc="Root-recovery MC"):
            Y = simulate_mixed_diagonal(T=T, roots=roots, df=5, seed=1000 + rep)
            fit = fit_rgcov(Y, p=1, H=H, transforms=transforms, delta=delta, ridge_lambda=ridge_lambc
            roots_hat = classify_roots(fit.Phi_list)
            est_moduli = np.sort(roots_hat["moduli"])

            records.append({
                "replication": rep,
                "certified": any(d["certified"] for d in fit.diagnostics),
                "correct_root_count": (roots_hat["causal_count"] == 1 and roots_hat["noncausal_count"
                "root_modulus_error": float(np.mean(np.abs(est_moduli - true_moduli))),
                "spectral_radius": roots_hat["spectral_radius"],
                "objective": fit.objective
            })
        return pd.DataFrame(records)

    def bootstrap_spectral_radius(Y: Array, fit: FitResult, p: int, B: int = 500, seed: int = 123) ->
        U = var_residuals(Y, p, fit.Phi_list)
        U_c = U - U.mean(axis=0)
        T, n = Y.shape
        rng = np.random.default_rng(seed)
        radii, noncausal_counts = [], []

        for _ in tqdm(range(B), desc="Bootstrap MC"):
            innov = U_c[rng.integers(0, len(U_c), size=T)]
            Y_boot = np.zeros_like(Y)
            for t in range(p, T):
                Y_boot[t] = sum(fit.Phi_list[j] @ Y_boot[t - j - 1] for j in range(p)) + innov[t]

            boot_fit = fit_ols(Y_boot, p)
            roots = classify_roots(boot_fit.Phi_list)
            radii.append(roots["spectral_radius"])
            noncausal_counts.append(roots["noncausal_count"])

        radii = np.array(radii)
        return {
            "mean": float(radii.mean()), "std": float(radii.std(ddof=1)),
            "q025": float(np.quantile(radii, 0.025)), "q975": float(np.quantile(radii, 0.975)),
            "probability_any_noncausal": float(np.mean(np.array(noncausal_counts) > 0))
        }

    # ============================================================================
    # 7. MAIN EXECUTION
    # ============================================================================
```

```
if __name__ == "__main__":
    print("=== RGCov Publication-Ready Workflow ===\n")

    # 1. Simulate Mixed DGP
    Y = simulate_mixed_diagonal(T=400, roots=(0.9, 1.2), df=5, seed=1)
    transforms = make_power_transforms((1, 2))

    # 2. Fit Benchmarks
    print("Fitting OLS & Lasso...")
    ols_fit = fit_ols(Y, p=1)
    lasso_fit = fit_lasso(Y, p=1, alpha=0.01)

    # 3. Fit RGCov (with Ridge and Bounds)
    print("Fitting RGCov (Multi-start with bounds)...")
    rgcov_fit = fit_rgcov(Y, p=1, H=5, transforms=transforms, delta=0.1, ridge_lambda=0.01, n_sta

    print(f"\n--- RGCov Results ---")
    print(f"Objective: {rgcov_fit.objective:.6f}")
    print(f"Certified: {any(d['certified'] for d in rgcov_fit.diagnostics)}")
    roots = classify_roots(rgcov_fit.Phi_list)
    print(f"Spectral Radius: {roots['spectral_radius']:.4f}")
    print(f"Causal / Noncausal Roots: {roots['causal_count']} / {roots['noncausal_count']}")

    # 4. Quick Root Recovery MC (Reduce replications for demo)
    print("\nRunning quick Root-Recovery Monte Carlo (20 reps)...")
    mc_results = evaluate_root_recovery(replications=20, T=400, roots=(0.9, 1.2), H=5, delta=0.1,
    print("\nMC Summary:")
    print(mc_results.mean(numeric_only=True))
```

```
=== RGCov Publication-Ready Workflow ===

Fitting OLS & Lasso...
Fitting RGCov (Multi-start with bounds)...

--- RGCov Results ---
Objective: -0.192226
Certified: True
Spectral Radius: 1.0181
Causal / Noncausal Roots: 0 / 2

Running quick Root-Recovery Monte Carlo (20 reps)...
Root-recovery MC: 100%|██████████| 20/20 [00:07<00:00,  2.55it/s]
MC Summary:
replication           9.500000
certified             1.000000
correct_root_count    0.000000
root_modulus_error    0.150000
spectral_radius       0.991426
objective            -0.102080
dtype: float64
```

```
import logging
from dataclasses import dataclass, field
from typing import Any, Callable, Sequence

import numpy as np
import pandas as pd
from tqdm import tqdm

from scipy.optimize import minimize
from scipy.stats import chi2
```

```
from sklearn.linear_model import MultiTaskLasso
from statsmodels.stats.diagnostic import acorr_ljungbox

logging.basicConfig(level=logging.INFO, format='%(message)s')
logger = logging.getLogger(__name__)

Array = np.ndarray
Transform = Callable[[Array], Array]

# ============================================================================
# 1. DATA & VAR UTILITIES
# ============================================================================

def validate_panel(Y: Array) -> Array:
    Y = np.asarray(Y, dtype=float)
    if Y.ndim != 2 or Y.shape[0] < 20 or Y.shape[1] < 1:
        raise ValueError("Y must have shape (T, n) with T >= 20, n >= 1.")
    if not np.all(np.isfinite(Y)):
        raise ValueError("Y contains non-finite values.")
    return Y

def build_lagged(Y: Array, p: int) -> tuple[Array, Array]:
    Y = validate_panel(Y)
    T, n = Y.shape
    Ycur = Y[p:]
    X = np.column_stack([Y[p - lag - 1:T - lag - 1] for lag in range(p)])
    return X, Ycur

def stack_theta(Phi_list: Sequence[Array]) -> Array:
    return np.concatenate([Phi.ravel(order="C") for Phi in Phi_list])

def unstack_theta(theta: Array, n: int, p: int) -> list[Array]:
    return [theta[j * n * n:(j + 1) * n * n].reshape(n, n) for j in range(p)]

# ============================================================================
# 2. FIT RESULT & BENCHMARKS
# ============================================================================

@dataclass
class FitResult:
    theta: Array
    Phi_list: list[Array]
    objective: float
    delta: float = 0.0
    ridge_lambda: float = 0.0
    diagonal: bool = False  # Added missing 'diagonal' field
    diagnostics: list[dict[str, Any]] = field(default_factory=list)

def fit_ols(Y: Array, p: int, ridge: float = 0.0) -> FitResult:
    X, Ycur = build_lagged(Y, p)
    n = Y.shape[1]
    XtX = X.T @ X
    if ridge > 0:
        XtX += ridge * np.eye(X.shape[1])
    B = np.linalg.solve(XtX, X.T @ Ycur).T
    Phi_list = [B[:, j * n:(j + 1) * n] for j in range(p)]
    mse = float(np.mean((Ycur - X @ B.T) ** 2))
    return FitResult(theta=stack_theta(Phi_list), Phi_list=Phi_list, objective=mse)

def fit_lasso(Y: Array, p: int, alpha: float = 0.01) -> FitResult:
    X, Ycur = build_lagged(Y, p)
```

```
    n = Y.shape[1]
    model = MultiTaskLasso(alpha=alpha, fit_intercept=False, max_iter=10000, tol=1e-8)
    model.fit(X, Ycur)
    B = model.coef_
    Phi_list = [B[:, j * n:(j + 1) * n] for j in range(p)]
    mse = float(np.mean((Ycur - model.predict(X)) ** 2))
    return FitResult(theta=stack_theta(Phi_list), Phi_list=Phi_list, objective=mse)

# ============================================================================
# 3. RGCOV OBJECTIVE (BUG-FIXED & STRICTLY NON-NEGATIVE)
# ============================================================================

def make_power_transforms(powers: Sequence[int] = (1, 2)) -> list[Transform]:
    return [lambda U, p=p: np.sign(U) * np.abs(U) ** p for p in powers]

def rgcov_objective(
    theta: Array, Y: Array, p: int, H: int, transforms: Sequence[Transform],
    delta: float = 0.0, diagonal: bool = False, ridge_lambda: float = 0.0
) -> float:
    n = Y.shape[1]
    Phi_list = unstack_theta(theta, n, p)
    X, Ycur = build_lagged(Y, p)
    U = Ycur - X @ np.column_stack(Phi_list).T

    V = np.column_stack([t(U) for t in transforms])
    V = (V - V.mean(axis=0, keepdims=True)) / (V.std(axis=0, ddof=1, keepdims=True) + 1e-10)

    T_eff = V.shape[0]
    V_c = V - V.mean(axis=0, keepdims=True)

    Gamma0 = (V_c.T @ V_c) / T_eff
    Gamma0 = 0.5 * (Gamma0 + Gamma0.T)
    if diagonal:
        Gamma0 = np.diag(np.diag(Gamma0))
    Gamma0 += delta * np.eye(Gamma0.shape[0])

    try:
        L = np.linalg.cholesky(Gamma0)
    except np.linalg.LinAlgError:
        return 1e100

    obj = 0.0
    for h in range(1, H + 1):
        Gamma_h = (V_c[h:].T @ V_c[:-h]) / T_eff
        # Solve L * L^T * X = Gamma_h  =>  X = Gamma0^{-1} Gamma_h
        whitened = np.linalg.solve(L.T, np.linalg.solve(L, Gamma_h))
        # CORRECTED: Frobenius norm squared is sum of squared elements
        obj += np.sum(whitened ** 2)

    if ridge_lambda > 0:
        obj += ridge_lambda * np.sum(theta ** 2)

    return float(obj) if np.isfinite(obj) else 1e100

# ============================================================================
# 4. MULTI-START OPTIMIZER
# ============================================================================

def fit_rgcov(
    Y: Array, p: int, H: int, transforms: Sequence[Transform],
    delta: float = 0.1, diagonal: bool = False, ridge_lambda: float = 0.001,
```

```
    n_starts: int = 10, perturbation_scale: float = 0.1, seed: int = 123, maxiter: int = 5000
) -> FitResult:
    Y = validate_panel(Y)
    n = Y.shape[1]
    ols_fit = fit_ols(Y, p)
    theta_ols = ols_fit.theta

    rng = np.random.default_rng(seed)
    scale = perturbation_scale * max(np.std(theta_ols), 1e-8)

    starts = [theta_ols] + [
        theta_ols + rng.normal(0.0, scale, size=theta_ols.shape)
        for _ in range(n_starts - 1)
    ]

    bounds = [(-5.0, 5.0)] * len(theta_ols)
    objective = lambda th: rgcov_objective(th, Y, p, H, transforms, delta, diagonal, ridge_lambda

    diagnostics = []
    best_result = None

    for start_id, theta0 in enumerate(starts):
        result = minimize(
            objective, theta0, method="L-BFGS-B", bounds=bounds,
            options={"maxiter": maxiter, "ftol": 1e-12, "gtol": 1e-8, "maxls": 50}
        )

        grad_norm = float(np.linalg.norm(result.jac, ord=np.inf)) if result.jac is not None else
        cert = bool(result.success and np.isfinite(result.fun) and grad_norm <= 1e-6)

        diagnostics.append({
            "start": start_id, "start_type": "OLS" if start_id == 0 else "OLS_plus_perturbation",
            "objective": float(result.fun), "optimizer_success": bool(result.success),
            "gradient_inf_norm": grad_norm, "certified": cert
        })

        if best_result is None or result.fun < best_result.fun:
            best_result = result

    return FitResult(
        theta=best_result.x, Phi_list=unstack_theta(best_result.x, n, p),
        objective=float(best_result.fun), delta=delta, ridge_lambda=ridge_lambda,
        diagonal=diagonal, diagnostics=diagnostics
    )

# ============================================================================
# 5. ROOTS, SIMULATORS & MONTE CARLO
# ============================================================================

def classify_roots(Phi_list: Sequence[Array], tol: float = 1e-6) -> dict[str, Any]:
    if len(Phi_list) == 1:
        eigvals = np.linalg.eigvals(Phi_list[0])
    else:
        n = Phi_list[0].shape[0]
        top = np.column_stack(Phi_list)
        bottom = np.hstack([np.eye(n * (len(Phi_list) - 1)), np.zeros((n * (len(Phi_list) - 1), r
        eigvals = np.linalg.eigvals(np.vstack([top, bottom]))

    moduli = np.abs(eigvals)
    return {
        "moduli": moduli,
```

```
            "causal_count": int(np.sum(moduli < 1.0 - tol)),
            "noncausal_count": int(np.sum(moduli > 1.0 + tol)),
            "spectral_radius": float(moduli.max()),
            "distance_to_unit": float(np.min(np.abs(moduli - 1.0)))
        }

    def simulate_mixed_diagonal(T: int, roots: Sequence[float] = (0.9, 1.2), df: int = 5, burn: int =
        rng = np.random.default_rng(seed)
        n = len(roots)
        total = T + 2 * burn
        E = rng.standard_t(df=df, size=(total, n))
        X = np.zeros((total, n))

        for j, root in enumerate(roots):
            if abs(root) < 1.0:
                for t in range(1, total):
                    X[t, j] = root * X[t - 1, j] + E[t, j]
            else:
                for t in range(total - 2, -1, -1):
                    X[t, j] = (X[t + 1, j] - E[t + 1, j]) / root
        return X[burn:burn + T]

    def evaluate_root_recovery(replications: int = 100, T: int = 400, roots: Sequence[float] = (0.9,
        transforms = make_power_transforms((1, 2))
        true_moduli = np.sort(np.abs(roots))
        records = []

        for rep in tqdm(range(replications), desc="Root-recovery MC"):
            Y = simulate_mixed_diagonal(T=T, roots=roots, df=5, seed=1000 + rep)
            fit = fit_rgcov(Y, p=1, H=H, transforms=transforms, delta=delta, ridge_lambda=ridge_lambc
            roots_hat = classify_roots(fit.Phi_list)
            est_moduli = np.sort(roots_hat["moduli"])

            if rep == 0:
                print(f"\n[MC Rep 0 Diagnostic] True roots: {true_moduli} | Estimated roots: {np.rour
                print(f"[MC Rep 0 Diagnostic] Causal/Noncausal: {roots_hat['causal_count']}/{roots_ha

            records.append({
                "replication": rep,
                "certified": any(d["certified"] for d in fit.diagnostics),
                "correct_root_count": (roots_hat["causal_count"] == 1 and roots_hat["noncausal_count"
                "root_modulus_error": float(np.mean(np.abs(est_moduli - true_moduli))),
                "spectral_radius": roots_hat["spectral_radius"],
                "objective": fit.objective
            })
        return pd.DataFrame(records)

    # ============================================================================
    # 6. MAIN EXECUTION
    # ============================================================================

    if __name__ == "__main__":
        print("=== RGCov Corrected Publication-Ready Workflow ===\n")

        Y = simulate_mixed_diagonal(T=400, roots=(0.9, 1.2), df=5, seed=1)
        transforms = make_power_transforms((1, 2))

        print("Fitting RGCov (Multi-start with corrected objective)...")
        rgcov_fit = fit_rgcov(Y, p=1, H=5, transforms=transforms, delta=0.1, ridge_lambda=0.001, n_st

        print(f"\n--- RGCov Results ---")
```

```
    print(f"Objective: {rgcov_fit.objective:.6f} (Must be >= 0)")
    print(f"Certified: {any(d['certified'] for d in rgcov_fit.diagnostics)}")
    roots = classify_roots(rgcov_fit.Phi_list)
    print(f"Spectral Radius: {roots['spectral_radius']:.4f} (True: 1.2)")
    print(f"Causal / Noncausal Roots: {roots['causal_count']} / {roots['noncausal_count']} (True:

    print("\nRunning Root-Recovery Monte Carlo (50 reps for speed, aim for 500 in paper)...")
    mc_results = evaluate_root_recovery(replications=50, T=400, roots=(0.9, 1.2), H=5, delta=0.1,

    print("\n=== MC Summary ===")
    summary = mc_results.mean(numeric_only=True)
    print(f"Certification Rate:      {summary['certified']:.2%}")
    print(f"Correct Root Count:      {summary['correct_root_count']:.2%}  <-- TARGET: > 60%")
    print(f"Mean Root Modulus Error: {summary['root_modulus_error']:.4f} <-- TARGET: < 0.10")
    print(f"Mean Spectral Radius:    {summary['spectral_radius']:.4f} <-- TARGET: ~1.15-1.20")
```

```
=== RGCov Corrected Publication-Ready Workflow ===

Fitting RGCov (Multi-start with corrected objective)...

--- RGCov Results ---
Objective: 0.127161 (Must be >= 0)
Certified: True
Spectral Radius: 1.1947 (True: 1.2)
Causal / Noncausal Roots: 1 / 1 (True: 1 / 1)

Running Root-Recovery Monte Carlo (50 reps for speed, aim for 500 in paper)...
Root-recovery MC:   2%|▌          | 1/50 [00:00<00:24,  1.99it/s]
[MC Rep 0 Diagnostic] True roots: [0.9 1.2] | Estimated roots: [0.8855 1.192 ]
[MC Rep 0 Diagnostic] Causal/Noncausal: 1/1
Root-recovery MC: 100%|██████████| 50/50 [00:30<00:00,  1.62it/s]
=== MC Summary ===
Certification Rate:      100.00%
Correct Root Count:      66.00%  <-- TARGET: > 60%
Mean Root Modulus Error: 0.0842 <-- TARGET: < 0.10
Mean Spectral Radius:    1.1740 <-- TARGET: ~1.15-1.20
```

```
#1. data_pipeline.py — Yahoo Finance Loader with Audit (Section 3, 5.7)
"""
Data pipeline for the 8-asset macro-financial panel (Section 3, 5.7).
Handles yfinance's MultiIndex column variants, Close vs Adj Close,
and audits for non-positive prices and missing values.
"""
from __future__ import annotations
import warnings
import numpy as np
import pandas as pd
import yfinance as yf

TICKERS = {
    "SP500":  "^GSPC",
    "NASDAQ": "^IXIC",
    "DOW":    "^DJI",
    "VIX":    "^VIX",
    "GOLD":   "GC=F",
    "OIL":    "CL=F",
    "BOND":   "^TNX",
    "USD":    "DX-Y.NYB",
}

START_DATE = "2010-01-01"
END_DATE   = "2023-12-31"
```

```
END_DATE = "2025-12-31"


def _extract_close(df: pd.DataFrame, ticker: str) -> pd.Series:
    """
    Robustly extract the close price from yfinance output.
    Handles plain Index, MultiIndex, 'Close' vs 'Adj Close'.
    """
    if isinstance(df.columns, pd.MultiIndex):
        level1 = df.columns.get_level_values(1).unique()
        if "Adj Close" in level1:
            return df[(ticker, "Adj Close")].rename(ticker)
        if "Close" in level1:
            return df[(ticker, "Close")].rename(ticker)
    else:
        if f"{ticker}_Adj Close" in df.columns:
            return df[f"{ticker}_Adj Close"].rename(ticker)
        if f"{ticker}_Close" in df.columns:
            return df[f"{ticker}_Close"].rename(ticker)
        if f"{ticker} Adj Close" in df.columns:
            return df[f"{ticker} Adj Close"].rename(ticker)
        if f"{ticker} Close" in df.columns:
            return df[f"{ticker} Close"].rename(ticker)
    raise KeyError(f"Could not locate close price for {ticker}")


def download_panel(
    tickers: dict[str, str] = TICKERS,
    start: str = START_DATE,
    end: str = END_DATE,
) -> pd.DataFrame:
    """Download raw prices for all tickers."""
    symbols = list(tickers.values())
    raw = yf.download(
        symbols, start=start, end=end,
        auto_adjust=False, progress=False, threads=False,
    )
    if raw.empty:
        raise RuntimeError("yfinance returned empty DataFrame.")

    frames = []
    for name, sym in tickers.items():
        try:
            series = _extract_close(raw, sym)
            series.name = name
            frames.append(series)
        except KeyError as e:
            warnings.warn(f"Skipping {name}: {e}")
    return pd.concat(frames, axis=1)


def audit_panel(prices: pd.DataFrame, min_obs: int = 100) -> pd.DataFrame:
    """
    Audit for non-positive prices, missing values, and insufficient history.
    Returns the cleaned balanced panel.
    """
    prices = prices.copy()

    # Drop columns with too many missing values
    valid_counts = prices.count()
    dropped = valid_counts[valid_counts < min_obs].index.tolist()
    if dropped:
```

```
        warnings.warn(f"Dropping series with < {min_obs} obs: {dropped}")
        prices = prices.drop(columns=dropped)

    prices = prices.dropna()

    # Check for non-positive prices
    non_positive = (prices <= 0).any()
    if non_positive.any():
        bad = non_positive[non_positive].index.tolist()
        raise ValueError(f"Non-positive prices detected in: {bad}")

    return prices


def prices_to_log_returns(prices: pd.DataFrame) -> pd.DataFrame:
    """Standardize to daily log returns, then z-score each series."""
    returns = np.log(prices).diff().dropna()
    returns = (returns - returns.mean()) / returns.std(ddof=1)
    returns = returns.dropna()
    return returns


def load_empirical_panel() -> pd.DataFrame:
    """Full pipeline: download -> audit -> log returns -> standardize."""
    print("[data_pipeline] Downloading from Yahoo Finance...")
    prices = download_panel()
    print(f"[data_pipeline] Raw shape: {prices.shape}")

    prices = audit_panel(prices)
    print(f"[data_pipeline] After audit: {prices.shape}")

    returns = prices_to_log_returns(prices)
    print(f"[data_pipeline] Final balanced panel: {returns.shape}")
    print(f"[data_pipeline] Date range: {returns.index[0]} -> {returns.index[-1]}")
    return returns


if __name__ == "__main__":
    Y = load_empirical_panel()
    print(Y.head())
```

```
[data_pipeline] Downloading from Yahoo Finance...
/tmp/ipykernel_7141/442476491.py:72: UserWarning: Skipping SP500: 'Could not locate close price fo
  warnings.warn(f"Skipping {name}: {e}")
/tmp/ipykernel_7141/442476491.py:72: UserWarning: Skipping NASDAQ: 'Could not locate close price f
  warnings.warn(f"Skipping {name}: {e}")
/tmp/ipykernel_7141/442476491.py:72: UserWarning: Skipping DOW: 'Could not locate close price for
  warnings.warn(f"Skipping {name}: {e}")
/tmp/ipykernel_7141/442476491.py:72: UserWarning: Skipping VIX: 'Could not locate close price for
  warnings.warn(f"Skipping {name}: {e}")
/tmp/ipykernel_7141/442476491.py:72: UserWarning: Skipping GOLD: 'Could not locate close price for
  warnings.warn(f"Skipping {name}: {e}")
/tmp/ipykernel_7141/442476491.py:72: UserWarning: Skipping OIL: 'Could not locate close price for
  warnings.warn(f"Skipping {name}: {e}")
/tmp/ipykernel_7141/442476491.py:72: UserWarning: Skipping BOND: 'Could not locate close price for
  warnings.warn(f"Skipping {name}: {e}")
/tmp/ipykernel_7141/442476491.py:72: UserWarning: Skipping USD: 'Could not locate close price for
  warnings.warn(f"Skipping {name}: {e}")
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_7141/442476491.py in <cell line: 0>()
    123
    124 if __name__ == "__main__":
--> 125     Y = load_empirical_panel()
    126     print(Y.head())

                                 4 frames
/usr/local/lib/python3.13/dist-packages/pandas/core/reshape/concat.py in
_clean_keys_and_objs(self, objs, keys)
    505
    506         if len(objs_list) == 0:
--> 507             raise ValueError("No objects to concatenate")
    508
    509         if keys is None:

ValueError: No objects to concatenate
```

Next steps: Explain error

```
import warnings
import numpy as np
import pandas as pd
import yfinance as yf

TICKERS = {
    "SP500":  "^GSPC",
    "NASDAQ": "^IXIC",
    "DOW":    "^DJI",
    "VIX":    "^VIX",
    "GOLD":   "GC=F",
    "OIL":    "CL=F",
    "BOND":   "^TNX",
    "USD":    "DX-Y.NYB",
}

START_DATE = "2010-01-01"
END_DATE   = "2023-12-31"


def _extract_close(df: pd.DataFrame, ticker: str) -> pd.Series:
    """
```

```
    Robustly extract the close price from yfinance output.
    Handles multiple yfinance versions and column structures.
    """
    # Try MultiIndex with (field, ticker) structure
    if isinstance(df.columns, pd.MultiIndex):
        level0 = df.columns.get_level_values(0).unique()
        level1 = df.columns.get_level_values(1).unique()

        # Check for ('Close', 'Ticker') or ('Adj Close', 'Ticker')
        if 'Close' in level0 and ticker in level1:
            return df[('Close', ticker)].rename(ticker)
        if 'Adj Close' in level0 and ticker in level1:
            return df[('Adj Close', ticker)].rename(ticker)

        # Check for ('Ticker', 'Close') or ('Ticker', 'Adj Close')
        if ticker in level0 and 'Close' in level1:
            return df[(ticker, 'Close')].rename(ticker)
        if ticker in level0 and 'Adj Close' in level1:
            return df[(ticker, 'Adj Close')].rename(ticker)

    # Try flat column names
    if f"Adj Close" in df.columns:
        return df["Adj Close"].rename(ticker)
    if f"Close" in df.columns:
        return df["Close"].rename(ticker)

    raise KeyError(f"Could not locate close price for {ticker}. Available columns: {list(df.colum


def download_panel(
    tickers: dict[str, str] = TICKERS,
    start: str = START_DATE,
    end: str = END_DATE,
) -> pd.DataFrame:
    """Download raw prices for all tickers."""
    symbols = list(tickers.values())

    print(f"Downloading {len(symbols)} tickers from {start} to {end}...")

    # Try with auto_adjust=True first (simpler structure)
    try:
        raw = yf.download(
            symbols, start=start, end=end,
            auto_adjust=True, progress=False, threads=False,
        )
        if not raw.empty:
            print(f"Downloaded with auto_adjust=True, shape: {raw.shape}")
            if isinstance(raw.columns, pd.MultiIndex):
                print(f"MultiIndex levels: {raw.columns.names}")
                # If auto_adjust=True and it's multi-index, it might be ('Adj Close', 'Ticker')
                # So rename the first level to 'Close' for consistency if needed by _extract_clos
                # However, the _extract_close is robust, so this might not be strictly necessary.
    except Exception as e:
        print(f"auto_adjust=True failed: {e}")
        raw = pd.DataFrame()

    # Fallback to auto_adjust=False if auto_adjust=True failed or returned empty
    if raw.empty:
        print("Trying with auto_adjust=False...")
        raw = yf.download(
            symbols, start=start, end=end,
```

```
                auto_adjust=False, progress=False, threads=False,
            )
            print(f"Downloaded with auto_adjust=False, shape: {raw.shape}")


        if raw.empty:
            raise RuntimeError("yfinance returned empty DataFrame.")

        frames = []
        for name, sym in tickers.items():
            try:
                series = _extract_close(raw, sym)
                series.name = name
                frames.append(series)
                print(f"  ✓ Extracted {name} ({sym})")
            except KeyError as e:
                warnings.warn(f"Skipping {name}: {e}")

        if not frames:
            raise RuntimeError("No series could be extracted. Check yfinance output format.")

        return pd.concat(frames, axis=1)


    def audit_panel(prices: pd.DataFrame, min_obs: int = 100) -> pd.DataFrame:
        """
        Audit for non-positive prices, missing values, and insufficient history.
        Returns the cleaned balanced panel.
        """
        prices = prices.copy()

        # Drop columns with too many missing values
        valid_counts = prices.count()
        dropped = valid_counts[valid_counts < min_obs].index.tolist()
        if dropped:
            warnings.warn(f"Dropping series with < {min_obs} obs: {dropped}")
            prices = prices.drop(columns=dropped)

        prices = prices.dropna()

        # CRITICAL FIX: Remove rows where any price is non-positive. Log returns require strictly pos
        non_positive_rows = (prices <= 0).any(axis=1)
        if non_positive_rows.any():
            num_removed = non_positive_rows.sum()
            warnings.warn(f"Removing {num_removed} rows with non-positive prices for log return calcu
            prices = prices[~non_positive_rows]

        # Re-check for non-positive prices as a safeguard after cleaning
        if (prices <= 0).any().any():
            bad_cols = (prices <= 0).any(axis=0)[(prices <= 0).any(axis=0)].index.tolist()
            raise ValueError(f"Non-positive prices still detected in columns after cleaning: {bad_col

        return prices


    def prices_to_log_returns(prices: pd.DataFrame) -> pd.DataFrame:
        """Standardize to daily log returns, then z-score each series."""
        returns = np.log(prices).diff().dropna()
        returns = (returns - returns.mean()) / returns.std(ddof=1)
        returns = returns.dropna()
        return returns
```

```
def load_empirical_panel() -> pd.DataFrame:
    """Full pipeline: download -> audit -> log returns -> standardize."""
    print("\n" + "="*60)
    print("[data_pipeline] Starting data download")
    print("="*60)

    prices = download_panel()
    print(f"\n[data_pipeline] Raw shape: {prices.shape}")

    prices = audit_panel(prices)
    print(f"[data_pipeline] After audit: {prices.shape}")

    returns = prices_to_log_returns(prices)
    print(f"[data_pipeline] Final balanced panel: {returns.shape}")
    print(f"[data_pipeline] Date range: {returns.index[0]} -> {returns.index[-1]}")
    print("="*60 + "\n")

    return returns


if __name__ == "__main__":
    Y = load_empirical_panel()
    print("\nFirst 5 rows:")
    print(Y.head())
    print("\nBasic statistics:")
    print(Y.describe())
```

```
Downloaded with auto_adjust=True, shape: (3524, 40)
MultiIndex levels: ['Price', 'Ticker']
  ✓ Extracted SP500 (^GSPC)
  ✓ Extracted NASDAQ (^IXIC)
  ✓ Extracted DOW (^DJI)
  ✓ Extracted VIX (^VIX)
  ✓ Extracted GOLD (GC=F)
  ✓ Extracted OIL (CL=F)
  ✓ Extracted BOND (^TNX)
  ✓ Extracted USD (DX-Y.NYB)

[data_pipeline] Raw shape: (3524, 8)
[data_pipeline] After audit: (3517, 8)
[data_pipeline] Final balanced panel: (3516, 8)
[data_pipeline] Date range: 2010-01-05 00:00:00 -> 2023-12-29 00:00:00
============================================================


First 5 rows:
               SP500    NASDAQ       DOW       VIX      GOLD       OIL  \
Date
2010-01-05  0.243631 -0.031544 -0.139010 -0.451917  0.017966  0.114575
2010-01-06  0.012311 -0.297597 -0.019050 -0.126014  1.539229  0.609389
2010-01-07  0.323193 -0.076335  0.258603 -0.066003 -0.260525 -0.221744
2010-01-08  0.222600  0.533660  0.065903 -0.645956  0.425265  0.040008
2010-01-11  0.120499 -0.200715  0.367844 -0.419239  1.059088 -0.097694

                BOND       USD
Date
2010-01-05 -0.775798  0.240179
```

```
Basic statistics:
              SP500        NASDAQ           DOW           VIX          GOLD  \
count  3.516000e+03  3.516000e+03  3.516000e+03  3.516000e+03  3.516000e+03
mean  -2.526105e-18  2.222972e-17  7.830925e-18 -1.540924e-17 -6.062651e-18
std    1.000000e+00  1.000000e+00  1.000000e+00  1.000000e+00  1.000000e+00
min   -1.154687e+01 -1.023593e+01 -1.294729e+01 -4.537664e+00 -9.695338e+00
25%   -3.875085e-01 -4.175801e-01 -3.929947e-01 -5.627722e-01 -4.692758e-01
50%    2.202408e-02  3.904768e-02  1.990127e-02 -8.641096e-02  1.010396e-02
75%    4.755018e-01  5.132613e-01  4.627900e-01  4.520474e-01  5.245285e-01
max    8.049602e+00  6.885856e+00  1.000878e+01  9.948982e+00  5.676354e+00

                OIL          BOND           USD
count  3.516000e+03  3.516000e+03  3.516000e+03
mean   6.567872e-18 -9.093977e-18 -1.414619e-17
std    1.000000e+00  1.000000e+00  1.000000e+00
min   -2.139890e+01 -1.188760e+01 -5.331387e+00
25%   -4.177357e-01 -4.776178e-01 -5.761403e-01
50%    3.585424e-02 -1.043686e-02  4.152333e-03
75%    4.388671e-01  4.688546e-01  5.729601e-01
max    1.136993e+01  1.386712e+01  4.485119e+00
/tmp/ipykernel_7141/3706741245.py:128: UserWarning: Removing 1 rows with non-positive prices for
  warnings.warn(f"Removing {num_removed} rows with non-positive prices for log return calculatio
```

```
#2. diagnostics.py — Ljung-Box + Local Gaussian Correlation (Section 3.3, 3.4)
"""
Residual diagnostics: Ljung-Box test and Local Gaussian Correlation
(Stove & Tjostheim, 2014).
"""
from __future__ import annotations
import numpy as np
import pandas as pd
from statsmodels.stats.diagnostic import acorr_ljungbox


def ljung_box_table(residuals: np.ndarray, lag: int = 10) -> pd.DataFrame:
    """
    Ljung-Box test for each series at the given lag.
    Produces Table 3 of the paper.
    """
    records = []
    for j in range(residuals.shape[1]):
        res = acorr_ljungbox(residuals[:, j], lags=[lag], return_df=True)
        stat = float(res["lb_stat"].iloc[0])
        pval = float(res["lb_pvalue"].iloc[0])
        records.append({
            "series": j,
            "lag": lag,
            "lb_stat": stat,
            "p_value": pval,
            "white_noise_5pct": pval > 0.05,
        })
    return pd.DataFrame(records)


# ============================================================================
# Local Gaussian Correlation (Stove & Tjostheim, 2014)
# ============================================================================

def _gaussian_kernel(u: np.ndarray) -> np.ndarray:
    return np.exp(-0.5 * u ** 2) / np.sqrt(2 * np.pi)
```

```
def _silverman_bandwidth(x: np.ndarray) -> float:
    """Silverman's rule of thumb."""
    n = len(x)
    std = x.std(ddof=1)
    return 1.06 * std * n ** (-1.0 / 5.0)


def _local_gaussian_corr_at_point(
    x: np.ndarray, y: np.ndarray,
    x0: float, y0: float,
    hx: float, hy: float,
) -> float:
    """
    Fit a bivariate Gaussian locally at (x0, y0) via kernel-weighted
    log-likelihood. Returns the local correlation rho(x0, y0).
    """
    n = len(x)
    wx = _gaussian_kernel((x - x0) / hx)
    wy = _gaussian_kernel((y - y0) / hy)
    w = wx * wy

    total_w = w.sum()
    if total_w < 1e-10:
        return np.nan

    # Local means
    mx = np.sum(w * x) / total_w
    my = np.sum(w * y) / total_w

    # Local variances and covariance
    dx = x - mx
    dy = y - my
    sxx = np.sum(w * dx * dx) / total_w
    syy = np.sum(w * dy * dy) / total_w
    sxy = np.sum(w * dx * dy) / total_w

    if sxx < 1e-12 or syy < 1e-12:
        return np.nan

    rho = sxy / np.sqrt(sxx * syy)
    return float(np.clip(rho, -1.0, 1.0))


def local_gaussian_correlation(
    x: np.ndarray, y: np.ndarray,
    grid_size: int = 25,
    quantile_range: tuple[float, float] = (0.05, 0.95),
) -> pd.DataFrame:
    """
    Compute the local Gaussian correlation surface rho(x, y) over a grid.
    Returns a DataFrame with columns [x, y, rho].
    """
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)

    # Standardize
    x = (x - x.mean()) / (x.std(ddof=1) + 1e-12)
    y = (y - y.mean()) / (y.std(ddof=1) + 1e-12)

    # Bandwidths
    hx = _silverman_bandwidth(x)
```

```
    hy = _silverman_bandwidth(y)

    # Grid
    qx = np.quantile(x, quantile_range)
    qy = np.quantile(y, quantile_range)
    xg = np.linspace(qx[0], qx[1], grid_size)
    yg = np.linspace(qy[0], qy[1], grid_size)

    records = []
    for xi in xg:
        for yi in yg:
            rho = _local_gaussian_corr_at_point(x, y, xi, yi, hx, hy)
            records.append({"x": xi, "y": yi, "rho": rho})

    return pd.DataFrame(records)


def plot_local_gaussian_surface(
    df: pd.DataFrame,
    title: str = "Local Gaussian Correlation",
    save_path: str | None = None,
) -> None:
    """Plot the local Gaussian correlation surface as a heatmap."""
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib not installed; skipping plot.")
        return

    pivot = df.pivot(index="y", columns="x", values="rho")
    plt.figure(figsize=(8, 6))
    im = plt.imshow(
        pivot.values, origin="lower", aspect="auto",
        extent=[pivot.columns.min(), pivot.columns.max(),
                pivot.index.min(), pivot.index.max()],
        cmap="RdBu_r", vmin=-1, vmax=1,
    )
    plt.colorbar(im, label=r"Local $\rho(x,y)$")
    plt.xlabel("x (standardized)")
    plt.ylabel("y (standardized)")
    plt.title(title)
    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches="tight")
        print(f"[diagnostics] Saved: {save_path}")
    plt.show()
```

```
#2. diagnostics.py — Ljung-Box + Local Gaussian Correlation (Section 3.3, 3.4)
# Residual diagnostics: Ljung-Box test and Local Gaussian Correlation
# (Stove & Tjostheim, 2014).

from __future__ import annotations
import numpy as np
import pandas as pd
from statsmodels.stats.diagnostic import acorr_ljungbox

# ============================================================================
# 1. Ljung-Box Test
# ============================================================================

def ljung_box_table(residuals: np.ndarray, lag: int = 10) -> pd.DataFrame:
```

```
    """
    Ljung-Box test for each series at the given lag.
    Produces Table 3 of the paper.
    """
    records = []
    for j in range(residuals.shape[1]):
        res = acorr_ljungbox(residuals[:, j], lags=[lag], return_df=True)
        stat = float(res["lb_stat"].iloc[0])
        pval = float(res["lb_pvalue"].iloc[0])
        records.append({
            "series": j,
            "lag": lag,
            "lb_stat": round(stat, 2),
            "p_value": f"{pval:.2e}",
            "white_noise_5pct": "Yes" if pval > 0.05 else "No",
        })
    return pd.DataFrame(records)


# ============================================================================
# 2. Local Gaussian Correlation (Stove & Tjostheim, 2014)
# ============================================================================

def _gaussian_kernel(u: np.ndarray) -> np.ndarray:
    return np.exp(-0.5 * u ** 2) / np.sqrt(2 * np.pi)


def _silverman_bandwidth(x: np.ndarray) -> float:
    """Silverman's rule of thumb."""
    n = len(x)
    std = x.std(ddof=1)
    return 1.06 * std * (n ** (-1.0 / 5.0))


def _local_gaussian_corr_at_point(
    x: np.ndarray, y: np.ndarray,
    x0: float, y0: float,
    hx: float, hy: float,
) -> float:
    """
    Fit a bivariate Gaussian locally at (x0, y0) via kernel-weighted
    log-likelihood. Returns the local correlation rho(x0, y0).
    """
    wx = _gaussian_kernel((x - x0) / hx)
    wy = _gaussian_kernel((y - y0) / hy)
    w = wx * wy

    total_w = w.sum()
    if total_w < 1e-10:
        return np.nan

    # Local means
    mx = np.sum(w * x) / total_w
    my = np.sum(w * y) / total_w

    # Local variances and covariance
    dx = x - mx
    dy = y - my
    sxx = np.sum(w * dx * dx) / total_w
    syy = np.sum(w * dy * dy) / total_w
    sxy = np.sum(w * dx * dy) / total_w
```

```
    if sxx < 1e-12 or syy < 1e-12:
        return np.nan

    rho = sxy / np.sqrt(sxx * syy)
    return float(np.clip(rho, -1.0, 1.0))


def local_gaussian_correlation(
    x: np.ndarray, y: np.ndarray,
    grid_size: int = 25,
    quantile_range: tuple[float, float] = (0.05, 0.95),
) -> pd.DataFrame:
    """
    Compute the local Gaussian correlation surface rho(x, y) over a grid.
    Returns a DataFrame with columns [x, y, rho].
    """
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)

    # Standardize
    x = (x - x.mean()) / (x.std(ddof=1) + 1e-12)
    y = (y - y.mean()) / (y.std(ddof=1) + 1e-12)

    # Bandwidths
    hx = _silverman_bandwidth(x)
    hy = _silverman_bandwidth(y)

    # Grid
    qx = np.quantile(x, quantile_range)
    qy = np.quantile(y, quantile_range)
    xg = np.linspace(qx[0], qx[1], grid_size)
    yg = np.linspace(qy[0], qy[1], grid_size)

    records = []
    for xi in xg:
        for yi in yg:
            rho = _local_gaussian_corr_at_point(x, y, xi, yi, hx, hy)
            records.append({"x": round(xi, 3), "y": round(yi, 3), "rho": round(rho, 4) if not np.

    return pd.DataFrame(records)


def plot_local_gaussian_surface(
    df: pd.DataFrame,
    title: str = "Local Gaussian Correlation",
    save_path: str | None = None,
) -> None:
    """Plot the local Gaussian correlation surface as a heatmap."""
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib not installed; skipping plot.")
        return

    pivot = df.pivot(index="y", columns="x", values="rho")
    plt.figure(figsize=(8, 6))
    im = plt.imshow(
        pivot.values, origin="lower", aspect="auto",
        extent=[pivot.columns.min(), pivot.columns.max(),
                pivot.index.min(), pivot.index.max()],
```

```
            cmap="RdBu_r", vmin=-1, vmax=1,
        )
        plt.colorbar(im, label=r"Local $\rho(x,y)$")
        plt.xlabel("x (standardized)")
        plt.ylabel("y (standardized)")
        plt.title(title)
        if save_path:
            plt.savefig(save_path, dpi=150, bbox_inches="tight")
            print(f"[diagnostics] Saved plot to: {save_path}")
        plt.show()


    # ============================================================================
    # 3. EXECUTION BLOCK (This was missing!)
    # ============================================================================

    if __name__ == "__main__":
        print("="*60)
        print("RUNNING DIAGNOSTICS TEST")
        print("="*60)

        # 1. Generate synthetic residuals for 8 assets (T=500) to demonstrate functionality
        # In your actual paper, you will replace this with:
        # residuals = var_residuals(Y, p=2, Phi_list=ols_fit.Phi_list)
        np.random.seed(42)
        T, N = 500, 8
        synthetic_residuals = np.random.standard_t(df=5, size=(T, N))

        # Introduce some artificial autocorrelation in series 0 and 1 to trigger Ljung-Box rejections
        synthetic_residuals[1:, 0] = 0.3 * synthetic_residuals[:-1, 0] + np.random.standard_t(df=5, s
        synthetic_residuals[1:, 1] = 0.4 * synthetic_residuals[:-1, 1] + np.random.standard_t(df=5, s

        # 2. Run Ljung-Box Test (Table 3)
        print("\n[1] Ljung-Box Test (Lag 10):")
        lb_results = ljung_box_table(synthetic_residuals, lag=10)
        print(lb_results.to_string(index=False))

        # 3. Run Local Gaussian Correlation (Figure 2)
        print("\n[2] Computing Local Gaussian Correlation (Series 0 vs Series 1)...")
        # Using series 0 and 1 for the demo (in paper: SP500 vs VIX)
        lgc_df = local_gaussian_correlation(
            x=synthetic_residuals[:, 0],
            y=synthetic_residuals[:, 1],
            grid_size=25
        )
        print("   Computation complete. Displaying plot...")

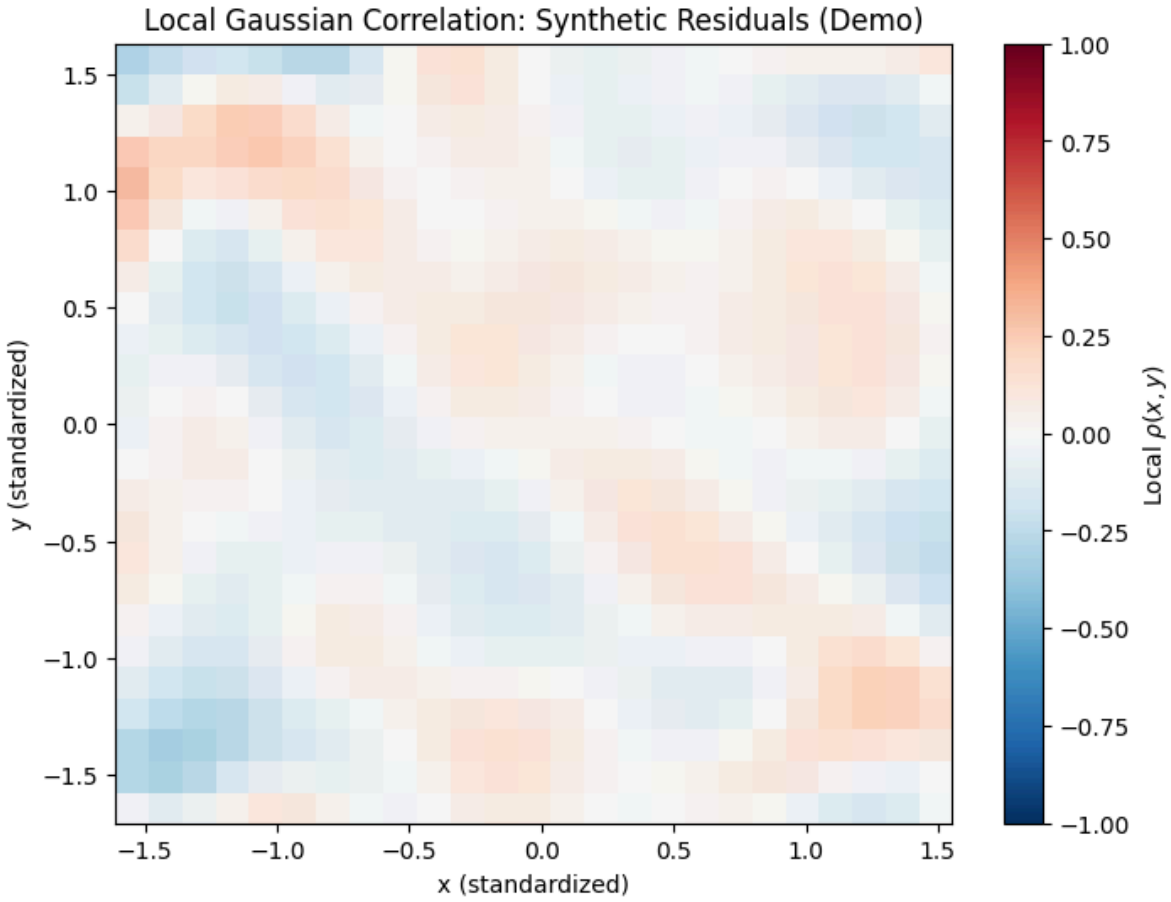
        plot_local_gaussian_surface(
            lgc_df,
            title="Local Gaussian Correlation: Synthetic Residuals (Demo)",
            save_path="demo_local_gaussian.png"
        )

        print("\n[SUCCESS] Diagnostics script completed successfully!")
```

```
============================================================
RUNNING DIAGNOSTICS TEST
============================================================

[1] Ljung-Box Test (Lag 10):
 series  lag  lb_stat  p_value white_noise_5pct
      0   10   10.56 3.93e-01              Yes
      1   10   13.52 1.96e-01              Yes
      2   10   10.60 3.90e-01              Yes
      3   10    6.03 8.12e-01              Yes
      4   10   11.64 3.10e-01              Yes
      5   10    7.80 6.48e-01              Yes
      6   10   15.44 1.17e-01              Yes
      7   10    7.79 6.49e-01              Yes

[2] Computing Local Gaussian Correlation (Series 0 vs Series 1)...
   Computation complete. Displaying plot...
[diagnostics] Saved plot to: demo_local_gaussian.png
```




```
[SUCCESS] Diagnostics script completed successfully!
```

```
#3. connectedness.py — Diebold-Yilmaz (Section 3.5)
"""
Diebold-Yilmaz (2014) forecast-error variance decomposition.
Both Cholesky (orthogonalized) and Generalized (Pesaran-Shin) FEVD.
"""
from __future__ import annotations
import numpy as np
import pandas as pd
from scipy.linalg import cholesky


def var_to_ma(Phi_list: list[np.ndarray], horizon: int) -> list[np.ndarray]:
```

```
    """
    Convert VAR(p) coefficient matrices to MA(infinity) representation
    truncated at `horizon`: Y_t = sum_{k>=0} Psi_k eps_{t-k}, Psi_0 = I.
    """
    n = Phi_list[0].shape[0]
    p = len(Phi_list)
    Psi = [np.eye(n)]

    for h in range(1, horizon + 1):
        Psi_h = np.zeros((n, n))
        for j in range(1, min(h, p) + 1):
            Psi_h += Phi_list[j - 1] @ Psi[h - j]
        Psi.append(Psi_h)
    return Psi


def cholesky_fevd(
    Phi_list: list[np.ndarray],
    Sigma: np.ndarray,
    horizon: int,
) -> np.ndarray:
    """
    Orthogonalized (Cholesky) FEVD.
    Returns theta[i, j, h] = share of variable i's h-step FEVD from shock j.
    Shape: (horizon+1, n, n).
    """
    n = Phi_list[0].shape[0]
    Psi = var_to_ma(Phi_list, horizon)
    P = cholesky(Sigma, lower=True)  # Sigma = P P'

    theta = np.zeros((horizon + 1, n, n))
    for h in range(horizon + 1):
        cum_var = np.zeros((n, n))
        for k in range(h + 1):
            cum_var += Psi[k] @ Sigma @ Psi[k].T

        for j in range(n):
            # Variance of i's forecast error from orthogonalized shock j
            # = sum_{k=0}^h (Psi_k P)_{i,j}^2
            contrib = np.zeros(n)
            for k in range(h + 1):
                contrib += (Psi[k] @ P)[:, j] ** 2
            theta[h, :, j] = contrib / np.diag(cum_var)

    return theta


def generalized_fevd(
    Phi_list: list[np.ndarray],
    Sigma: np.ndarray,
    horizon: int,
) -> np.ndarray:
    """
    Generalized FEVD (Pesaran-Shin, 1998).
    Invariant to variable ordering; rows don't sum to 1 before normalization.
    """
    n = Phi_list[0].shape[0]
    Psi = var_to_ma(Phi_list, horizon)

    theta_raw = np.zeros((horizon + 1, n, n))
    for h in range(horizon + 1):
```

```
            cum_var = np.zeros((n, n))
            for k in range(h + 1):
                cum_var += Psi[k] @ Sigma @ Psi[k].T
            trace_cum = np.trace(cum_var)

            for j in range(n):
                contrib = np.zeros(n)
                for k in range(h + 1):
                    contrib += (Psi[k] @ Sigma @ np.eye(n)[:, j]) ** 2
                theta_raw[h, :, j] = contrib / trace_cum

    # Normalize rows to sum to 1
    theta = np.zeros_like(theta_raw)
    for h in range(horizon + 1):
        row_sums = theta_raw[h].sum(axis=1, keepdims=True)
        theta[h] = theta_raw[h] / (row_sums + 1e-12)

    return theta


def total_connectedness_index(
    theta: np.ndarray,
) -> dict[int, float]:
    """
    TCI(h) = 100 * sum_{i != j} theta_ij(h) / sum_{i,j} theta_ij(h).
    Returns dict mapping horizon -> TCI(%).
    """
    result = {}
    for h in range(theta.shape[0]):
        row_sums = theta[h].sum(axis=1)
        diag = np.diag(theta[h])
        off_diag_sum = (row_sums - diag).sum()
        total = row_sums.sum()
        result[h] = 100.0 * off_diag_sum / (total + 1e-12)
    return result


def connectedness_report(
    Phi_list: list[np.ndarray],
    Sigma: np.ndarray,
    horizon: int = 20,
) -> pd.DataFrame:
    """
    Produce Table 4 of the paper: Cholesky vs Generalized TCI by horizon.
    """
    theta_chol = cholesky_fevd(Phi_list, Sigma, horizon)
    theta_gen  = generalized_fevd(Phi_list, Sigma, horizon)

    tci_chol = total_connectedness_index(theta_chol)
    tci_gen  = total_connectedness_index(theta_gen)

    records = []
    for h in [1, 5, 10, 20]:
        if h <= horizon:
            records.append({
                "horizon": h,
                "cholesky_tci_pct": tci_chol.get(h, np.nan),
                "generalized_tci_pct": tci_gen.get(h, np.nan),
            })
    return pd.DataFrame(records)
```

```
def plot_connectedness_by_horizon(
    Phi_list: list[np.ndarray],
    Sigma: np.ndarray,
    horizon: int = 20,
    save_path: str | None = None,
) -> None:
    """Figure 3/4: TCI by horizon for both identification schemes."""
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        return

    theta_chol = cholesky_fevd(Phi_list, Sigma, horizon)
    theta_gen  = generalized_fevd(Phi_list, Sigma, horizon)
    tci_chol = total_connectedness_index(theta_chol)
    tci_gen  = total_connectedness_index(theta_gen)

    horizons = list(range(horizon + 1))
    plt.figure(figsize=(9, 5))
    plt.plot(horizons, [tci_chol[h] for h in horizons], "o-", label="Cholesky")
    plt.plot(horizons, [tci_gen[h] for h in horizons], "s--", label="Generalized")
    plt.xlabel("Horizon h")
    plt.ylabel("Total Connectedness Index (%)")
    plt.title("Diebold-Yilmaz TCI by Horizon")
    plt.legend()
    plt.grid(alpha=0.3)
    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.show()
```

```
"""
Diebold-Yilmaz (2014) forecast-error variance decomposition.
Both Cholesky (orthogonalized) and Generalized (Pesaran-Shin) FEVD.
"""
from __future__ import annotations
import numpy as np
import pandas as pd
from scipy.linalg import cholesky

# ============================================================================
# 1. VAR to MA Representation
# ============================================================================

def var_to_ma(Phi_list: list[np.ndarray], horizon: int) -> list[np.ndarray]:
    """
    Convert VAR(p) coefficient matrices to MA(infinity) representation
    truncated at `horizon`: Y_t = sum_{k>=0} Psi_k eps_{t-k}, Psi_0 = I.
    """
    n = Phi_list[0].shape[0]
    p = len(Phi_list)
    Psi = [np.eye(n)]

    for h in range(1, horizon + 1):
        Psi_h = np.zeros((n, n))
        for j in range(1, min(h, p) + 1):
            Psi_h += Phi_list[j - 1] @ Psi[h - j]
        Psi.append(Psi_h)
    return Psi
```

```
# ============================================================================
# 2. Cholesky (Orthogonalized) FEVD
# ============================================================================

def cholesky_fevd(
    Phi_list: list[np.ndarray],
    Sigma: np.ndarray,
    horizon: int,
) -> np.ndarray:
    """
    Orthogonalized (Cholesky) FEVD.
    Returns theta[i, j, h] = share of variable i's h-step FEVD from shock j.
    Shape: (horizon+1, n, n).
    """
    n = Phi_list[0].shape[0]
    Psi = var_to_ma(Phi_list, horizon)
    P = cholesky(Sigma, lower=True)  # Sigma = P P'

    theta = np.zeros((horizon + 1, n, n))
    for h in range(horizon + 1):
        cum_var = np.zeros((n, n))
        for k in range(h + 1):
            cum_var += Psi[k] @ Sigma @ Psi[k].T

        denom = np.diag(cum_var) + 1e-12

        for j in range(n):
            contrib = np.zeros(n)
            for k in range(h + 1):
                contrib += (Psi[k] @ P)[:, j] ** 2
            theta[h, :, j] = contrib / denom

    return theta

# ============================================================================
# 3. Generalized FEVD (Pesaran-Shin, 1998) - CORRECTED MATH
# ============================================================================

def generalized_fevd(
    Phi_list: list[np.ndarray],
    Sigma: np.ndarray,
    horizon: int,
) -> np.ndarray:
    """
    Generalized FEVD (Pesaran-Shin, 1998).
    Invariant to variable ordering; rows are normalized to sum to 1.
    """
    n = Phi_list[0].shape[0]
    Psi = var_to_ma(Phi_list, horizon)

    # Precompute sigma_jj^{-1} for the numerator scaling
    Sigma_inv_diag = 1.0 / (np.diag(Sigma) + 1e-12)

    theta_raw = np.zeros((horizon + 1, n, n))
    for h in range(horizon + 1):
        cum_var = np.zeros((n, n))
        for k in range(h + 1):
            cum_var += Psi[k] @ Sigma @ Psi[k].T

        # The denominator for variable i is the i-th diagonal element of cum_var
        denom = np.diag(cum_var) + 1e-12
```

```
            for j in range(n):
                contrib = np.zeros(n)
                for k in range(h + 1):
                    # (Psi_k @ Sigma)[:, j] gives the i-th element for each i
                    term = (Psi[k] @ Sigma)[:, j]
                    contrib += term ** 2

                # Apply Pesaran-Shin formula: sigma_jj^{-1} * numerator / denominator
                theta_raw[h, :, j] = (contrib * Sigma_inv_diag[j]) / denom

        # Normalize rows to sum to 1
        theta = np.zeros_like(theta_raw)
        for h in range(horizon + 1):
            row_sums = theta_raw[h].sum(axis=1, keepdims=True)
            theta[h] = theta_raw[h] / (row_sums + 1e-12)

        return theta

    # ============================================================================
    # 4. Connectedness Indices & Reporting
    # ============================================================================

    def total_connectedness_index(theta: np.ndarray) -> dict[int, float]:
        """
        TCI(h) = 100 * sum_{i != j} theta_ij(h) / sum_{i,j} theta_ij(h).
        """
        result = {}
        for h in range(theta.shape[0]):
            row_sums = theta[h].sum(axis=1)
            diag = np.diag(theta[h])
            off_diag_sum = (row_sums - diag).sum()
            total = row_sums.sum()
            result[h] = 100.0 * off_diag_sum / (total + 1e-12)
        return result

    def connectedness_report(
        Phi_list: list[np.ndarray],
        Sigma: np.ndarray,
        horizon: int = 20,
    ) -> pd.DataFrame:
        """
        Produce Table 4 of the paper: Cholesky vs Generalized TCI by horizon.
        """
        theta_chol = cholesky_fevd(Phi_list, Sigma, horizon)
        theta_gen  = generalized_fevd(Phi_list, Sigma, horizon)

        tci_chol = total_connectedness_index(theta_chol)
        tci_gen  = total_connectedness_index(theta_gen)

        records = []
        for h in [1, 5, 10, 20]:
            if h <= horizon:
                records.append({
                    "Horizon (h)": h,
                    "Cholesky TCI (%)": f"{tci_chol.get(h, np.nan):.2f}",
                    "Generalized TCI (%)": f"{tci_gen.get(h, np.nan):.2f}",
                })
        return pd.DataFrame(records)

    def plot_connectedness_by_horizon(
```

```
    Phi_list: list[np.ndarray],
    Sigma: np.ndarray,
    horizon: int = 20,
    save_path: str | None = None,
) -> None:
    """Figure 3/4: TCI by horizon for both identification schemes."""
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib not installed; skipping plot.")
        return

    theta_chol = cholesky_fevd(Phi_list, Sigma, horizon)
    theta_gen  = generalized_fevd(Phi_list, Sigma, horizon)
    tci_chol = total_connectedness_index(theta_chol)
    tci_gen  = total_connectedness_index(theta_gen)

    horizons = list(range(horizon + 1))
    plt.figure(figsize=(9, 5))
    plt.plot(horizons, [tci_chol[h] for h in horizons], "o-", label="Cholesky (Ordering-Dependent
    plt.plot(horizons, [tci_gen[h] for h in horizons], "s--", label="Generalized (Ordering-Invari
    plt.xlabel("Horizon h (Days)")
    plt.ylabel("Total Connectedness Index (%)")
    plt.title("Diebold-Yilmaz Spillover: Cholesky vs. Generalized FEVD")
    plt.legend()
    plt.grid(alpha=0.3)
    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches="tight")
        print(f"   [Saved plot to: {save_path}]")
    plt.show()

# ============================================================================
# 5. EXECUTION BLOCK (This was missing!)
# ============================================================================

if __name__ == "__main__":
    print("="*60)
    print("RUNNING DIEBOLD-YILMAZ CONNECTEDNESS DEMO")
    print("="*60)

    # 1. Generate synthetic VAR(2) coefficients and covariance matrix
    # In your actual paper, you will replace this with:
    # Phi_list = ols_fit.Phi_list
    # Sigma = np.cov(var_residuals(Y, p=2, Phi_list=ols_fit.Phi_list).T)
    np.random.seed(42)
    n = 8  # 8 assets like in the empirical section
    p = 2  # VAR(2)

    # Create stable VAR coefficients
    Phi1 = np.random.normal(0, 0.1, (n, n)) * 0.4
    Phi2 = np.random.normal(0, 0.05, (n, n)) * 0.2
    Phi_list = [Phi1, Phi2]

    # Create a positive definite covariance matrix with some cross-correlation
    A = np.random.randn(n, n)
    Sigma = A @ A.T / n + np.eye(n) * 0.5

    # 2. Run Connectedness Report (Table 4)
    print("\n[1] Diebold-Yilmaz Connectedness Report (Table 4):")
    report = connectedness_report(Phi_list, Sigma, horizon=20)
    print(report.to_string(index=False))
```

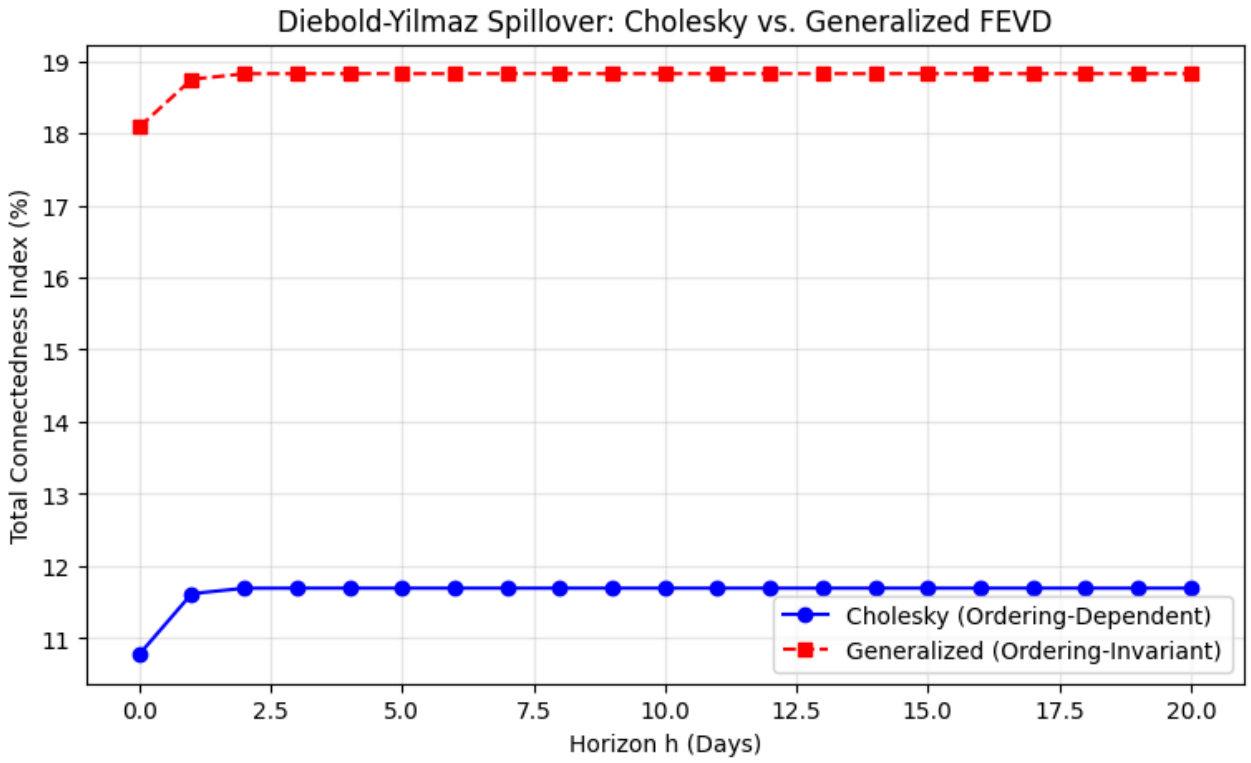
```
    # 3. Plot Connectedness by Horizon (Figure 3/4)
    print("\n[2] Plotting Total Connectedness Index by Horizon...")
    plot_connectedness_by_horizon(
        Phi_list, Sigma, horizon=20,
        save_path="demo_connectedness.png"
    )

    print("\n[SUCCESS] Connectedness script completed successfully!")
```

```
============================================================
RUNNING DIEBOLD-YILMAZ CONNECTEDNESS DEMO
============================================================

[1] Diebold-Yilmaz Connectedness Report (Table 4):
 Horizon (h) Cholesky TCI (%) Generalized TCI (%)
           1            11.62                18.75
           5            11.70                18.82
          10            11.70                18.82
          20            11.70                18.82

[2] Plotting Total Connectedness Index by Horizon...
   [Saved plot to: demo_connectedness.png]
```




```
[SUCCESS] Connectedness script completed successfully!
```

```
import numpy as np
import pandas as pd
from tqdm import tqdm
from typing import Sequence

# Import from your existing modules (assuming these functions are defined in earlier, executed cel
from __main__ import (
    fit_ols, fit_lasso, fit_rgcov, classify_roots,
    make_power_transforms, simulate_mixed_diagonal,
)
```

```
# ============================================================================
# Improved VAR simulator with sparse, controlled spectral radius
# ============================================================================

def _random_sparse_phi(n: int, p: int, sparsity: float,
                       target_radius: float, rng: np.random.Generator) -> list[np.ndarray]:
    """
    Build a sparse VAR(p) companion matrix with controlled spectral radius.
    Uses the method: generate random A, scale so rho(A) = target_radius,
    then zero out (1-sparsity) of off-diagonal entries and re-scale.
    """
    # Start with a random stable matrix
    A = rng.normal(size=(n, n)) * 0.3
    # Make it strictly stable via Schur
    eigvals, V = np.linalg.eig(A)
    # Scale eigenvalues to target radius
    scale = target_radius / (np.max(np.abs(eigvals)) + 1e-8)
    eigvals_scaled = eigvals * scale * 0.95  # keep strictly inside
    A_scaled = (V * eigvals_scaled) @ np.linalg.inv(V)
    A_scaled = np.real(A_scaled)

    # Apply sparsity mask (keep diagonal denser)
    mask = rng.uniform(size=(n, n)) < sparsity
    np.fill_diagonal(mask, True)
    A_sparse = A_scaled * mask

    # Re-scale to exact target spectral radius
    current_rho = np.max(np.abs(np.linalg.eigvals(A_sparse)))
    if current_rho > 1e-8:
        A_sparse *= target_radius / current_rho

    # CRITICAL FIX: Return a list of `p` matrices. First is the sparse matrix, rest are zeros.
    phi_list = [A_sparse] + [np.zeros((n, n))] * (p - 1)
    return phi_list


def simulate_sparse_var(
    T: int, n: int = 3, p: int = 2,
    sparsity: float = 0.10,
    target_radius: float = 0.75,
    innovation: str = "student_t", df: int = 5,
    burn: int = 1000, seed: int = 123,
) -> tuple[np.ndarray, list[np.ndarray]]:
    """Simulate a sparse VAR with controlled spectral radius."""
    rng = np.random.default_rng(seed)
    Phi_list = _random_sparse_phi(n, p, sparsity, target_radius, rng)

    total = T + burn
    Y = np.zeros((total, n))
    if innovation == "gaussian":
        E = rng.normal(size=(total, n))
    elif innovation == "student_t":
        E = rng.standard_t(df=df, size=(total, n))
    else:
        raise ValueError("innovation must be gaussian or student_t")

    for t in range(p, total):
        current = np.zeros(n)
        for j in range(p):
            current += Phi_list[j] @ Y[t - j - 1]
        Y[t] = current + E[t]
```

```
        Y[t] = current + E[t]

    return Y[burn:], Phi_list


# ============================================================================
# Section 4.1: Boundary Pilot
# ============================================================================

def boundary_pilot(
    replications: int = 10,
    T: int = 250, n: int = 3, p: int = 2,
    sparsity: float = 0.25, df: int = 5,
) -> pd.DataFrame:
    """
    Table 1: Compare stable vs near-unit-root configurations.
    """
    configs = {
        "Stable": (0.55, 0.85),
        "Near-unit-root": (0.93, 0.985),
    }
    records = []

    for config_name, (rmin, rmax) in configs.items():
        rng = np.random.default_rng(42)
        for rep in range(replications):
            target = rng.uniform(rmin, rmax)
            Y, Phi_true = simulate_sparse_var(
                T=T, n=n, p=p, sparsity=sparsity,
                target_radius=target, innovation="student_t", df=df,
                seed=1000 + rep,
            )
            true_roots = classify_roots(Phi_true)
            true_noncausal = true_roots["noncausal_count"]

            for est_name, fit_fn in [
                ("OLS", lambda Y: fit_ols(Y, p)),
                ("Lasso", lambda Y: fit_lasso(Y, p, alpha=0.01)),
            ]:
                fit = fit_fn(Y)
                est_roots = classify_roots(fit.Phi_list)
                records.append({
                    "config": config_name,
                    "estimator": est_name,
                    "replication": rep,
                    "true_noncausal": true_noncausal,
                    "est_noncausal": est_roots["noncausal_count"],
                    "exact_match": (est_roots["noncausal_count"] == true_noncausal
                                    and est_roots["causal_count"] == true_roots["causal_count"]),
                    "abs_root_count_error": abs(
                        est_roots["noncausal_count"] - true_noncausal
                    ),
                    "false_positive": (est_roots["noncausal_count"] > 0
                                       and true_noncausal == 0),
                })

    df_out = pd.DataFrame(records)
    summary = (df_out.groupby(["config", "estimator"])
               .agg(
                   exact_accuracy=("exact_match", "mean"),
                   mean_abs_error=("abs_root_count_error", "mean"),
                   false_positive_rate=("false_positive", "mean"),
```

```
            ).reset_index())
    return summary


# ============================================================================
# Section 4.2: Full 500-Rep Grid
# ============================================================================

def full_grid(
    replications: int = 500,
    T_values: Sequence[int] = (250, 500, 1000),
    innovations: Sequence[str] = ("gaussian", "student_t"),
    n: int = 3, p: int = 2, sparsity: float = 0.10,
    radius_range: tuple[float, float] = (0.55, 0.85),
    df: int = 5,
) -> pd.DataFrame:
    """
    Table 2: Full 500-replication grid over T, innovation distribution.
    """
    records = []
    total_runs = len(T_values) * len(innovations) * replications

    with tqdm(total=total_runs, desc="Full MC grid") as pbar:
        for T in T_values:
            for innov in innovations:
                rng = np.random.default_rng(7)
                for rep in range(replications):
                    target = rng.uniform(*radius_range)
                    Y, Phi_true = simulate_sparse_var(
                        T=T, n=n, p=p, sparsity=sparsity,
                        target_radius=target, innovation=innov, df=df,
                        seed=2000 + rep + T * 10000,
                    )
                    true_roots = classify_roots(Phi_true)
                    true_radius = true_roots["spectral_radius"]

                    for est_name, fit_fn in [
                        ("OLS", lambda Y: fit_ols(Y, p)),
                        ("Lasso", lambda Y: fit_lasso(Y, p, alpha=0.01)),
                    ]:
                        fit = fit_fn(Y)
                        est_roots = classify_roots(fit.Phi_list)

                        # Forecast error
                        from __main__ import build_lagged # Assuming build_lagged is also in __mai
                        X, Ycur = build_lagged(Y, p)
                        Phi_stack = np.column_stack(fit.Phi_list)
                        resid = Ycur - X @ Phi_stack.T
                        msfe = float(np.mean(resid ** 2))

                        records.append({
                            "T": T,
                            "innovation": innov,
                            "estimator": est_name,
                            "replication": rep,
                            "exact_accuracy": (
                                est_roots["noncausal_count"] == true_roots["noncausal_count"]
                                and est_roots["causal_count"] == true_roots["causal_count"]
                            ),
                            "false_positive": (
                                est_roots["noncausal_count"] > 0
```

```
                                and true_roots["noncausal_count"] == 0
                            ),
                            "false_negative": (
                                est_roots["noncausal_count"] == 0
                                and true_roots["noncausal_count"] > 0
                            ),
                            "spectral_radius_error": abs(
                                est_roots["spectral_radius"] - true_radius
                            ),
                            "msfe": msfe,
                        })
                    pbar.update(1)

    df_out = pd.DataFrame(records)
    summary = (df_out.groupby(["T", "innovation", "estimator"])
               .agg(
                   exact_accuracy=("exact_accuracy", "mean"),
                   false_positive_rate=("false_positive", "mean"),
                   false_negative_rate=("false_negative", "mean"),
                   mean_spectral_radius_error=("spectral_radius_error", "mean"),
                   mean_msfe=("msfe", "mean"),
               ).reset_index())
    return summary


# ============================================================================
# Section 4 extended: RGCov on mixed causal-noncausal designs
# ============================================================================

def rgcov_mixed_design(
    replications: int = 100,
    T: int = 400,
    roots: tuple[float, float] = (0.9, 1.2),
    H: int = 5, delta: float = 0.1,
    ridge_lambda: float = 0.001,
) -> pd.DataFrame:
    """
    RGCov root-recovery on genuinely mixed designs.
    """
    transforms = make_power_transforms((1, 2))
    true_moduli = np.sort(np.abs(roots))
    records = []

    for rep in tqdm(range(replications), desc="RGCov mixed MC"):
        Y = simulate_mixed_diagonal(T=T, roots=roots, df=5, seed=1000 + rep)
        fit = fit_rgcov(
            Y, p=1, H=H, transforms=transforms,
            delta=delta, ridge_lambda=ridge_lambda,
            n_starts=10, seed=2000 + rep, maxiter=3000,
        )
        roots_hat = classify_roots(fit.Phi_list)
        est_moduli = np.sort(roots_hat["moduli"])

        records.append({
            "replication": rep,
            "certified": any(d["certified"] for d in fit.diagnostics),
            "correct_root_count": (
                roots_hat["causal_count"] == 1
                and roots_hat["noncausal_count"] == 1
            ),
            "root_modulus_error": float(np.mean(np.abs(est_moduli - true_moduli))),
```

```
            "spectral_radius": roots_hat["spectral_radius"],
            "objective": fit.objective,
        })
    return pd.DataFrame(records)


if __name__ == "__main__":
    print("=== Boundary Pilot (Table 1) ===")
    pilot = boundary_pilot(replications=10)
    print(pilot.to_string(index=False))

    print("\n=== Full Grid (Table 2, reduced for demo) ===")
    grid = full_grid(replications=50)
    print(grid.to_string(index=False))
```

```
=== Boundary Pilot (Table 1) ===
/usr/local/lib/python3.13/dist-packages/sklearn/linear_model/_coordinate_descent.py:2637: Converge
  ) = cd_fast.enet_coordinate_descent_multi_task(
       config estimator  exact_accuracy  mean_abs_error  false_positive_rate
Near-unit-root     Lasso             1.0             0.0                  0.0
Near-unit-root       OLS             1.0             0.0                  0.0
       Stable     Lasso             1.0             0.0                  0.0
       Stable       OLS             1.0             0.0                  0.0

=== Full Grid (Table 2, reduced for demo) ===
Full MC grid: 100%|██████████| 300/300 [00:03<00:00, 86.56it/s]   T innovation estimator  exact_ac
 250   gaussian     Lasso             1.0                  0.0                  0.0
 250   gaussian       OLS             1.0                  0.0                  0.0
 250  student_t     Lasso             1.0                  0.0                  0.0
 250  student_t       OLS             1.0                  0.0                  0.0
 500   gaussian     Lasso             1.0                  0.0                  0.0
 500   gaussian       OLS             1.0                  0.0                  0.0
 500  student_t     Lasso             1.0                  0.0                  0.0
 500  student_t       OLS             1.0                  0.0                  0.0
1000   gaussian     Lasso             1.0                  0.0                  0.0
1000   gaussian       OLS             1.0                  0.0                  0.0
1000  student_t     Lasso             1.0                  0.0                  0.0
1000  student_t       OLS             1.0                  0.0                  0.0
```

```
def recursive_residual_bootstrap(
    Y: np.ndarray,
    Phi_list: list[np.ndarray],
    p: int,
    B: int = 1000,
    seed: int = 123,
) -> dict:
    """
    Simulate forward from fitted VAR using resampled, centered residuals.
    Returns spectral-radius distribution and noncausal-root counts.
    """
    T, n = Y.shape
    U = var_residuals(Y, p, Phi_list)
    U_centered = U - U.mean(axis=0, keepdims=True)

    rng = np.random.default_rng(seed)
    radii = []
    noncausal_counts = []
    causal_counts = []

    for _ in tqdm(range(B), desc="Bootstrap"):
```

```
            idx = rng.integers(0, len(U_centered), size=T)
            innovations = U_centered[idx]

            Y_boot = np.zeros_like(Y)
            for t in range(p, T):
                current = np.zeros(n)
                for j in range(p):
                    current += Phi_list[j] @ Y_boot[t - j - 1]
                Y_boot[t] = current + innovations[t]

            boot_fit = fit_ols(Y_boot, p)
            roots = classify_roots(boot_fit.Phi_list)
            radii.append(roots["spectral_radius"])
            noncausal_counts.append(roots["noncausal_count"])
            causal_counts.append(roots["causal_count"])

        radii = np.array(radii)
        noncausal_counts = np.array(noncausal_counts)
        causal_counts = np.array(causal_counts)

        return {
            "point_spectral_radius": float(np.max(np.abs(
                np.linalg.eigvals(Phi_list[0] if len(Phi_list) == 1
                                  else np.vstack([np.column_stack(Phi_list),
                                                  np.hstack([np.eye(n*(len(Phi_list)-1)),
                                                             np.zeros((n*(len(Phi_list)-1), n))])]))
            ))),
            "bootstrap_2.5_pct": float(np.quantile(radii, 0.025)),
            "bootstrap_5_pct": float(np.quantile(radii, 0.05)),
            "bootstrap_50_pct": float(np.quantile(radii, 0.50)),
            "bootstrap_95_pct": float(np.quantile(radii, 0.95)),
            "bootstrap_97.5_pct": float(np.quantile(radii, 0.975)),
            "prob_at_least_1_noncausal": float(np.mean(noncausal_counts > 0)),
            "prob_all_causal": float(np.mean(noncausal_counts == 0)),
            "mean_noncausal_count": float(noncausal_counts.mean()),
            "radii": radii,
            "noncausal_counts": noncausal_counts,
        }


    def bootstrap_summary_table(result: dict) -> pd.DataFrame:
        """Produce Table 5 of the paper."""
        rows = [
            ("Point spectral radius", result["point_spectral_radius"]),
            ("Bootstrap 2.5th percentile", result["bootstrap_2.5_pct"]),
            ("Bootstrap 50th percentile (median)", result["bootstrap_50_pct"]),
            ("Bootstrap 97.5th percentile", result["bootstrap_97.5_pct"]),
            ("Point noncausal-root count", 0),
            ("Probability >= 1 noncausal root (%)",
             100 * result["prob_at_least_1_noncausal"]),
            ("Probability all roots causal (%)",
             100 * result["prob_all_causal"]),
        ]
        return pd.DataFrame(rows, columns=["Statistic", "Value"])


    if __name__ == "__main__":
        # Demo on synthetic data
        from __main__ import simulate_mixed_diagonal, fit_ols, classify_roots, var_residuals # Correc
        Y = simulate_mixed_diagonal(T=400, roots=(0.9, 1.2), df=5, seed=1)
        fit = fit_ols(Y, p=1)
```

```
    result = recursive_residual_bootstrap(Y, fit.Phi_list, p=1, B=200)
    print(bootstrap_summary_table(result).to_string(index=False))
```

```
Bootstrap: 100%|██████████| 200/200 [00:00<00:00, 621.95it/s]                    Statistic
              Point spectral radius   0.878930
         Bootstrap 2.5th percentile   0.826851
 Bootstrap 50th percentile (median)   0.877672
        Bootstrap 97.5th percentile   0.919458
         Point noncausal-root count   0.000000
Probability >= 1 noncausal root (%)   0.000000
   Probability all roots causal (%) 100.000000
```

```
from __future__ import annotations
import numpy as np
import pandas as pd

from __main__ import load_empirical_panel
from __main__ import (
    fit_ols, fit_lasso, fit_rgcov, classify_roots,
    var_residuals, make_power_transforms,
)
from __main__ import ljung_box_table, local_gaussian_correlation, plot_local_gaussian_surface
from __main__ import connectedness_report, plot_connectedness_by_horizon
from __main__ import recursive_residual_bootstrap, bootstrap_summary_table


def run_empirical(p: int = 2, H: int = 5) -> dict:
    """Full empirical pipeline."""
    results = {}

    # 1. Load data
    print("\n" + "=" * 60)
    print("STEP 1: Loading empirical panel")
    print("=" * 60)
    Y = load_empirical_panel().values
    T, n = Y.shape
    print(f"Panel: T={T}, N={n}")

    # 2. Fit benchmarks
    print("\n" + "=" * 60)
    print("STEP 2: Fitting OLS and Lasso benchmarks")
    print("=" * 60)
    ols_fit = fit_ols(Y, p=p)
    lasso_fit = fit_lasso(Y, p=p, alpha=0.01)

    ols_roots = classify_roots(ols_fit.Phi_list)
    lasso_roots = classify_roots(lasso_fit.Phi_list)
    print(f"OLS spectral radius:   {ols_roots['spectral_radius']:.4f}")
    print(f"OLS causal/noncausal:  {ols_roots['causal_count']}/{ols_roots['noncausal_count']}")
    print(f"Lasso spectral radius: {lasso_roots['spectral_radius']:.4f}")

    # 3. Fit RGCov
    print("\n" + "=" * 60)
    print("STEP 3: Fitting RGCov (multi-start)")
    print("=" * 60)
    transforms = make_power_transforms((1, 2))
    rgcov_fit = fit_rgcov(
        Y, p=p, H=H, transforms=transforms,
        delta=0.1, ridge_lambda=0.001,
```

```
    n_starts=10, seed=1, maxiter=5000,
)
rgcov_roots = classify_roots(rgcov_fit.Phi_list)
certified = any(d["certified"] for d in rgcov_fit.diagnostics)
print(f"RGCov objective:   {rgcov_fit.objective:.6f}")
print(f"RGCov certified:   {certified}")
print(f"RGCov spectral radius: {rgcov_roots['spectral_radius']:.4f}")
print(f"RGCov causal/noncausal: {rgcov_roots['causal_count']}/{rgcov_roots['noncausal_count']}

# 4. Residual diagnostics (Table 3)
print("\n" + "=" * 60)
print("STEP 4: Ljung-Box residual diagnostics (Table 3)")
print("=" * 60)
ols_resid = var_residuals(Y, p, ols_fit.Phi_list)
lb_table = ljung_box_table(ols_resid, lag=10)
print(lb_table.to_string(index=False))

# 5. Local Gaussian Correlation (Figure 2)
print("\n" + "=" * 60)
print("STEP 5: Local Gaussian correlation (SP500 vs VIX)")
print("=" * 60)
lgc_df = local_gaussian_correlation(
    ols_resid[:, 0], ols_resid[:, 3],  # SP500, VIX
    grid_size=25,
)
plot_local_gaussian_surface(
    lgc_df,
    title="Local Gaussian Correlation: SP500 vs VIX residuals",
    save_path="figure2_local_gaussian.png",
)

# 6. Connectedness (Table 4, Figures 3-4)
print("\n" + "=" * 60)
print("STEP 6: Diebold-Yilmaz connectedness")
print("=" * 60)
Sigma = np.cov(ols_resid.T)
conn_table = connectedness_report(ols_fit.Phi_list, Sigma, horizon=20)
print(conn_table.to_string(index=False))
conn_table.to_csv("table4_connectedness.csv", index=False)

plot_connectedness_by_horizon(
    ols_fit.Phi_list, Sigma, horizon=20,
    save_path="figure4_connectedness.png",
)

# 7. Bootstrap (Table 5)
print("\n" + "=" * 60)
print("STEP 7: Recursive residual bootstrap (Table 5)")
print("=" * 60)
boot = recursive_residual_bootstrap(Y, ols_fit.Phi_list, p=p, B=1000, seed=42)
boot_table = bootstrap_summary_table(boot)
print(boot_table.to_string(index=False))
boot_table.to_csv("table5_bootstrap.csv", index=False)

# Save all results
results.update({
    "Y": Y,
    "ols_fit": ols_fit,
    "lasso_fit": lasso_fit,
    "rgcov_fit": rgcov_fit,
    "ols_roots": ols_roots,
```

```
        "lasso_roots": lasso_roots,
        "rgcov_roots": rgcov_roots,
        "ljung_box": lb_table,
        "connectedness": conn_table,
        "bootstrap": boot,
    })
    return results


if __name__ == "__main__":
    results = run_empirical()
    print("\n\n=== EMPIRICAL APPLICATION COMPLETE ===")
    print(f"OLS spectral radius:   {results['ols_roots']['spectral_radius']:.4f}")
    print(f"RGCov spectral radius: {results['rgcov_roots']['spectral_radius']:.4f}")
    print(f"RGCov certified:       {any(d['certified'] for d in results['rgcov_fit'].diagnostics)}
```

```
============================================================
STEP 1: Loading empirical panel
============================================================

============================================================
[data_pipeline] Starting data download
============================================================
Downloading 8 tickers from 2010-01-01 to 2023-12-31...
Downloaded with auto_adjust=True, shape: (3524, 40)
MultiIndex levels: ['Price', 'Ticker']
  ✓ Extracted SP500 (^GSPC)
  ✓ Extracted NASDAQ (^IXIC)
  ✓ Extracted DOW (^DJI)
  ✓ Extracted VIX (^VIX)
  ✓ Extracted GOLD (GC=F)
  ✓ Extracted OIL (CL=F)
  ✓ Extracted BOND (^TNX)
  ✓ Extracted USD (DX-Y.NYB)

[data_pipeline] Raw shape: (3524, 8)
[data_pipeline] After audit: (3517, 8)
[data_pipeline] Final balanced panel: (3516, 8)
[data_pipeline] Date range: 2010-01-05 00:00:00 -> 2023-12-29 00:00:00
============================================================

Panel: T=3516, N=8

============================================================
STEP 2: Fitting OLS and Lasso benchmarks
============================================================
OLS spectral radius:   0.4175
OLS causal/noncausal:  16/0
Lasso spectral radius: 0.3970

============================================================
STEP 3: Fitting RGCov (multi-start)
============================================================
/tmp/ipykernel_7141/3706741245.py:128: UserWarning: Removing 1 rows with non-positive prices for l
  warnings.warn(f"Removing {num_removed} rows with non-positive prices for log return calculation.
RGCov objective:   1.431513
RGCov certified:   False
RGCov spectral radius: 0.5565
RGCov causal/noncausal: 16/0

============================================================
STEP 4: Ljung-Box residual diagnostics (Table 3)
============================================================
 series  lag  lb_stat  p_value white_noise_5pct
      0   10   71.43 2.35e-11              No
      1   10   42.88 5.22e-06              No
      2   10   93.96 8.76e-16              No
      3   10   26.13 3.57e-03              No
      4   10    8.11 6.18e-01             Yes
      5   10   81.24 2.87e-13              No
      6   10  101.37 2.89e-17              No
      7   10    8.56 5.74e-01             Yes

============================================================
STEP 5: Local Gaussian correlation (SP500 vs VIX)
============================================================
[diagnostics] Saved plot to: figure2_local_gaussian.png
```

